Learning Inventory Control Policies with Evolution Strategies

across Lost-Sales, Dual-Sourcing, and Multi-Echelon Problems

Nima Manaf Willem van Jaarsveld
Rob Basten

School of Industrial Engineering, Eindhoven University of Technology, The Netherlands

June 21, 2026

Abstract

We study whether a single, generic learning recipe — evolution strategies optimizing compact, interpretable policies — can match or beat strong problem-specific heuristics and published deep reinforcement learning (DRL) across a range of classical inventory control problems. Using Covariance Matrix Adaptation Evolution Strategies (CMA-ES) we learn the parameters of small policy function approximators (linear maps, one-hidden-layer networks, and soft decision trees) for ten problems: lost sales, fixed-ordering-cost lost sales, dual sourcing, divergent multi-echelon control with special delivery, perishable inventory, general-network backorder, serial (Clark–Scarf) multi-echelon control, one-warehouse multi-retailer distribution, ameliorating inventory, and a production/assembly/distribution network. A recurring design lever is that the *action parameterization is part of the policy*: rather than fixing an external action grid, each policy carries its own decoder (a soft-gated direct or ordinal quantity, or soft-tree leaves) and, where a structured heuristic exists, learns inside that heuristic's coordinate system.

Across a 24-instance vanilla lost-sales surface, learned policies are instance-best in 22 of 24 cases, and in 47 of 48 on the 48-instance fixed-cost surface. On the six dual-sourcing instances of Gijsbrechts et al. [11] the learned soft tree matches the capped dual-index heuristic — itself a $\leq 0.11\%$ proxy for the bounded-DP optimum — on every instance, and thus sits well inside the band that the published A3C learner (0.51–1.85% gap) does not reach. On both reported multi-echelon settings a direct-level soft tree improves on the best in-environment constant base-stock by $14.7\% \pm 1.6\%$ and $12.0\% \pm 2.3\%$ over five optimizer seeds (the published A3C figures, 8.95%/12.09%, are cross-protocol context). On three further problems, a compact soft tree beats the best tuned base-stock for perishable inventory ($+1.166\% \pm 0.030\%$ and $+0.824\% \pm 0.065\%$ under FIFO and LIFO issuing, five-seed means), beats the published constant-base-stock benchmark of a general-network backorder system by over 20%, and matches the proven serial (Clark–Scarf) optimum. On the remaining networked rows we report comparator-specific verdicts rather than a single leaderboard claim: the one-warehouse multi-retailer policy robustly beats the best same-protocol tuned heuristic on the asymmetric three-retailer (+4.61%), high-CV ten-retailer (+7.07%), and the previously-only-tied strongly heterogeneous ten-retailer (+12.44%, five-seed mean, all seeds below the best heuristic) instances but does not robustly beat the published PPO scalar (it sits $\approx 3\%$ above PPO on the hardest row); the ameliorating-inventory result beats a purchase-only order-up-to heuristic while remaining a large gap-to-bound; and the production/assembly/distribution network is reported as learned-versus-own-heuristic research: a new residual base-stock-backbone action head robustly beats the environment's own pairwise base-stock on the mixed topology (-2.20%, five-seed mean, all seeds below the best heuristic), while on the serial and pure-assembly topologies a five-seed audit finds seed-noisy parity (mean $-2.2\% \pm 3.0\%$ and $+2.1\% \pm 5.0\%$): the earlier single-run beats there do not replicate across optimizer seeds. The hardest-instance multi-retailer geometry (a constant leaf) was found by a search over the decoder classes, scored by the same seed-robust heuristic comparison.

The policies carry tens to a few hundred parameters, are trained without per-problem hyperparameter tuning, and the reported single-state actions are validated against an independent rollout. The results suggest evolution strategies are a practical, portable, and interpretable alternative for learning inventory policies.

1 Introduction

Inventory management is a fundamental sequential decision problem in operations management: each period a controller decides how much to order so as to balance the cost of holding stock against the cost of shortages. Many such problems are naturally formulated as Markov decision processes (MDPs), but exact dynamic programming is tractable only for small instances because the state space grows with the lead time and the number of stocking locations. The literature has therefore relied either on problem-specific *heuristics* that exploit structural properties — base-stock, (s, S), (s, nQ), dual-index, and capped dual-index policies [32, 33] — or, more recently, on *deep reinforcement learning* (DRL), which represents the policy as a neural network and tunes it from simulated experience [11, 3]. Heuristics require specialized analysis and simplifying assumptions that limit their reach; DRL is generic but demands substantial compute and extensive hyperparameter tuning, and its networks — typically thousands to millions of parameters — are hard to interpret [11, 15].

This paper asks a different question. Rather than proposing a new method for one problem, we ask whether a single, lightweight learning recipe can serve as a *portable* alternative to bespoke heuristics across structurally different inventory problems, while staying small enough to interpret and cheap enough to retrain. The recipe is to optimize the parameters of a compact policy function approximator with Covariance Matrix Adaptation Evolution Strategies (CMA-ES), a gradient-free, population-based optimizer that has been shown to scale to neural-network policies [4, 7]. CMA-ES is attractive here for three reasons that hold across all ten problems we study. First, it optimizes the realized objective — the simulated long-run average cost — directly, with no value-function bootstrapping, reward shaping, or credit assignment to individual actions, so the same loop applies unchanged whether the action is a scalar order, a regular/expedite pair, or a pair of echelon order-up-to levels. Second, its built-in covariance adaptation supplies exploration automatically, removing the exploration-strategy design that DRL requires. Third, because it never differentiates the policy, it accommodates non-smooth, discretized, and clipped action maps that are awkward for backpropagation but natural for inventory controls.

A second, methodological observation runs through every problem we study: *the action parameterization is part of the policy, not a fixed property of the environment.* A learner that can only emit a raw scalar order is implicitly restricted to whatever that scalar can express; a learner whose decoder is shaped like the relevant structured heuristic — an ordinal quantity for lost sales, capped-dual-index coordinates for dual sourcing, direct order-up-to levels for multi-echelon — searches a far more useful action geometry. We treat this decoder choice as a first-class design dimension and show that it, rather than network size, drives several of the results, including the fixed-cost order/no-order decision and the multi-echelon operating region.

Our contributions are as follows.

1. **A cross-problem evolution-strategies result.** We show that CMA-ES-trained compact policies are competitive across ten inventory problems — lost sales, fixed-cost lost sales, dual sourcing, divergent and serial multi-echelon, general-network backorder, perishable inventory, one-warehouse multi-retailer distribution, ameliorating inventory, and a production/assembly/distribution network: they beat strong heuristics where the comparator is a

heuristic, match proven optima where it is optimal (the dual-sourcing capped dual-index proxy and the serial Clark–Scarf optimum), and clear the published A3C reference on dual sourcing while treating cross-protocol PPO rows as context — using one optimization loop and per-problem horizons of thousands to tens of thousands of periods, without per-problem hyperparameter tuning. Learned policies are instance-best in 22/24 vanilla and 47/48 fixed-cost lost-sales instances, match the capped dual-index optimal proxy on all six dual-sourcing instances, and improve on the best in-environment constant base-stock by $14.7\% \pm 1.6\%$ and $12.0\% \pm 2.3\%$ (five-seed means) on the two multi-echelon settings; on the three further benchmark problems a soft tree beats the best tuned base-stock for perishable inventory, beats the published benchmark of a general-network backorder system by over 20%, and matches the proven serial Clark–Scarf optimum. On the final three problems the same recipe now has deliberately narrower verdicts: on one-warehouse multi-retailer it gives seed-robust same-protocol beats of the best tuned heuristic on the asymmetric three-retailer, high-CV ten-retailer, and previously-only-tied strongly heterogeneous ten-retailer instances (+12.44%, five-seed mean) while not claiming a robust PPO beat; on ameliorating inventory it raises long-run average profit far above the best order-up-to purchase policy while reporting the gap to the perfect-information LP as a bound; and on the production/assembly/distribution network a new residual base-stock-backbone action head upgrades the former heuristic-match on the mixed topology to a seed-robust -2.20% beat, while the serial and pure-assembly topologies are honest seed-noisy parity under a five-seed audit ($-2.2\% \pm 3.0\%$ and $+2.1\% \pm 5.0\%$; the earlier single-run beats there do not replicate). The two hardest-instance geometries were found by a search over the decoder classes, scored by the same seed-robust heuristic comparison.

2. **Action parameterization as policy design.** We formalize and evaluate a family of policy-owned decoders (soft-gated direct quantity, soft-gated ordinal quantity, soft trees with linear leaves) and per-problem action geometries (capped-dual-index coordinates; direct-level vs. reduced-grid multi-echelon actions), and show that the decoder and its action geometry — rather than the backbone width — are the consequential design choice: with the right decoder a compact policy matches or beats the structured heuristics, including by reaching operating regions (such as the multi-echelon order-up-to levels) that a fixed action grid cannot express.

3. **Compact, interpretable, and verifiable policies.** The learned policies carry tens to a few hundred parameters — orders of magnitude fewer than published DRL networks — and we give a worked example mapping concrete states to orders for each decoder, with the reported single-state actions independently validated against an independent rollout.

The remainder of the paper is organized as follows. Section 2 reviews related work on learned inventory policies, evolution strategies, dual sourcing, and multi-echelon control. Section 3 presents the shared machinery: the policy interface (a learned map from state to valid action), the CMA-ES optimizer, and the evaluation protocol common to all problems. Sections 4–12 then treat each problem as a self-contained unit — lost sales and fixed-cost lost sales, dual sourcing, divergent multi-echelon control, perishable inventory, general-network backorder, serial multi-echelon control, one-warehouse multi-retailer distribution, ameliorating inventory, and a production/assembly/distribution network — each stating its formulation, its policy (input state, internals, output action), its instance-level experiment setup, and its results, with interpretation of where each decoder wins and where heuristics still lead. Section 13 draws managerial insights, and Section 14 concludes with limitations and future work.

2 Related work

Classical inventory problems are easily cast as MDPs but are typically intractable by exact dynamic programming because of the curse of dimensionality [37], motivating both analytical heuristics — from the optimal (s, S) structure of Scarf [33] and the multi-echelon echelon-base-stock policy of Clark and Scarf [34] onward — and, recently, learned policies [8, 2, 3]. A central idea in the DRL line is to represent the policy and/or value function with a neural network that maps states to a distribution over order quantities [3]. Deep Q-Networks were applied early to the beer game and to perishable inventory; A3C was used to demonstrate cross-problem flexibility on dual sourcing, lost sales, and one-warehouse multi-retailer control [11], and proximal policy optimization has since been adopted for joint ordering, asymmetric one-warehouse multi-retailer systems, and non-stationary demand [15]. These methods share a state–action–reward learning structure: rewards over a trajectory are attributed to individual actions, and network parameters are tuned to promote favorable actions. Their generality comes at the cost of heavy tuning and large, opaque networks.

Evolution strategies take a different route. CMA-ES is a gradient-free, population-based optimizer that adapts a full covariance matrix to search complex, high-dimensional, ill-conditioned landscapes [4]; although not originally a policy-learning method, it was shown to train neural-network control policies at scale as a competitive alternative to gradient-based DRL [7], and even linear policies found by simple random search can be competitive on hard control benchmarks [29]. Within inventory control, evolutionary methods have chiefly been used to tune the parameters of fixed-form heuristics rather than to learn generic policies; learning generic ordering policies with evolution strategies, and doing so across several structurally distinct problems, is the gap this paper addresses.

A second thread our methodology draws on treats the *action parameterization* — not only the network weights — as a design variable. In reinforcement learning more broadly, learned action representations [38], methods for large or structured discrete action spaces [39], deliberate action-space shaping [41], and residual policies that learn a correction around a known controller [40] all show that how a controller is allowed to act can matter as much as how it is trained; our policy-owned decoders and the residual base-stock-backbone head are instances of this idea specialized to inventory control. Inside inventory management the same instinct appears as embedding the structure of optimal policies into the learner: Maggiar et al. [27] build structure-informed networks that bake monotonicity and other analytic properties into a *gradient-based* deep RL policy over an overlapping problem set, and Madeka et al. [42] apply a generic deep RL controller to a large-scale periodic-review lost-sales system with stochastic vendor lead times and correlated demand. We share the premise that inventory structure belongs in the policy rather than being rediscovered by a large network, but instantiate it differently: we tune *compact*, policy-owned decoders — often in the coordinate system of the best known heuristic — with a single *gradient-free* recipe (CMA-ES) across ten families, reporting an explicit match/beat/context verdict (with gap-to-bound where no optimum is known) for each. The decoder idea is thus shared with this lineage; our specific contribution is the gradient-free, compact, breadth-and-protocol instantiation rather than a claim to have originated it.

The problems we study anchor our comparisons to well-developed literatures. For *lost sales*, myopic and base-stock heuristics and the capped base-stock policy are strong benchmarks [8, 9], and the fixed-ordering-cost variant adds (s, S), (s, nQ), and modified (s, S, q) comparators [1]. For *dual sourcing*, the structured policies are the single-index, dual-index [16], capped dual-index [36], and tailored base-surge heuristics [17]; Gijsbrechts et al. [11] benchmark A3C against these and against a bounded-DP optimum on small instances. For divergent *multi-echelon* control with partial lost sales, the relevant family includes the neuro-dynamic-programming study of Van Roy et al. [10],

the partial-lost-sales two-echelon model of Nahmias and Smith [12], multilocation approximations [13], the broad typology of de Kok et al. [14], the classical one-warehouse N-retailer distribution problem of Schwarz [35], and recent DRL for one-warehouse multi-retailer systems [15]. We use these as external benchmarks and otherwise train directly for each problem.

3 Policy representation, optimization, and evaluation

This section states the shared machinery used by every problem: the policy interface — a learned deterministic map from a problem's state to a valid action — the CMA-ES optimizer, and the evaluation protocol. Each problem's environment, its concrete policy internals, and its results are then developed in its own self-contained section (Sections 4–12).

3.1 The action parameterization is part of the policy

We treat each policy as a single deterministic map $\pi_{\boldsymbol{\theta}}$ whose *input is always the problem's state* $\mathbf{S}_t$ and whose *output is always a valid action* a_t for that problem, with everything in between owned by the policy and learned by CMA-ES. Concretely, $\pi_{\boldsymbol{\theta}}$ applies, in order: (i) its own divide-by-scale *normalization* of the raw state, $\mathbf{x} = \mathbf{S}_t/\kappa$ with a fixed per-policy scale $\kappa > 0$ (a policy hyperparameter); (ii) a *backbone* (a linear map, a one-hidden-layer network, or a soft tree) mapping $\mathbf{x}$ to latent outputs; (iii) a *decoder / action geometry* that turns those latent outputs into a candidate action — an integer order, a regular/expedite order pair, or a pair of echelon order-up-to levels — optionally living in a structured heuristic's coordinate system; and (iv) a *projection/rounding* step onto the problem's valid action set (nonnegative integer rounding, clipping to caps). The normalization scale, the backbone weights, the decoder, and the action geometry are all parameters of the policy, not fixed properties of the environment; in particular the map from latent outputs to a feasible action is learned, not imposed. Where a structured heuristic exists, we let the decoder live in that heuristic's coordinate system so the search explores a useful action geometry rather than an arbitrary raw-quantity space. This makes the decoder a first-class design dimension that each problem's Policy subsection instantiates by naming its concrete input state, internals, and valid-action output. Figure 1 summarizes this interface as a single pipeline: everything between the input state and the output action is the learned policy $\pi_{\boldsymbol{\theta}}$.

The two backbones and the generic soft-tree split/leaf mechanism are defined once here; the problem-specific decoders and action geometries are defined in each problem's Policy subsection. We write σ for the logistic function and $\mathrm{softplus}(z) = \ln(1 + e^z)$, and we denote nearest-nonnegative-integer rounding by $\lfloor z \rceil_{\geq 0} := \max\{0, \lfloor z \rceil\}$, where $\lfloor \cdot \rceil$ rounds to the nearest integer. The two backbones are a linear map $u_\theta(x) = Wx + b$ and a one-hidden-layer network $u_\theta(x) = V\phi(Ux + c) + e$ of width $H = 8$ with the self-normalizing SELU activation ϕ [6]; both are illustrated, side by side, in Figure 6.

Soft-tree split/leaf mechanism. A soft decision tree with oblique splits maps the normalized feature vector $x \in \mathbb{R}^d$ to leaf path-probabilities $\pi_\ell(x)$: each internal node n routes right with probability $\sigma(w_n^\top x + b_n)$ and left otherwise, and $\pi_\ell(x)$ is the product of the routing probabilities along the path to leaf ℓ, so $\sum_\ell \pi_\ell(x) = 1$. Each leaf emits a leaf output (a constant, a vector, or a softplus-affine quantity), and the tree's latent output is the path-probability mixture $\sum_\ell \pi_\ell(x)\,(\text{leaf output})_\ell$. We report depth-1 and depth-2 trees. This same machinery, with scalar softplus-affine leaves (lost sales, Section 4.2), constant leaves in capped-dual-index coordinates (dual sourcing, Section 5.2), or vector-valued order-up-to leaves (multi-echelon, Section 6.2), supplies the structured decoder in each problem. Figure 2 illustrates the depth-2 case.

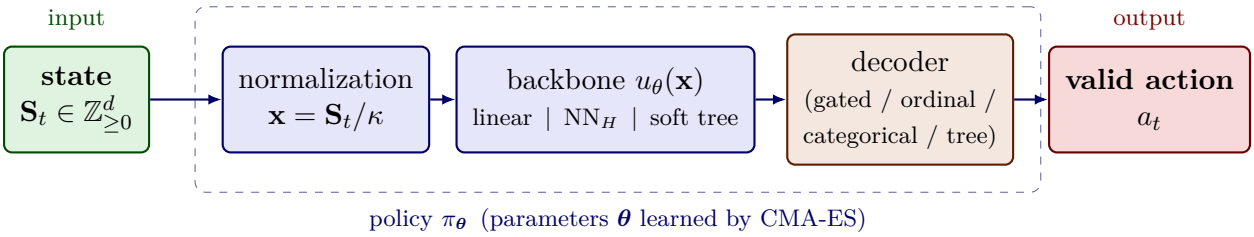


Figure 1: The policy interface as one pipeline. The input is always the problem's state $\mathbf{S}_t$ and the output is always a valid action a_t; in between, the policy applies its own divide-by-scale normalization $\mathbf{x} = \mathbf{S}_t/\kappa$, a backbone (linear map, one-hidden-layer network NN_H, or soft tree) producing latent outputs, a decoder / action geometry, and a projection onto the valid action set. The normalization scale κ, the backbone weights, the decoder, and the action geometry are all parameters of the policy $\pi_{\boldsymbol{\theta}}$ (dashed box), learned by CMA-ES, rather than fixed properties of the environment. The *categorical* decoder is shown only as the prior-version baseline representation this paper moves away from; no categorical results are reported here (see Figure 7).

3.2 Optimization with CMA-ES

We optimize the policy parameter vector $\boldsymbol{\theta}$ with Covariance Matrix Adaptation Evolution Strategies, a gradient-free, population-based optimizer [4, 7]. CMA-ES models the population distribution as a multivariate Gaussian $\mathbb{P}_\psi(\boldsymbol{\theta}) \sim \mathcal{N}(\boldsymbol{\mu}, \sigma^2\boldsymbol{\Sigma}) = \boldsymbol{\mu} + \sigma\,\mathcal{N}(0, \boldsymbol{\Sigma})$ and, at each iteration, samples a population, evaluates each member's simulated average cost, and updates $(\boldsymbol{\mu}, \sigma, \boldsymbol{\Sigma})$ from the best members (Algorithm 1). The mean is updated by weighted recombination of the N' best members,

$$\boldsymbol{\mu}_{i+1} = \boldsymbol{\mu}_i + \alpha_\mu \sum_{n'=1}^{N'} w^{n'} \left(\boldsymbol{\theta}^{n'}_{i+1} - \boldsymbol{\mu}_i\right), \qquad \sum_{n'} w^{n'} = 1,$$

the covariance is adapted by combined rank-one (evolution-path) and rank-μ updates with exponential smoothing, and the global step size σ is adapted online by cumulative step-length adaptation; we use the reference implementation of Hansen et al. [5]. Relative to A3C/PPO-style DRL, CMA-ES has few hyperparameters, supplies its own exploration through $\boldsymbol{\Sigma}$, and optimizes the realized objective directly, so the loop is unchanged across our ten problems. Figure 3 summarizes this generational loop.

Algorithm 1 CMA-ES policy optimization (used for all ten problems)

1: **Input:** population size N, initial mean $\boldsymbol{\mu}_1$, step size σ, simulator and policy decoder
2: **for** iteration $i = 1, 2, \ldots$ **do**
3: Sample $\boldsymbol{\theta}^{(n)}_i \sim \mathcal{N}(\boldsymbol{\mu}_i, \sigma_i^2\boldsymbol{\Sigma}_i)$ for $n = 1, \ldots, N$
4: Evaluate average cost $\hat{C}_T(\boldsymbol{\theta}^{(n)}_i)$ by rolling out the decoded policy in the simulator
5: Update $\boldsymbol{\mu}_{i+1}, \sigma_{i+1}, \boldsymbol{\Sigma}_{i+1}$ from the N' lowest-cost members (weighted recombination; rank-one and rank-μ covariance updates; cumulative step-length adaptation)
6: **end for**
7: **Output:** best evaluated policy parameters $\boldsymbol{\theta}^\star$

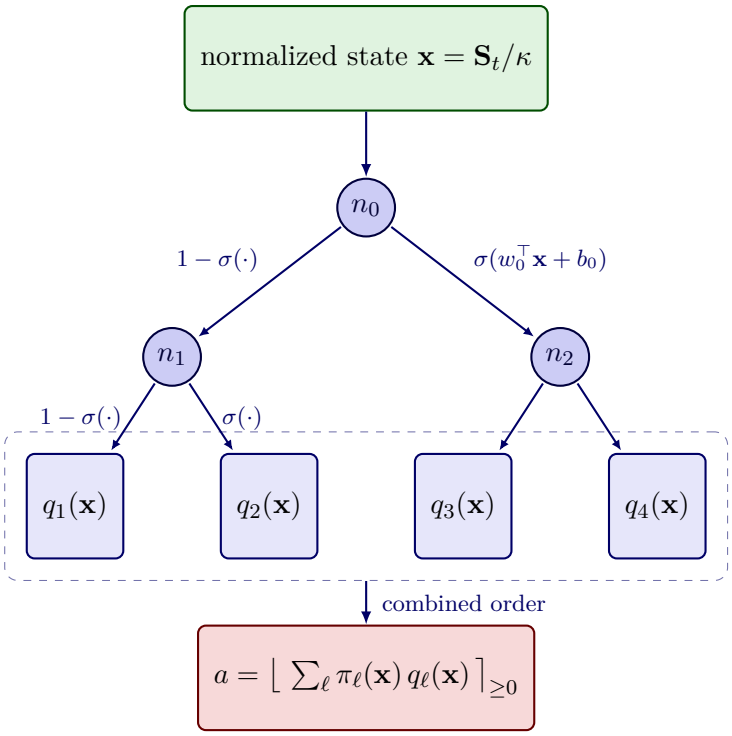


Figure 2: The soft decision tree (depth 2). The normalized state is routed *softly*: each internal node n sends probability $\sigma(w_n^\top \mathbf{x} + b_n)$ right and the remainder left, so leaf ℓ receives path probability $\pi_\ell(\mathbf{x})$ — the product of the routing probabilities along its root-to-leaf path, with $\sum_\ell \pi_\ell(\mathbf{x}) = 1$. The action is the rounded path-probability mixture of the leaf quantities, $a = \lfloor \sum_\ell \pi_\ell(\mathbf{x})\, q_\ell(\mathbf{x}) \rceil_{\geq 0}$.

Warm-starting (dual sourcing). When a strong static control exists in the policy's coordinate system, we warm-start CMA-ES at it. For dual sourcing the best static capped-dual-index control is near-optimal, so we initialize $\boldsymbol{\mu}_1$ at the encoded capped-dual-index solution and let CMA-ES refine around it; without the warm start the search occasionally lands in a worse basin. The improvements are thus a matter of action geometry and reliable optimization, not extra capacity.

Divergence handling. We run CMA-ES with a single, fixed configuration across every problem and do not tune it per instance, consistent with the lightly-tuned, portable recipe used throughout. Under this deliberately untuned setting a small fraction of runs occasionally diverge to a degenerate policy with a runaway cost. We flag a run as diverged when its evaluated long-run cost exceeds five times the instance's best heuristic baseline (or, on instances without such a baseline, five times the median learned cost), mark it with a dagger (†) in the result tables, and make the comparison over the remaining policies. These failures stem from the single untuned configuration rather than from the environment or the policy class: standard remedies — a smaller initial step size, CMA-ES restarts, or light per-instance tuning — would be expected to remove them, but we retain one configuration throughout to keep the recipe tuning-free.

3.3 Evaluation protocol

Policies are evaluated by simulation under protocols matched to each problem; the shared elements are stated once here and each problem's experiment setup states only its instance-specific details.

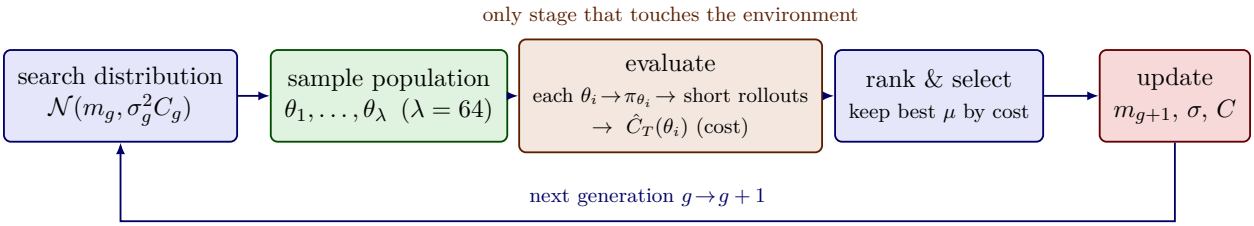


Figure 3: The CMA-ES training loop (one generation). A population of $\lambda = 64$ policy-parameter vectors is sampled from the search distribution $\mathcal{N}(m_g, \sigma_g^2 C_g)$; each is decoded into a policy and evaluated by short rollouts (the only stage that touches the environment), giving a simulated long-run cost $\hat{C}_T(\theta_i)$; the best μ members update the mean, step size, and covariance. The optimizer is gradient-free and uses only the *ranking* of the costs.

Training uses CMA-ES with a single fixed configuration — population 64 and a short training horizon of 2000 periods — with no per-instance tuning. Evaluation reports the long-run average cost over a long simulated horizon with a warm-up discard, averaged over multiple independent evaluation seeds; each problem's experiment setup states its own horizon and evaluation protocol. We hold every headline learned-policy result to one target — robustness to CMA-ES optimization randomness — established by whichever of two mechanisms a problem affords. For problems benchmarked over a large grid of heterogeneous instances — the vanilla and fixed-cost lost-sales surfaces of Section 4, with 24 and 48 instances — robustness comes from *breadth*: each cell is a single optimizer run, reported across the whole grid, so that a result sustained across dozens of independent instances cannot reflect one lucky seed, the optimizer randomness being independent from one instance to the next. For every other problem, which carries only a handful of instances and so offers no such breadth, robustness comes from *depth*: we report the mean ± standard deviation across *five* independent CMA-ES optimizer seeds — the cross-seed standard deviation (written with the subscript "seed"), distinct from the within-policy evaluation standard error — and state that all five seeds clear the comparator. Neither mechanism reports a best-of-N figure: the lost-sales cells are single runs rather than the best of several seeds, and the few-instance results are seed means rather than the best seed. Where the cost gaps between policies fall within the per-seed noise — as in dual sourcing, whose margins are sub-percent — we additionally adopt a common-random-number (CRN) protocol, evaluating every policy on the same demand-path seeds so that the dominant per-seed variance cancels and the paired cost difference becomes statistically meaningful. Each problem's reported single-state actions are independently validated against an independent rollout, with the cross-check detailed in that problem's section.[1]

4 Lost sales and fixed-cost lost sales

4.1 Problem formulation

Figures 4 and 5 depict the vanilla and fixed-ordering-cost environments as flow networks — the order request and its lead-time shipment, the demand draw, and where each cost is charged.

[1] *Reproducibility.* The code, the policy forwards, the worked-example script, and the instances used in the validation appendices (e.g. the canonical Poisson lost-sales instance ($p = 4$, $L = 4$) and the Bijvank et al. (2015) Table 1 instance ($L = 2$, $p = 14$, $K = 5$)) are available in our open-source repository, where the independent policy-forward implementations used for the cross-checks are also exposed.

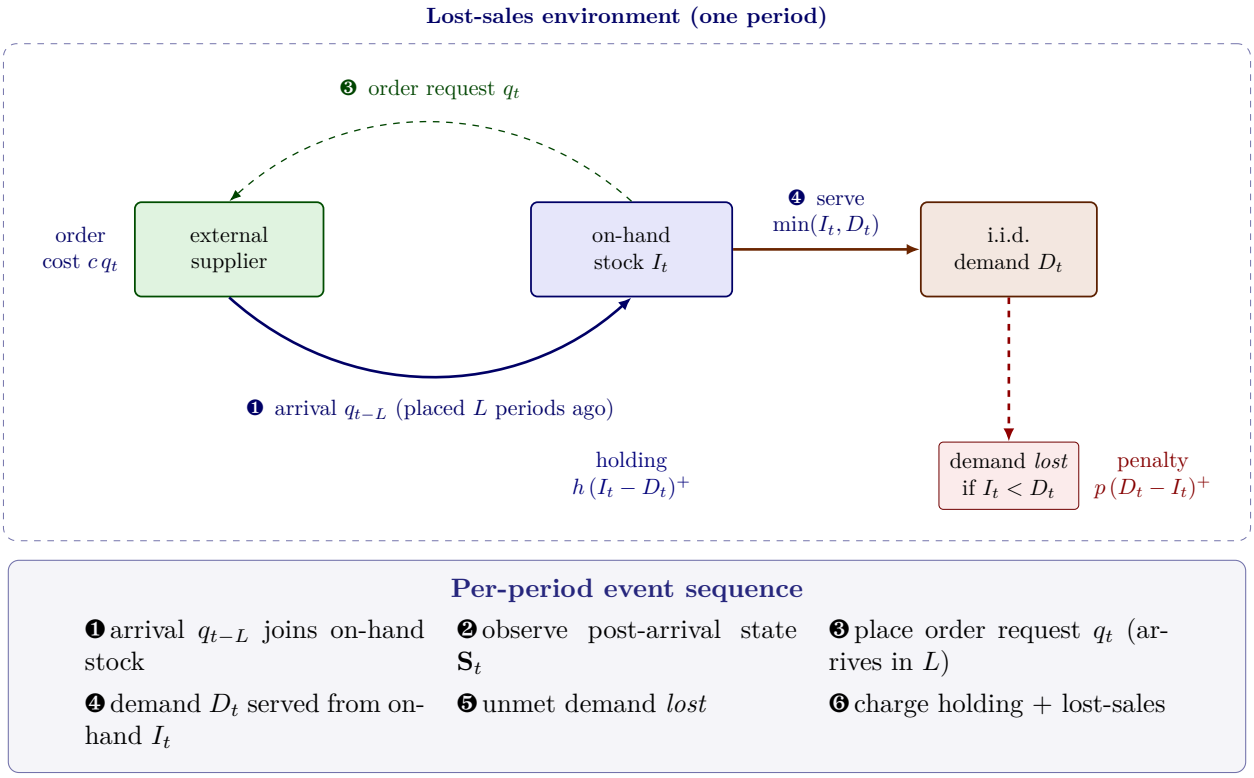


Figure 4: Vanilla lost-sales environment, drawn as an event-based interaction. Each period the stock point *sends an order request* q_t to the supplier (thin dashed control arrow), and the supplier *ships*; the goods *arrive* after the fixed lead time $L \geq 1$ (solid material arrow). i.i.d. demand D_t is then served from on-hand stock I_t and any *unmet demand is lost*. Holding cost h is charged on leftover on-hand inventory and a penalty p on each lost unit; the dashed box is the environment boundary and the boxed panel beneath gives the per-period event sequence (Section 4.1).

We study single-item, periodic-review lost-sales inventory control and its fixed-ordering-cost variant. Demand and order quantities are integer, and demand is stationary. The controller seeks a stationary ordering policy that minimizes the long-run average cost.

Timing of a period. Periods are indexed $t \in \{1, 2, \ldots, T\}$, and we are mainly interested in the limit $T \to \infty$. Within period t, events occur in the following order: (i) the order q_{t-L} placed L periods earlier arrives and is added to the leftover on-hand inventory, giving the on-hand stock x_t available to serve demand; (ii) observing the state, the controller places an integer replenishment order $q_t \in \mathbb{Z}_{\geq 0}$, which will arrive at the start of period $t + L$ after the fixed lead time $L \geq 1$; (iii) demand D_t is realized and served from on-hand stock, sales are limited to x_t and *unmet demand is lost* rather than backordered; (iv) holding and lost-sales costs are charged on the period's outcome.

State. Let $(\cdot)^+ = \max(\cdot, 0)$. Writing I_t for the on-hand stock right after the period-t arrival, $I_t = x_t$ is the inventory available to serve demand, and the still-outstanding orders are the pipeline $(q_{t-L+1}, \ldots, q_{t-1})$ of the $L - 1$ orders placed but not yet received. The post-arrival system state is the L-dimensional vector

$$\mathbf{S}_t = \big(I_t,\, q_{t-L+1}, \ldots, q_{t-1}\big) \in \mathbb{Z}_{\geq 0}^{L}, \tag{1}$$

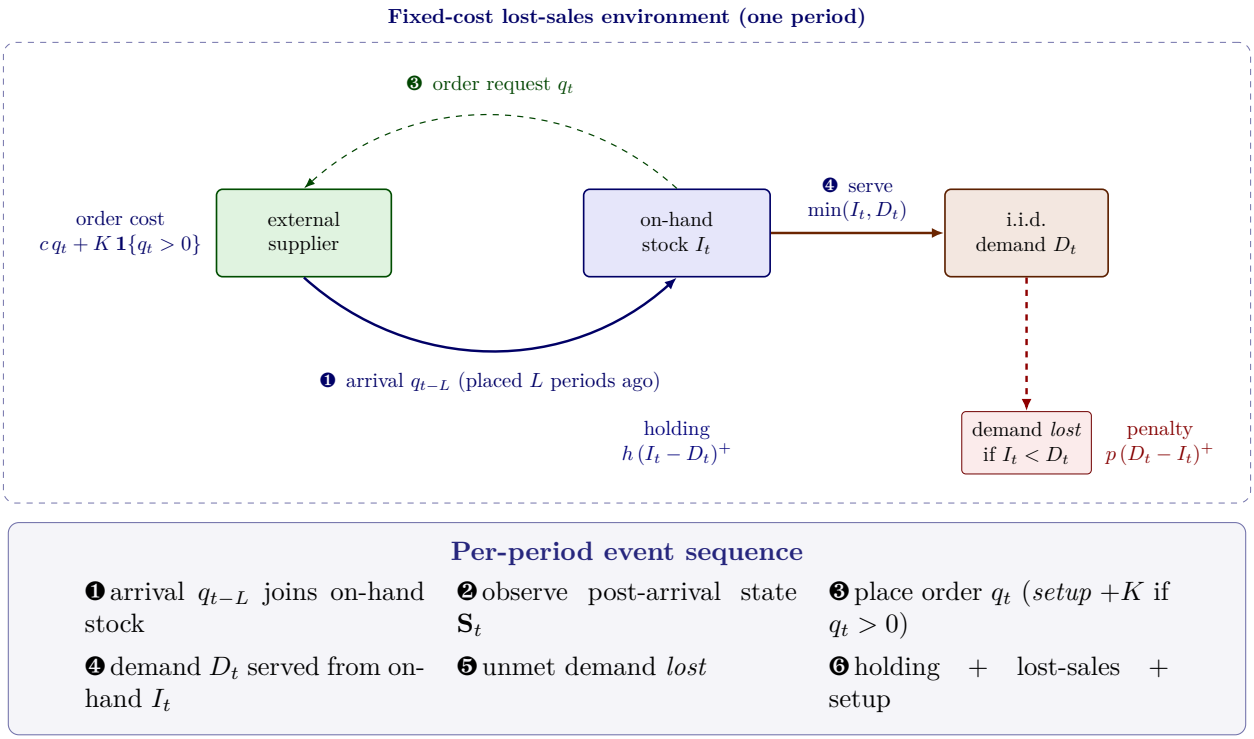


Figure 5: Fixed-cost lost-sales environment, drawn as an event-based interaction. Identical topology, request-then-ship ordering, and lost-sales rule to Figure 4; the only difference is that placing the order request additionally incurs a fixed setup cost K whenever a strictly positive order is requested (Section 4.1, Eq. (6)).

whose first coordinate is the on-hand inventory (the arriving order q_{t-L} has already been folded into it) and whose remaining $L-1$ coordinates are the outstanding pipeline. The cost in period t depends only on the first coordinate of $\mathbf{S}_t$, but I_t cannot be controlled directly: it arises as a complex function of the ordering policy through the dynamics below, and the state space grows exponentially in the lead time, which makes exact solution by policy or value iteration intractable and the problem a notoriously hard one [2].

Action. The decision in period t is the order quantity $q_t \in \{0, 1, \ldots, \bar{q}\}$, where $\bar{q}$ is a maximum order size (a loose cap, e.g. a provable bound on the optimal order or a high demand quantile). A stationary policy $\pi_{\boldsymbol{\theta}}$ maps the observed state to an order, $q_t = \pi_{\boldsymbol{\theta}}(\mathbf{S}_t)$.

Transition. Let D_t denote the demand in period t, with $D_t \sim D$ stationary across periods. The inventory on hand evolves as the leftover after demand plus the order that arrives one lead time later:

$$I_t = \big(I_{t-1} - D_{t-1}\big)^+ + q_{t-L}, \tag{2}$$

so the unsatisfied demand $(D_{t-1} - I_{t-1})^+$ is lost and never carried forward. The pipeline advances by dropping the just-arrived order and appending the new order: the state updates from $\mathbf{S}_t = (I_t, q_{t-L+1}, \ldots, q_{t-1})$ to

$$\mathbf{S}_{t+1} = \Big(\big(I_t - D_t\big)^+ + q_{t-L+1},\ \ q_{t-L+2}, \ldots, q_{t-1},\ q_t\Big). \tag{3}$$

One-period cost. A holding cost h is charged per unit of inventory left on hand at the end of the period and a penalty p per unit of lost sales. For realized demand D_t and on-hand stock I_t, the cost is

$$c_t = h\left(I_t - D_t\right)^+ + p\left(D_t - I_t\right)^+, \tag{4}$$

with expectation $\mathbb{E}[c_t] = h\,\mathbb{E}\big[(I_t - D_t)^+\big] + p\,\mathbb{E}\big[(D_t - I_t)^+\big]$ taken over the demand. There is no fixed cost in the vanilla problem.

Objective. Let $\bar{C}_T(\boldsymbol{\theta}) = \frac{1}{T}\sum_{t=1}^{T}\mathbb{E}[c_t]$ be the average expected cost over a horizon of T periods under policy $\pi_{\boldsymbol{\theta}}$. Because demand is stationary the whole problem is stationary, and the objective is to minimize the long-run average expected cost,

$$\min_{\boldsymbol{\theta}}\ \bar{C}(\boldsymbol{\theta}), \qquad \bar{C}(\boldsymbol{\theta}) = \lim_{T\to\infty}\bar{C}_T(\boldsymbol{\theta}). \tag{5}$$

This infinite-horizon average-cost criterion is the theoretical target; in practice we estimate it from a simulation rollout $\hat{C}_T(\boldsymbol{\theta}) = \frac{1}{T}\sum_{t=1}^{T} c_t$ over a long horizon with a warm-up discard, as detailed in Section 3.3.

Fixed-cost lost sales. The fixed-ordering-cost variant is identical in state (1), dynamics (2)–(3), and lost-sales rule; only the one-period cost gains a single setup term charged whenever a strictly positive order is placed,

$$c_t^K = c_t + K\,\mathbf{1}\{q_t > 0\}, \tag{6}$$

with fixed cost $K \geq 0$ and indicator $\mathbf{1}\{\cdot\}$, and the objective (5) is taken with c_t^K in place of c_t. This term makes the *order/no-order* decision — whether to replenish at all — an explicit part of the MDP: it is no longer optimal to order in every period, so an optimal policy must trade the setup cost against the holding and lost-sales costs, and it is precisely on this decision that the action parameterizations of Section 4.2 differ most.

4.2 Policy functions

This section instantiates the policy interface of Section 3.1 for lost sales and, because the lost-sales action is a single integer order, it is also where the decoder family is defined; the dual-sourcing and multi-echelon policies (Sections 5.2 and 6.2) reuse the soft-tree mechanism with their own action geometries.

Input. The policy input is this problem's state $\mathbf{S}_t$ of (1), the L-vector $[I_t, q_{t-L+1}, \ldots, q_{t-1}]$ of post-arrival on-hand inventory and the outstanding pipeline (feature dimension $d = L$). **Output.** A valid action is an integer order $q_t \in \{0, 1, \ldots, \bar{q}\}$. **Internals.** The policy first normalizes the raw state by the divide-by-scale map $\mathbf{x} = \mathbf{S}_t/\kappa$ with scale $\kappa = 20$ (the maximum order size $\bar{q}$); it then applies a backbone (the linear map, the one-hidden-layer network, or the soft-tree mechanism of Section 3.1; see Figure 6) to obtain latent outputs $\mathbf{u} = u_\theta(\mathbf{x})$; finally a decoder reads $\mathbf{u}$ and rounds to a nonnegative integer. We use three policy-owned decoders, each a clean map from the latent outputs to the order and each illustrated in Figure 7. Under a fixed ordering cost the two gated decoders expose the order/no-order boundary explicitly through their gate, which is what makes them the relevant choice for the fixed-cost variant.

Soft-gated direct quantity (L-SGD, NN-SGD). The backbone splits its latent output into a gate score $g(\mathbf{x}) \in \mathbb{R}$ and a quantity score $q(\mathbf{x}) \in \mathbb{R}$, and the decoder maps the pair to the gated softplus quantity

$$a = \Big\lfloor \underbrace{\sigma(g(\mathbf{x}))}_{\text{whether}} \cdot \underbrace{\text{softplus}(q(\mathbf{x}))}_{\text{how much}} \Big\rceil_{\geq 0}.$$

The logistic gate $\sigma(g) \in (0, 1)$ decides *whether* to order and the nonnegative softplus head decides *how much*; this factorization is what lets the policy switch ordering off cleanly, and it matters most under a fixed ordering cost.

Soft-gated ordinal quantity (L-SGO, NN-SGO). Here the backbone emits a gate score $g(\mathbf{x})$ together with M ordinal threshold scores $o_1(\mathbf{x}), \ldots, o_M(\mathbf{x})$, and the decoder gates a sum of independent threshold probabilities,

$$a = \Big\lfloor \sigma(g(\mathbf{x})) \sum_{k=1}^{M} \sigma(o_k(\mathbf{x})) \Big\rceil_{\geq 0}.$$

Each $\sigma(o_k) \in (0, 1)$ acts as a soft "one more unit" indicator, so the unrounded quantity moves in unit-scale increments and the threshold count M fixes both the largest representable order and the decoder's parameter count; we use $M = 20$ thresholds.

Soft trees with linear leaves (Tree-1, Tree-2). Using the soft-tree split/leaf mechanism of Section 3.1, each leaf ℓ emits a local softplus-affine quantity $q_\ell(\mathbf{x}) = \text{softplus}(\alpha_\ell^\top \mathbf{x} + \beta_\ell)$, and the decoder returns the rounded path-probability mixture of the leaf quantities,

$$a = \Big\lfloor \sum_{\ell} \pi_\ell(\mathbf{x})\, q_\ell(\mathbf{x}) \Big\rceil_{\geq 0},$$

where $\pi_\ell(\mathbf{x})$ are the routing path-probabilities of Section 3.1. The oblique splits let the tree partition the state into regimes and order a different affine quantity in each; we report depth-1 (Tree-1) and depth-2 (Tree-2) trees.

Parameter counts. For an $L = 4$ lost-sales pipeline state ($d = 4$, $d + 1 = 5$), each learned policy's trainable parameter count follows in closed form from its decoder. The linear and network direct-quantity policies carry $2(d+1) = 10$ and $H(d+1) + 2(H+1) = 58$ parameters; the depth-1 and depth-2 soft trees carry $3(d + 1) = 15$ and $7(d + 1) = 35$; and with $M = 20$ thresholds the ordinal policies carry $(M + 1)(d + 1) = 105$ (L-SGO) and $H(d + 1) + (M + 1)(H + 1) = 229$ (NN-SGO). These are orders of magnitude below the thousands-to-millions of parameters typical of DRL policies for the same problems [11, 15].

From state to order: a worked example. To make the decoders concrete we trace how each maps a fixed state to an order, using validated numbers reproduced by our released script (see the reproducibility note). We use the canonical Poisson-demand instance with $L = 4$, $h = 1$, $p = 4$, and Poisson mean 5. The policy input is the 4-vector $[I_{\text{eff}}, q_{t-3}, q_{t-2}, q_{t-1}]$: coordinate 0 is the effective inventory at decision time (the environment folds on-hand into the oldest in-transit slot) and coordinates 1–3 are the three still-outstanding pipeline orders; the policy normalizes by $\kappa = 20$ before acting and the action is the order quantity. Table 1 shows the order each decoder returns for two states, a near-empty system $[2, 0, 0, 0]$ and a well-stocked, full-pipeline system $[8, 5, 5, 5]$.

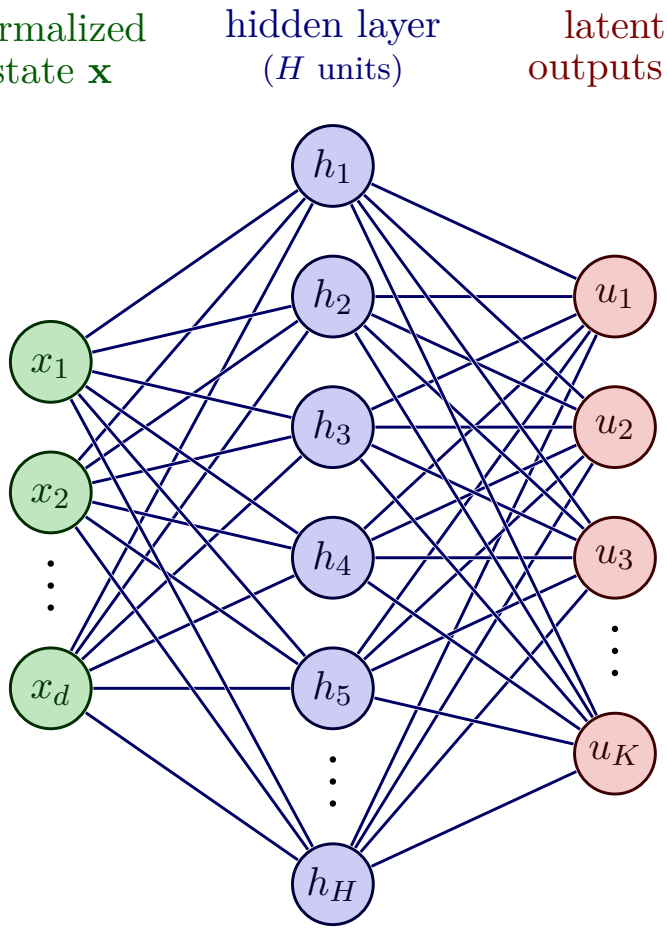


Figure 6: The two backbones. Both map the normalized state $\mathbf{x} = \mathbf{S}_t/\kappa$ to the latent outputs $\mathbf{u}$ that the decoder reads. The *linear* backbone is the single affine map $u_q = \sum_i x_i\, w_{q,i} + b_q$ (input layer connected directly to the latent outputs; the hidden layer is absent). The *one-hidden-layer network* (NN_H) inserts a hidden layer of width $H = 8$ with the SELU activation ϕ, $\mathbf{h} = \phi(U\mathbf{x} + c)$ and $\mathbf{u} = V\mathbf{h} + e$. Input nodes are green, hidden nodes blue, and latent-output nodes red.

Table 1: Worked example: order returned by each learned lost-sales decoder for two states of the Poisson-demand instance with $L = 4$, $h = 1$, $p = 4$, and Poisson mean 5. State coordinate 0 is effective inventory; coordinates 1–3 are the outstanding pipeline. Values are reproduced by our released script (see the reproducibility note) and, for the soft trees, equal the independent policy forward by construction.

Decoder	State $[2, 0, 0, 0]$	State $[8, 5, 5, 5]$
Tree-1 (depth-1 linear leaf)	11	3
Tree-2 (depth-2 linear leaf)	14	3
L-SGD (linear soft-gated direct)	12	3
NN-SGD (network soft-gated direct)	6	4
L-SGO (linear soft-gated ordinal)	7	4
NN-SGO (network soft-gated ordinal)	6	4

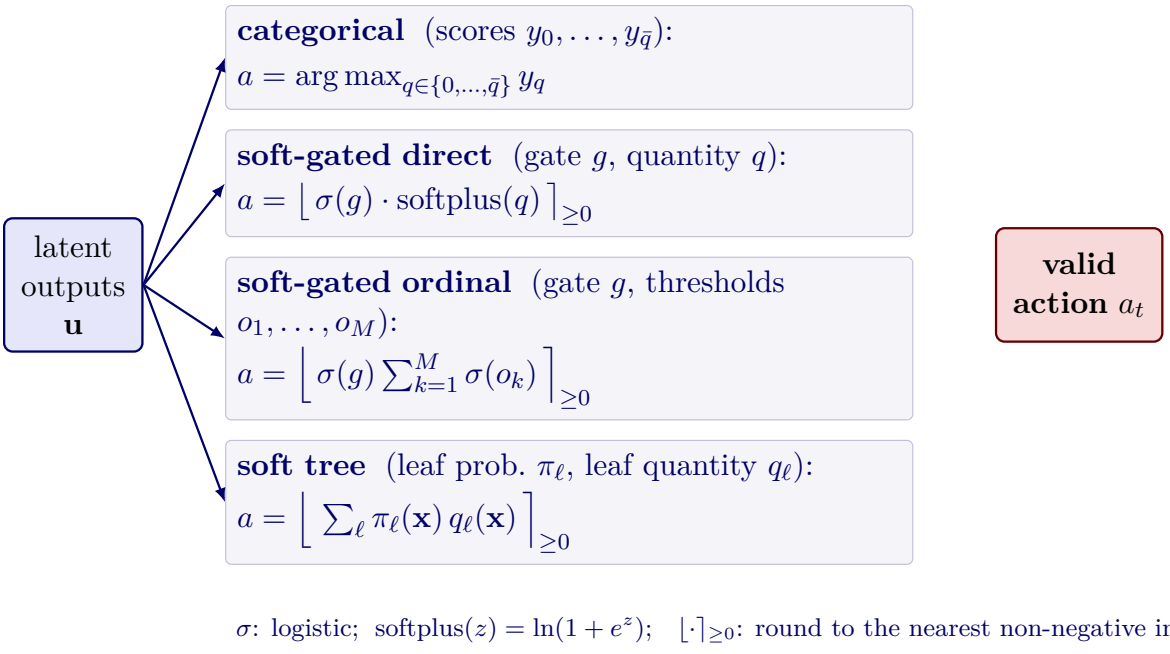


Figure 7: The decoder heads, each a map from the latent outputs $\mathbf{u}$ to an integer order a. The *categorical* head is the classical score-per-order-size representation of the prior version (one score y_q per candidate order, action chosen by arg max); the three heads used in this paper are the *soft-gated direct* quantity (L-SGD, NN-SGD), the *soft-gated ordinal* quantity (L-SGO, NN-SGO), and the *soft tree* with softplus-affine linear leaves (Tree-1, Tree-2). All four land in the same valid action set $\{0, \ldots, \bar{q}\}$ after the round-to-nonnegative-integer projection $\lfloor \cdot \rceil_{\geq 0}$.

The arithmetic behind two representative cells illustrates the decoders. For state $[2, 0, 0, 0]$ the linear soft-gated direct policy (L-SGD) opens its gate, $\sigma(7.435) = 0.9994$, and emits $\mathrm{softplus}(11.888) = 11.888$; the product is 11.881 and the decoder returns $\lfloor 11.881 \rceil_{\geq 0} = 12$. The depth-2 soft tree routes the same state almost entirely to one leaf (path probabilities $[0, 0, 1.0, 0]$) whose linear-leaf quantity is 14.048, giving the order $\lfloor 14.048 \rceil_{\geq 0} = 14$; this value equals the independent policy forward at that state by construction. For the well-stocked state $[8, 5, 5, 5]$ every decoder backs off to a near-replenishment order of 3 or 4, as expected when the pipeline already covers near-term demand.

Validation. All six lost-sales policies were validated by a full rollout cost match between two independent implementations on a fixed Poisson(5) demand sequence of 100,000 periods; every absolute cost difference was exactly 0.000 (well under the 0.05 tolerance), confirming that the divide-by-scale normalization is applied identically in both implementations, and no reported action was withheld. The lost-sales soft-tree single-state actions equal the independent policy forward by construction, and the two independent implementations agree at each reported state.

4.3 Experiment setup and heuristic comparators

The learned columns are the six policies of Section 4.2 (L-SGD, NN-SGD, L-SGO, NN-SGO, Tree-1, Tree-2); we describe the problem-specific heuristic comparators here.

The vanilla heuristic columns are Myopic-1 (M1), Myopic-2 (M2), and the standard vector base-stock policy (SVBS). M1 minimizes the lead-time-deep myopic action value with no future ordering;

M2 adds one discounted continuation step using the next-period M1 action; SVBS computes vector base-stock levels over the remaining lead-time positions and orders the minimum raise implied by those levels. The fixed-cost heuristic columns use the inventory position

$$P_t = x_t + \sum_{j=1}^{L-1} q_{t-j}.$$

The (s, S) policy orders $S - P_t$ when $P_t \le s$ and otherwise orders zero. The (s, nQ) policy orders the smallest positive multiple nQ that raises $P_t + nQ$ above s, again only when $P_t \le s$. The modified (s, S, q) policy orders $\min\{q, S - P_t\}$ when $P_t \le s$ and zero otherwise. Thus the setup-cost comparators expose their order/no-order threshold and their own batching or capped-raise parameters as part of the policy definition. The learned-policy parameter counts for the $L = 4$ pipeline state are given in Section 4.2.

Vanilla instances. We benchmark on a multi-instance surface rather than a single example. The vanilla lost-sales surface has holding cost $h = 1$, mean demand $\mu = 5$, lead times $L \in \{4, 6, 8, 10\}$, lost-sales penalties $p \in \{4, 19\}$, and the three demand families below, giving $3 \times 4 \times 2 = 24$ instances.

Fixed-cost instances. The fixed-cost problem uses the same vanilla surface and adds the two setup costs $K \in \{5, 25\}$, giving $2 \times 3 \times 4 \times 2 = 48$ fixed-cost instances.

Demand families. The three reported families are mean-preserving, all with $\mathbb{E}[D_t] = 5$:

- Poisson demand with mean 5,
- Geometric demand with mean 5,
- positively correlated two-state Markov-modulated Poisson (MMPP2+) demand with $(\lambda_{\text{low}}, \lambda_{\text{high}}) = (3, 7)$ and $p_{00} = p_{11} = 0.9$.

For the MMPP2+ case the latent regime $S_t \in \{\text{low}, \text{high}\}$ follows a two-state Markov chain with stay-probabilities p_{00}, p_{11}, and $D_t \mid S_t \sim \text{Poisson}(\lambda_{S_t})$. Because $p_{00} = p_{11}$ the stationary regime probabilities are $1/2$ each, preserving the mean $\frac{1}{2}(3) + \frac{1}{2}(7) = 5$. The temporal dependence is set by the regime eigenvalue $\rho_{\text{regime}} = p_{00} + p_{11} - 1 = 0.8$; after Poisson observation noise the implemented lag-one demand autocorrelation is

$$\text{Corr}(D_t, D_{t+1}) = \frac{\pi_{\text{low}}\pi_{\text{high}}(\lambda_{\text{high}} - \lambda_{\text{low}})^2(p_{00} + p_{11} - 1)}{\text{Var}(D_t)} = \frac{3.2}{9} \approx 0.356,$$

whereas the Poisson and Geometric families are i.i.d. (zero autocorrelation at every lag).

Training and evaluation. We use the lost-sales protocol of Section 3.3: CMA-ES training (population 64, horizon 2000) and evaluation of the long-run average cost over horizon 10^6 with 10 consecutive seeds. The benchmark heuristics are the myopic-1, myopic-2, and standard vector base-stock (SVBS) policies for vanilla lost sales, and the (s, S), (s, nQ), and modified (s, S, q) policies for the fixed-cost variant, using literature-reported parameters where available [8, 1]. Each cell of the surface is a single CMA-ES *optimizer* run — the 10 evaluation seeds above average only the demand-path noise of a fixed policy, not the optimization. Following the breadth protocol of Section 3.3, robustness on this problem is read from the consistency of the learned-versus-heuristic verdict across the 24- and 48-instance grids, the per-cell optimizer-seed averaging being reserved for the few-instance problems that lack such breadth.

4.4 Results

Tables 2 and 3 report the mean cost of the six learned policy approximators against the heuristic baselines across the reported vanilla and fixed-cost benchmark surfaces. Both tables use the same layout: policy rows, lead-time column groups, and demand-family subcolumns. A dagger marks a diverged CMA-ES run (excluded from the comparison); bold marks the best reported heuristic or learned policy in each instance column. We report the two sub-problems in turn.

4.4.1 Vanilla lost sales

Table 2 reports the mean cost of the six learned policies against the myopic-1 (M1), myopic-2 (M2), and standard vector base-stock (SVBS) heuristics across the 24-instance vanilla surface.

Table 2: Vanilla lost-sales benchmark: average cost by policy, shortage cost, lead time, and demand family across the reported 24-instance surface.

		Lead time $L = 4$			Lead time $L = 6$			Lead time $L = 8$			Lead time $L = 10$		
Shortage p	Policy	Pois	Geom	MMPP2+	Pois	Geom	MMPP2+	Pois	Geom	MMPP2+	Pois	Geom	MMPP2+
4	Optimal	4.730	10.610										
	M1	5.065	11.217	9.823	5.422	11.650	11.189	5.698	11.924	12.256	5.890	12.075	13.136
	M2	4.819	10.800	9.452	5.051	11.080	10.591	5.199	11.270	11.460	5.310	11.400	12.155
	SVBS	5.836	13.728	10.312	6.781	15.738	12.271	7.694	17.632	13.995	8.038	19.421	15.190
	L-SGD	4.750	10.645	8.571	4.897	10.790	9.171	**4.974**	**10.850**	9.445	5.048	10.921	9.563
	NN-SGD	4.774	10.687	8.565	4.900	10.791	9.643	5.033	10.983	9.674	5.066	10.993	9.697
	L-SGO	4.768	10.647	**8.549**	**4.893**	10.823	**9.109**	†	10.884	**9.443**	16.978	10.991	9.725
	NN-SGO	4.786	10.692	8.577	5.064	10.799	9.183	4.978	†	9.481	†	10.995	9.726
	Tree-1	**4.749**	**10.621**	8.579	4.895	**10.776**	9.159	4.975	10.857	9.445	**5.013**	10.934	9.590
	Tree-2	4.750	10.670	8.595	4.936	10.820	9.302	4.977	10.892	9.446	5.067	**10.912**	**9.547**
19	Optimal	8.890	22.950										
	M1	9.173	23.836	17.182	10.269	25.821	20.129	11.092	27.407	22.399	11.820	28.672	24.218
	M2	8.953	23.280	17.644	9.831	24.930	20.474	10.511	26.210	22.552	11.090	27.270	**23.913**
	SVBS	9.565	26.429	16.531	11.234	30.103	19.677	12.333	33.008	**22.173**	13.336	35.761	24.457
	L-SGD	8.932	**22.993**	16.686	9.759	24.486	19.862	10.261	25.374	22.378	10.634	26.287	24.197
	NN-SGD	**8.927**	23.455	**16.341**	10.131	24.483	19.650	10.430	25.757	22.211	10.772	26.129	24.406
	L-SGO	9.003	23.240	16.491	9.794	24.370	**19.549**	10.330	25.766	22.233	†	26.171	24.345
	NN-SGO	8.974	23.140	16.435	9.758	24.474	19.693	10.490	25.662	22.303	10.733	**26.037**	24.357
	Tree-1	8.936	23.011	16.697	**9.711**	**24.349**	19.915	**10.259**	**25.209**	22.339	**10.629**	26.064	24.101
	Tree-2	8.941	23.043	16.683	9.720	24.430	19.864	10.272	25.382	22.301	10.698	26.174	24.193

Optimal entries are shown only for verified literature optimum anchors; blanks mean no true optimum anchor is available for that expanded instance. †: diverged CMA-ES run (excluded). Bold: best reported heuristic or learned policy in the instance column.

The column winners make the comparison easy to read. On the vanilla surface, learned policies are best in 22 of 24 instances; Tree-1 is the most frequent winner (9 instances), followed by L-SGO (5 instances). The two instances still won by classical baselines are the high-penalty MMPP2+ cases with $L = 8, 10$.

Interpretation. On the vanilla surface, where the action is a plain quantity, the soft trees (Tree-1, Tree-2) and the linear ordinal decoder (L-SGO) dominate because a mild state-dependent raise is all that is needed and the smaller decoders optimize more reliably; the extra capacity of the network decoders does not pay for itself. The two vanilla instances still won by a classical baseline are the high-penalty ($p = 19$) MMPP2+ cases at the longest lead times $L = 8$ and $L = 10$, where SVBS ($L = 8$) and Myopic-2 ($L = 10$) edge the learned policies; here the combination of a regime-switching, autocorrelated demand and a deep pipeline is the hardest setting for a stationary compact policy, and closing this gap is an open item we return to in Section 13.

4.4.2 Fixed-cost lost sales

Table 3 repeats the vanilla instance surface and adds the two setup costs $K \in \{5, 25\}$, giving 48 instances, and replaces the vanilla heuristics with the (s, S), (s, nQ), and modified (s, S, q) comparators.

Table 3: Fixed-order-cost lost-sales benchmark: average cost by policy, setup cost, shortage cost, lead time, and demand family across the reported 48-instance surface.

			Lead time $L = 4$			Lead time $L = 6$			Lead time $L = 8$			Lead time $L = 10$		
Setup K	Shortage p	Policy	Pois	Geom	MMPP2+	Pois	Geom	MMPP2+	Pois	Geom	MMPP2+	Pois	Geom	MMPP2+
5	4	(s, S)	9.369	14.130	12.158	9.772	14.697		9.965	14.839		10.015	15.024	
		(s, nQ)	9.211	13.968	12.113	9.385	14.177		9.569	14.389		9.600	14.542	
		(s, S, q)	9.202	13.939	12.077	9.380	14.299		9.562	14.426		9.826	14.524	
		L-SGD	8.784	14.069	12.314	9.558	13.933	12.681	9.588	14.136	**12.722**	8.934	14.023	12.818
		NN-SGD	**8.733**	13.649	13.655	**8.871**	**13.825**	12.854	†	13.910	12.940	8.933	**13.861**	12.816
		L-SGO	8.784	13.755	**11.807**	9.009	13.876	12.613	8.993	**13.847**	12.751	8.943	13.942	12.808
		NN-SGO	8.748	**13.637**	11.831	8.871	13.841	**12.363**	8.909	14.144	13.010	9.153	13.866	12.817
		Tree-1	8.771	14.057	12.146	8.872	13.933	12.691	**8.905**	14.134	12.770	**8.931**	13.871	**12.791**
		Tree-2	8.783	13.751	11.982	9.560	13.837	12.665	8.954	13.865	12.735	8.934	14.019	12.899
5	19	(s, S)	13.889	26.888		14.841	28.669		15.816	31.221		21.667	35.693	
		(s, nQ)	13.657	26.878		14.694	28.872		15.301	29.870		19.152	33.406	
		(s, S, q)	13.560	26.835		14.476	28.543		15.618	31.067		21.529	35.603	
		L-SGD	13.619	27.260	21.350	14.265	28.195	24.567	14.704	28.977	27.001	15.151	29.576	28.632
		NN-SGD	13.590	**26.531**	**20.079**	†	27.941	23.306	14.770	†	25.924	**15.030**	29.822	**28.042**
		L-SGO	13.574	26.696	20.172	14.417	28.243	**23.255**	†	29.046	26.071	15.780	30.219	28.241
		NN-SGO	**13.475**	26.602	20.208	**14.167**	**27.859**	23.542	14.788	**28.936**	**25.805**	†	29.879	28.079
		Tree-1	13.628	27.071	21.309	14.268	28.250	24.552	14.701	28.957	27.087	15.108	**29.517**	28.673
		Tree-2	13.613	27.078	21.325	14.260	28.177	24.642	**14.700**	28.962	†	15.044	29.615	28.847
25	4	(s, S)	16.002	18.428	17.342	16.136	18.736		16.331	18.925		16.562	19.036	
		(s, nQ)	15.840	**18.231**	17.351	15.979	18.579		15.837	18.649		16.020	18.872	
		(s, S, q)	15.877	**18.231**	17.318	16.060	18.579		15.806	18.648		16.331	18.890	
		L-SGD	16.645	19.992	19.995	**15.812**	19.992	19.995	16.988	19.992	19.995	17.491	19.992	18.043
		NN-SGD	**15.572**	19.992	19.995	17.075	19.992	19.995	17.018	19.992	19.995	17.451	19.992	17.946
		L-SGO	17.983	19.992	19.995	15.847	19.992	19.995	15.810	18.509	19.995	**15.651**	19.992	**17.786**
		NN-SGO	15.627	19.992	**17.279**	†	**18.424**	**17.840**	**15.656**	**18.431**	**17.753**	16.271	**18.633**	17.831
		Tree-1	20.004	19.992	19.995	18.321	19.992	19.995	19.497	19.992	19.995	17.864	19.992	19.995
		Tree-2	16.203	19.992	19.995	18.167	19.992	19.995	16.632	19.992	19.995	19.710	19.992	19.995
25	19	(s, S)	21.104	32.726		22.073	35.015		25.969	38.859		33.668	43.651	
		(s, nQ)	21.016	32.946		22.283	34.586		22.664	35.812		25.247	38.055	
		(s, S, q)	21.016	32.772		22.373	35.116		25.971	38.865		33.995	43.652	
		L-SGD	21.166	32.668	†	22.878	34.569	33.398	24.641	35.287	35.916	23.727	36.354	36.525
		NN-SGD	20.845	32.645	27.434	21.709	**34.021**	**30.199**	**22.345**	35.389	32.862	**22.772**	†	34.924
		L-SGO	21.672	32.937	**27.386**	22.447	34.256	31.004	24.816	36.025	33.133	25.213	36.400	36.811
		NN-SGO	20.854	**32.435**	27.423	**21.598**	34.185	30.495	22.752	**35.070**	**32.537**	23.061	**36.005**	**34.373**
		Tree-1	21.173	32.640	28.727	21.914	35.108	33.330	22.654	35.884	35.822	23.750	36.241	37.530
		Tree-2	**20.770**	33.081	28.216	22.243	34.425	33.603	22.622	35.164	32.999	22.977	36.158	36.324

Blank heuristic cells indicate no literature/catalog heuristic value for that MMPP2+ instance. †: diverged CMA-ES run (excluded). Bold: best reported heuristic or learned policy in the instance column.

On the fixed-cost surface, learned policies are instance-best among the available policies in 47 of 48 instances. NN-SGO is the most frequent winner (20 instances), followed by NN-SGD (13 instances); the only heuristic instance win is Geometric $L = 4, p = 4, K = 25$, where (s, nQ) and modified (s, S, q) tie. Fixed-cost lost sales is therefore the sharper architecture test: the fixed charge makes the order/no-order boundary central, and the best-performing decoder changes with demand family and setup cost.

Interpretation. The pattern of decoder wins is informative. Once a setup charge K makes the order/no-order decision dominant, the gated decoders — which factor the decision into an explicit gate (*whether* to order) and a quantity head (*how much*) — win most instances, inverting the vanilla preference for plain-quantity decoders; the ordinal head (NN-SGO, 20 wins) edges the direct head (NN-SGD, 13 wins) where the quantity must batch into a few discrete levels. A limitation is visible directly in the table: at the high setup cost $K = 25$, $p = 4$ several learned policies collapse onto

a degenerate *never-order* policy. The repeated entries 19.992 (Geometric) and 19.995 (MMPP2+) are the cost of a policy that effectively stops ordering to avoid the setup charge, which Tree-1 and Tree-2 reach across most $K = 25, p = 4$ instances; the gated ordinal decoder (NN-SGO) is the one architecture that consistently escapes this collapse and remains instance-best, underscoring that the decoder, not the optimizer, governs the order/no-order boundary.

5 Dual sourcing

5.1 Problem formulation

Figure 8 depicts this environment as a flow network — the regular and expedited supply modes, the demand draw, and where holding and backorder costs are charged.

We consider the single-item, periodic-review dual-sourcing inventory model of Gijsbrechts et al. [11]. Demand and order quantities are integer, and the objective is to minimize the long-run average cost, which consists of procurement, inventory-holding, and backorder costs. A single item is replenished from two suppliers: a slow *regular* supplier with lead time $l_r \geq 1$ and unit cost c_r, and a fast *expedited* supplier with lead time $l_e = 0$ and unit cost $c_e > c_r$. Demand is independent and identically distributed across periods, $D_t \sim D$, with D uniform on $\{0, 1, 2, 3, 4\}$. We write $(x)^+ = \max(0, x)$ and $(x)^- = \max(0, -x)$.

Timing of a period. At the beginning of period $t \in \{1, 2, \ldots, T\}$ the regular order placed l_r periods earlier, $q^r_{t-l_r}$, arrives and is added to the net inventory carried in from the previous period. We let I_t denote the resulting net inventory at the start of period t, after this regular arrival but before the current orders and demand. The controller then places a regular order q^r_t (which arrives l_r periods later, at the start of period $t + l_r$) and an expedited order q^e_t (which, because $l_e = 0$, arrives immediately within period t). Demand D_t is then realized and served from the on-hand stock $I_t + q^e_t$; unmet demand is *backordered* rather than lost, so net inventory may become negative. Holding and backorder costs are charged on the end-of-period net inventory.

State. Because only the expedited lead time is zero, the regular pipeline must be tracked while the expedited pipeline need not be. Following the post-decision (reduced) state of Gijsbrechts et al. [11], the MDP state is the l_r-dimensional vector

$$\mathbf{S}_t = \big(I_t,\ q^r_{t-l_r+1},\ q^r_{t-l_r+2},\ \ldots,\ q^r_{t-1}\big),$$

whose first component I_t is the net inventory at the start of period t (after the regular arrival $q^r_{t-l_r}$) and whose remaining $l_r - 1$ components are the outstanding regular orders still in transit, ordered by remaining lead time. For $l_r = 2$ this reduces to $\mathbf{S}_t = (I_t,\ q^r_{t-1})$.

Action. The decision in period t is the order pair

$$\mathbf{a}_t = \big(q^r_t,\ q^e_t\big), \qquad q^r_t \in \{0, 1, \ldots, \bar{q}_r\}, \quad q^e_t \in \{0, 1, \ldots, \bar{q}_e\},$$

where $\bar{q}_r$ and $\bar{q}_e$ are the regular and expedited order caps.

Transition. Define the end-of-period net inventory after the expedited arrival and demand realization,

$$J_t \ = \ I_t + q^e_t - D_t.$$

The expedited order arrives within the period and contributes to $I_t + q_t^e$; the next-period net inventory then absorbs the regular order due next period, $q_{t-l_r+1}^r$, while the remaining regular pipeline shifts forward by one slot and the freshly placed regular order q_t^r enters the last slot:

$$\begin{aligned} I_{t+1} = J_t + q_{t-l_r+1}^r \;&=\; \big(I_t + q_t^e - D_t\big) + q_{t-l_r+1}^r, \\ \mathbf{S}_{t+1} &= \big(I_{t+1},\; q_{t-l_r+2}^r,\; \ldots,\; q_{t-1}^r,\; q_t^r\big). \end{aligned}$$

When $l_r = 1$ there is no in-transit pipeline and the state collapses to $I_{t+1} = I_t + q_t^e - D_t + q_t^r$.

One-period cost. The cost incurred in period t is the linear procurement cost of the two orders plus the end-of-period holding and backorder costs on the net inventory J_t:

$$c_t \;=\; c_r\, q_t^r + c_e\, q_t^e + h\,(J_t)^+ + b\,(J_t)^-, \tag{7}$$

where h is the holding cost per unit of end-of-period on-hand stock and b is the backorder penalty per unit short. Equivalently, $h\,(J_t)^+ + b\,(J_t)^- \;=\; h\,(I_t + q_t^e - D_t)^+ + b\,(D_t - I_t - q_t^e)^+$. Taking expectations over the demand,

$$\mathbb{E}[c_t] = c_r\, q_t^r + c_e\, q_t^e + h\,\mathbb{E}\big[(I_t + q_t^e - D_t)^+\big] + b\,\mathbb{E}\big[(D_t - I_t - q_t^e)^+\big].$$

Objective. A stationary policy $\pi_{\boldsymbol{\theta}}$ maps the state $\mathbf{S}_t$ to the order pair (q_t^r, q_t^e). Denoting the average expected cost over a horizon of T periods under $\pi_{\boldsymbol{\theta}}$ by $\bar{C}_T(\boldsymbol{\theta}) = \frac{1}{T}\sum_{t=1}^{T}\mathbb{E}[c_t]$, the objective is to minimize the long-run average expected cost, obtained as the limit

$$\bar{C}(\boldsymbol{\theta}) \;=\; \lim_{T\to\infty} \bar{C}_T(\boldsymbol{\theta}), \qquad \min_{\boldsymbol{\theta}}\; \bar{C}(\boldsymbol{\theta}). \tag{8}$$

Because demand is stationary, the system is stationary and we seek a well-performing stationary policy. As for the other problems, $\bar{C}(\boldsymbol{\theta})$ is estimated by simulation: a finite-horizon average cost $\hat{C}_T(\boldsymbol{\theta})$ with a warm-up period is used during training and a long horizon for evaluation, in line with the standard inventory-simulation protocol.

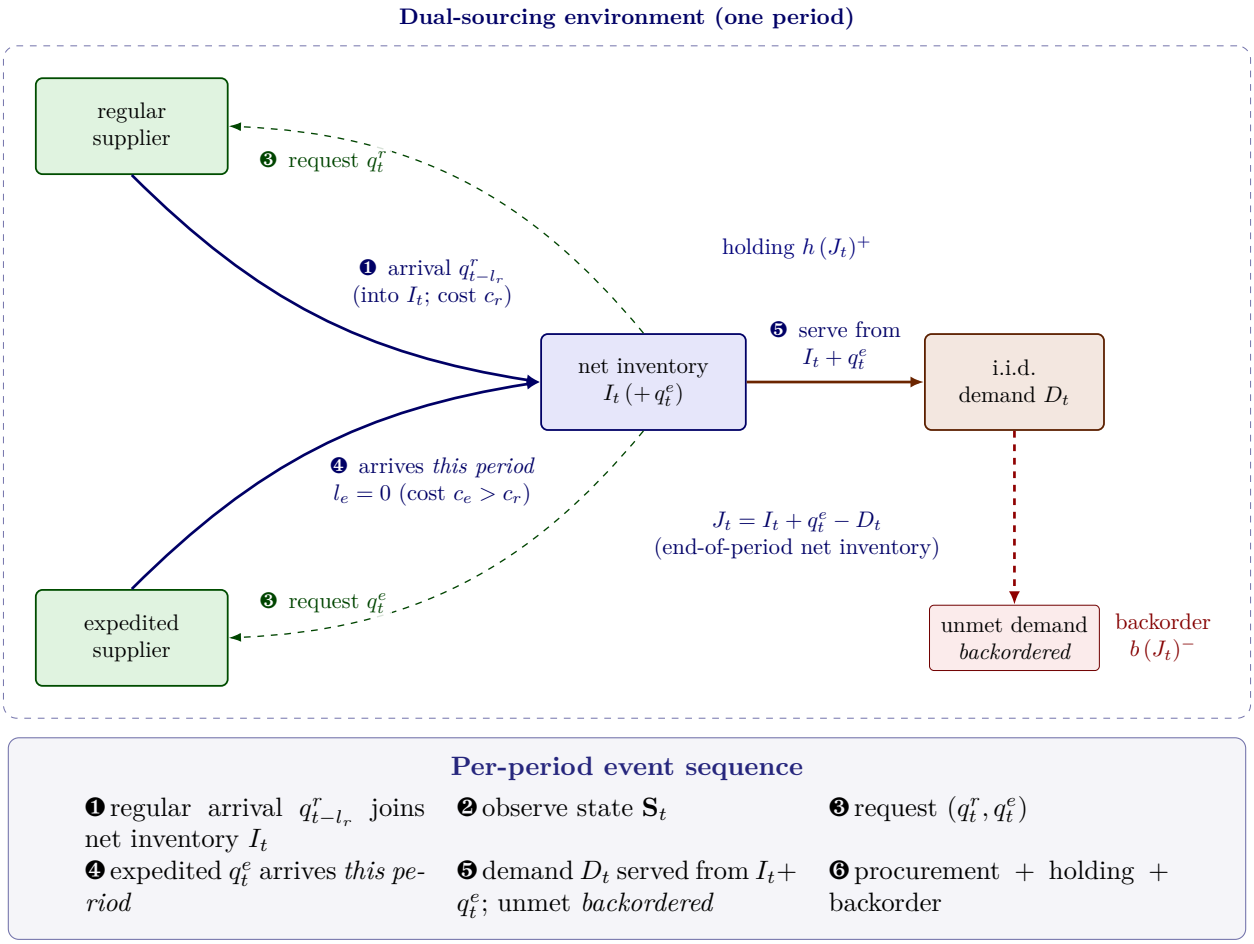


Figure 8: Dual-sourcing environment of Gijsbrechts et al. [11], drawn as an event-based interaction. Each period the stock point sends two *order requests* (thin dashed control arrows): a regular request q_t^r and an expedited request q_t^e. The slow *regular* supplier ships and the goods arrive after the lead time $l_r \geq 1$ (unit cost c_r); the fast *expedited* supplier ships *within the same period* ($l_e = 0$, unit cost $c_e > c_r$) and is available to serve demand (solid material arrows). *Unmet demand is backordered*, so net inventory may go negative. Holding cost h and backorder penalty b are charged on the end-of-period net inventory J_t; the boxed panel beneath gives the per-period event sequence (Section 5.1).

5.2 Policy

Dual sourcing tests whether the same recipe (Figure 1) carries over when the action is a structured order pair rather than a scalar; here the backbone is the soft tree and the decoder lives in the capped-dual-index coordinate system. **Input.** The policy input is this problem's reduced post-decision state $\mathbf{S}_t = (I_t, q^r_{t-l_r+1}, \ldots, q^r_{t-1})$ (for $l_r = 2$, (I_t, q^r_{t-1}): net inventory and the regular pipeline). **Output.** A valid action is an order pair (q^r_t, q^e_t) within the caps $\bar{q}_r, \bar{q}_e$. **Internals.** Following the policy interface of Section 3.1, $\pi_{\boldsymbol{\theta}}$ normalizes the state and applies the soft-tree split/leaf mechanism of Section 3.1 as its backbone; the decoder is the leaf output, which emits a control in *factorized capped-dual-index coordinates* $(s_e,\ \Delta_r,\ \bar{c}_r)$ with $s_r = s_e + \Delta_r$, where s_e is the expedite order-up-to level, s_r the regular order-up-to level, and $\bar{c}_r$ a cap on the regular order. The capped-dual-index action geometry then projects this control onto the valid order pair: the expedited order raises net inventory toward s_e and the regular order raises the inventory position toward s_r subject to the cap $\bar{c}_r$ and the order caps. We compare a small-cap variant with $\bar{c}_r$ on the discrete grid $\{1, 2, 3, 4, 6, 8, 12\}$ (axis-aligned, constant leaves) and a continuous capped-dual-index-targets variant; both keep the policy in the heuristic's coordinate system while letting the targets depend on state, and a raw direct-order output cannot express the capped dual-index control. We warm-start CMA-ES at the encoded capped-dual-index solution as described in Section 3.2. The comparison is therefore one of action geometry and reliable optimization rather than tree capacity, and is made against the classical structured heuristics introduced above.

Decision path: a worked example. We use the instance with $l_r = 2$, $c_e = 105$, where the best model is a depth-2 axis-aligned constant-leaf soft tree with the small-cap capped-dual-index adapter. The reduced post-decision state is $[I_t, q_{t-1}]$ (net inventory and the regular pipeline). A soft-tree leaf emits a 3-control $(s_e, \Delta_r, \bar{c}_r)$ on the discrete grid, and the adapter maps it to orders: $s_r = s_e + \max(\Delta_r, 0)$, expedite $= \text{clip}(s_e - I_t, 0, 12)$, and regular $= \min(\max(0, s_r - (I_t + q_{t-1}) - \text{expedite}), \bar{c}_r, 12)$. For state $[6, 0]$ the tree selects the leaf control $(4, 3, 2)$ (dominant leaf probability 0.87), and the adapter returns (expedite $= 0$, regular $= 1$); for the leaner state $[2, 0]$ the same leaf control yields (expedite $= 2$, regular $= 2$) — the expedite supplier covers the immediate net-inventory shortfall while the capped regular order rebuilds the position. The dual-sourcing policy was validated by a full rollout cost match between two independent implementations on a fixed Uniform$\{0, \ldots, 4\}$ demand sequence of 100,000 periods; every absolute cost difference was exactly 0.000 (well under the 0.05 tolerance), confirming that the divide-by-scale normalization is applied identically in both implementations, and no reported action was withheld.

5.3 Experiment setup

We use the six small-scale benchmark instances of Gijsbrechts et al. [11] (their Figure 9), with $l_r \in \{2, 3, 4\}$ and $c_e \in \{105, 110\}$ and the common parameters of Table 4. The published comparators are the optimality gaps, relative to the bounded-DP optimum, of the single-index, dual-index, capped dual-index, and tailored base-surge heuristics together with the A3C learner of Gijsbrechts et al. [11]: the capped dual-index gap is $\leq 0.11\%$ on every row, making it both the best heuristic and an effective optimal proxy, whereas A3C ranges from 0.51% to 1.85%. We therefore benchmark the learned policy against the *capped dual-index* cost as the optimal proxy rather than our own bounded DP (see Appendix A.4). Policies are trained with warm-started CMA-ES — five independent optimizer seeds per instance, per the depth standard of Section 3.3 — and evaluated under the common-random-number protocol (80 shared demand-path seeds, horizon 10^5); the paired comparison has standard error well below the per-demand-seed standard deviation, so the

sub-percent margins reported below are statistically meaningful at the per-seed level, and the table reports their mean ± standard deviation across the five optimizer seeds.

Table 4: Dual-sourcing benchmark instances of Gijsbrechts et al. [11]: the six instances are the full cross of the regular lead time l_r and the expedited unit cost c_e. All share $l_e = 0$, $c_r = 100$, $h = 5$, $b = 495$, demand $U\{0, \ldots, 4\}$, and order caps of 12.

Design dimension	Values
Regular lead time l_r	2, 3, 4
Expedited unit cost c_e	105, 110

5.4 Results

Table 5 compares the learned policy against the capped dual-index (CDI) optimal proxy under the common-random-number protocol, reported per the depth standard of Section 3.3 as the mean ± sample standard deviation across five independent CMA-ES optimizer seeds per instance. The headline reading is that the learned soft tree *matches* CDI on every instance: on two instances every optimizer seed reproduces the CDI cost exactly (paired gap 0.000±0.000), and on the remaining four the seed-mean paired gap is at most +0.052% — in all cases inside CDI's own published optimality band ($\leq 0.11\%$ gap to the bounded-DP optimum). No instance shows a seed-robust beat or loss. (The earlier candidate search produced individual policies negligibly below CDI, by -0.009% and -0.041%; those were best-of-candidates selections and are superseded by the seed-mean figures here.)

Interpretation. The salient result is therefore reliability of reproduction rather than a new champion: by living in the capped-dual-index coordinate system and warm-starting at the CDI solution, CMA-ES recovers the near-optimal control on every instance. Because CDI is the best static capped-dual-index comparator and a $\leq 0.11\%$ proxy for the bounded-DP optimum, the learned policy thereby sits at the same near-optimal reference level on every instance column — comfortably inside the band that the published A3C learner, with optimality gaps of 0.51–1.85%, does not reach. The action geometry, not the learner's capacity, is what makes this attainable: a raw direct-order decoder cannot express the capped dual-index control and does not reach this level.

A heuristic-near-optimal anchor. CDI's near-optimality here is a robust property of the problem, not an artifact of these six instances. Sweeping the levers that most stress static dual-sourcing control — the expedited premium $c_e - c_r$, the penalty-to-holding ratio, demand variability, and the expedited order capacity — the capped dual-index cost stays within 0.4% of the *exact* bounded-DP optimum on every cell whose value iteration we could validate (Appendix A.4). Dual sourcing is therefore the suite's *heuristic-near-optimal anchor*: a regime in which a compact static heuristic already captures essentially all attainable value, so the right outcome for a learned policy is to *match* it — which the ES policy does, seed-robustly — not to beat it. This is the deliberate counterpoint to the multi-echelon and one-warehouse problems, where the best structured heuristic is well short of optimal and the same ES recipe improves on it by double-digit margins; together they show the method behaving correctly across the difficulty range the suite spans.

Table 5: Dual-sourcing: learned soft-tree policy vs. the capped dual-index (CDI) optimal proxy. Learned cells are the mean ± sample standard deviation across $N = 5$ independent CMA-ES optimizer seeds; every policy is scored under paired common-random-number evaluation (80 demand-path seeds, horizon 10^5). $\Delta\%$ is the per-seed paired gap to CDI (negative is better); the A3C row reports the published optimality gap from Gijsbrechts et al. [11].

	Regular lead time $l_r = 2$		Regular lead time $l_r = 3$		Regular lead time $l_r = 4$	
Policy / metric	$c_e = 105$	$c_e = 110$	$c_e = 105$	$c_e = 110$	$c_e = 105$	$c_e = 110$
CDI proxy	216.769	219.786	216.864	220.449	216.879	220.846
Learned soft tree ($N = 5$)	216.883 ± 0.141	219.786 ± 0.000	216.884 ± 0.020	220.449 ± 0.000	216.895 ± 0.023	220.848 ± 0.005
Learned vs. CDI (%)	$+0.052 \pm 0.065$ (match)	$+0.000 \pm 0.000$ (match)	$+0.009 \pm 0.009$ (match)	$+0.000 \pm 0.000$ (match)	$+0.007 \pm 0.010$ (match)	$+0.001 \pm 0.002$ (match)
Published A3C gap	0.52%	0.80%	0.82%	0.51%	1.85%	1.33%

All six instances match CDI to within its own published optimality band ($\leq 0.11\%$): the largest five-seed-mean paired gap is $+0.052\%$, and on two instances every optimizer seed reproduces CDI exactly. We report these as matches; no cell is claimed as an improvement or a loss.

6 Multi-echelon inventory control: one warehouse, R retailers with special delivery

6.1 Problem formulation

Figure 9 depicts this environment as a flow network — the warehouse and retailers, the order requests and shipments, the special delivery, and where each cost is charged.

We formalize the divergent two-echelon system as a discrete-time MDP under the long-run average-cost criterion, mirroring the lost-sales formulation above. A single capacitated warehouse (echelon 0) replenishes R identical capacitated retailers (echelon $i \in \{1, \dots, R\}$) over periods $t \in \{1, 2, \dots, T\}$; we are interested in the limit $T \to \infty$. The warehouse orders from an external manufacturer with lead time $l_w \geq 0$; shipments from the warehouse reach a retailer after a lead time $l_r \geq 0$. Retailer i faces i.i.d. integer demand $d_{i,t}$ obtained by rounding a normal draw to the nearest non-negative integer, $d_{i,t} = \lfloor \max(0, z) \rceil$ with $z \sim \mathcal{N}(\mu, \sigma^2)$, independently across retailers and periods. We adopt the faithful Gijsbrechts et al. (2022) cost convention [11] used for all experiments below, in which a period's arriving order is merged into on-hand stock *before* ordering and holding cost is charged on *end-of-period* on-hand inventory; the alternative Van Roy et al. (1997) convention [10] (which restores the warehouse's post-shipment installation position and charges holding on the post-decision pre-demand stock) is used only to reproduce the published constant-base-stock costs and is not the convention stated here. We write $x^+ = \max(0, x)$.

Timing of a period. Entering period t, the warehouse holds on-hand stock I^w_{t-1} with an outstanding-order pipeline $\left(q^w_{t-l_w}, \dots, q^w_{t-1}\right)$, and each retailer i holds I^i_{t-1} with pipeline $\left(s^i_{t-l_r}, \dots, s^i_{t-1}\right)$ of in-transit shipments. The period proceeds in five stages: (i) the order $q^w_{t-l_w}$ scheduled to arrive now is added to warehouse on-hand and the shipment $s^i_{t-l_r}$ to each retailer's on-hand, giving the available stocks $\widetilde{I}^w_t = I^w_{t-1} + q^w_{t-l_w}$ and $\widetilde{I}^i_t = I^i_{t-1} + s^i_{t-l_r}$; the warehouse then places a new order q^w_t from the external source against its *pre-shipment* installation inventory position. (ii) Each retailer raises its inventory position toward a shared order-up-to target y^r_t, requesting $\rho^i_t = \left(y^r_t - \mathrm{IP}^i_t\right)^+$ units; these requests are filled from current warehouse available stock $\widetilde{I}^w_t$, and when stock is short they are rationed by an allocation rule $\mathcal{A}$ that maximizes the minimum resulting retailer inventory position (equivalently, minimizes the spread of retailer positions), yielding the shipped quantities s^i_t that leave warehouse stock into the outbound retailer pipelines. (iii) Demand $d_{i,t}$ is realized at each retailer and served from retailer available on-hand, $\sigma^i_t = \min(\widetilde{I}^{i+}_t, d_{i,t})$, leaving unmet demand $u^i_t = d_{i,t} - \sigma^i_t$. (iv) Each unmet unit independently requests a same-day *special delivery* from the

leftover warehouse stock with probability P_w and is otherwise lost; the accepted requests are filled up to the leftover warehouse stock, the rest are lost sales — a hybrid of backlogging and lost sales. (v) The transportation pipelines advance one stage, appending q_t^w and s_t^i as the freshest in-transit orders.

State. The pre-decision MDP state collects the warehouse on-hand level and its inbound pipeline together with each retailer's on-hand level and inbound pipeline,

$$\mathbf{S}_t = \Big(I_{t-1}^w;\quad \underbrace{q_{t-l_w}^w, \dots, q_{t-1}^w}_{l_w \text{ warehouse pipeline}} \;;\; \big(I_{t-1}^i\big)_{i=1}^R;\; \big(\underbrace{s_{t-l_r}^i, \dots, s_{t-1}^i}_{l_r \text{ retailer pipeline}}\big)_{i=1}^R \Big),$$

an $(1+l_w)+R\,(1+l_r)$-dimensional vector. From it we form the post-arrival available stocks $\widetilde{I}_t^w, \widetilde{I}_t^i$ defined above and the (pre-shipment) installation inventory positions

$$\mathrm{IP}_t^w = \widetilde{I}_t^w + \sum_{k=1}^{l_w-1} q_{t-k}^w, \qquad \mathrm{IP}_t^i = \widetilde{I}_t^i + \sum_{k=1}^{l_r-1} s_{t-k}^i.$$

Action. The control is the pair $a_t = (q_t^w,\, y_t^r)$ consisting of the warehouse order quantity q_t^w and the shared retailer order-up-to level y_t^r (a learned policy may emit a warehouse order-up-to level y_t^w and set $q_t^w = \big[\min(y_t^w, C^w) - \mathrm{IP}_t^w\big]^+$). The order is feasible subject to the per-period production capacity C^m and the warehouse inventory-position cap C^w, and the retailer target is capped by the retailer inventory-position cap C^r:

$$q_t^w = \min\Big(q_t^w,\; C^m,\; \big(C^w - \mathrm{IP}_t^w\big)^+\Big), \qquad y_t^r \le C^r.$$

The desired retailer orders $\rho_t^i = \big(y_t^r - \mathrm{IP}_t^i\big)^+$ are met by the shipped quantities

$$\big(s_t^i\big)_{i=1}^R = \mathcal{A}\Big(\big(\rho_t^i\big)_{i=1}^R,\; \widetilde{I}_t^{w+}\Big), \qquad \sum_{i=1}^R s_t^i \le \widetilde{I}_t^{w+},$$

where $\mathcal{A}$ is the min-shortage (max–min position) allocation when warehouse stock cannot cover all requests, and $s_t^i = \rho_t^i$ otherwise. Because the warehouse orders against its *pre*-shipment position IP_t^w, the units shipped downstream are not deducted when sizing q_t^w; they reduce warehouse on-hand only through the next-period state.

Transition. Let $W_t = \big(\widetilde{I}_t^{w+} - \sum_{i=1}^R s_t^i\big)^+$ denote the warehouse stock left after regular shipments. Total unmet demand is $U_t = \sum_{i=1}^R u_t^i$; the number of accepted special-delivery requests $A_t \sim \mathrm{Binomial}(U_t, P_w)$ is filled up to W_t, so the expedited (special-delivery) units and the lost units are

$$E_t = \min(A_t,\, W_t), \qquad \ell_t = U_t - E_t.$$

The next-period on-hand inventories are the end-of-period stocks

$$\begin{aligned} I_t^w &= W_t - E_t, \\ I_t^i &= \widetilde{I}_t^i - \sigma_t^i = \big(I_{t-1}^i + s_{t-l_r}^i\big) - \min\big(\widetilde{I}_t^{i+}, d_{i,t}\big), \end{aligned}$$

and the pipelines shift forward, dropping the just-arrived stage and appending the new order and shipments:

$$\big(q_{t-l_w+1}^w, \dots, q_{t-1}^w,\, q_t^w\big), \qquad \big(s_{t-l_r+1}^i, \dots, s_{t-1}^i,\, s_t^i\big)_{i=1}^R,$$

which together with I_t^w and $\big(I_t^i\big)_{i=1}^R$ define $\mathbf{S}_{t+1}$.

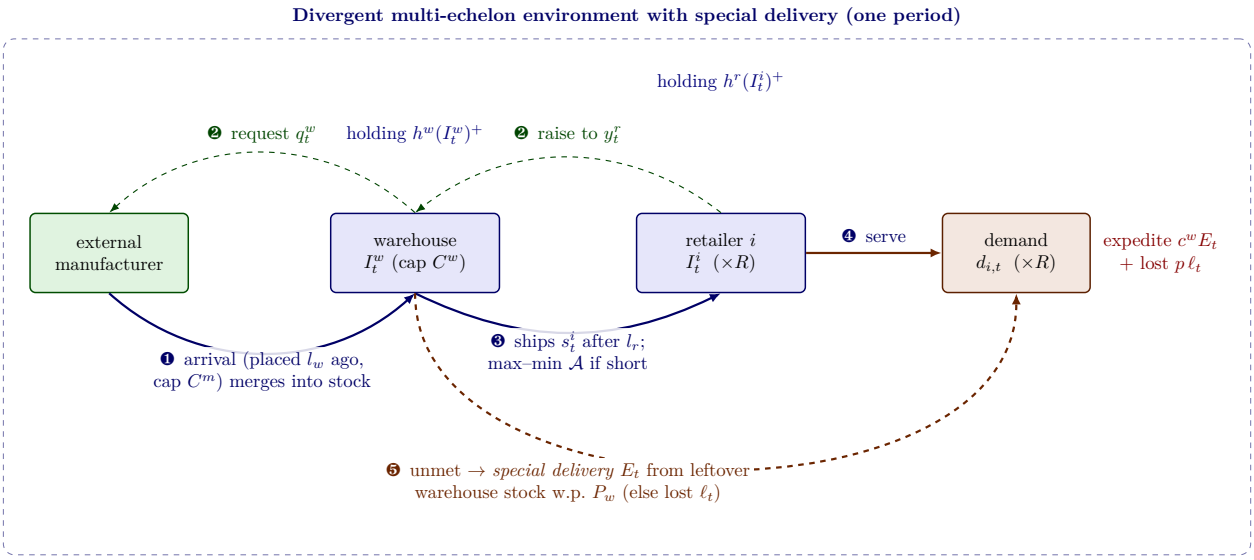
One-period cost. Holding cost is charged on end-of-period on-hand inventory at the warehouse rate h^w and retailer rate h^r; each expedited unit costs c^w and each lost unit a penalty p. There is no separate charge for regular retailer shipments. The period cost is

$$c_t = h^w \left(I_t^w\right)^+ + h^r \sum_{i=1}^{R} \left(I_t^i\right)^+ + c^w \, E_t + p \, \ell_t. \tag{9}$$

Objective. A stationary policy $\pi_{\boldsymbol{\theta}}$ maps the state $\mathbf{S}_t$ to the action $a_t = (q_t^w, y_t^r)$. Writing $\bar{C}_T(\boldsymbol{\theta}) = \frac{1}{T}\sum_{t=1}^{T} \mathbb{E}[c_t]$ for the expected average cost over T periods under $\pi_{\boldsymbol{\theta}}$, with the expectation taken over the demand process, the objective is to minimize the long-run average expected cost

$$\bar{C}(\boldsymbol{\theta}) = \lim_{T\to\infty} \bar{C}_T(\boldsymbol{\theta}).$$

As in the lost-sales problem this limit is estimated by the empirical average cost $\hat{C}_T(\boldsymbol{\theta}) = \frac{1}{T}\sum_{t=1}^{T} c_t$ over a long simulated horizon with a warm-up discarded, which is the quantity CMA-ES minimizes.



Per-period event sequence

❶ arrivals merge into warehouse & retailer on-hand (before ordering)

❷ warehouse orders q_t^w (pre-shipment IP); retailers raise to y_t^r

❸ warehouse ships s_t^i (max–min $\mathcal{A}$ if short)

❹ each retailer serves demand $d_{i,t}$

❺ unmet → special delivery from leftover stock (w.p. P_w) else lost

❻ holding + expedite + lost

Figure 9: Divergent multi-echelon environment with special delivery [11, 10], drawn as an event-based interaction. Each period the warehouse *sends an order request* q_t^w to the external manufacturer, which ships; the goods arrive after lead time l_w (production cap C^m, position cap C^w). Each retailer in turn *requests* a raise to the shared order-up-to level y_t^r (thin dashed control arrows); the warehouse *ships* s_t^i, arriving after lead time l_r, rationed by the max–min allocation rule $\mathcal{A}$ when stock is short (solid material arrows). Each retailer serves its own i.i.d. demand from on-hand stock; each unmet unit independently requests a same-period *special delivery* from leftover warehouse stock with probability P_w (filled up to that stock) and is otherwise lost. Holding cost is charged on end-of-period on-hand stock at the warehouse (h^w) and retailers (h^r), each expedited unit costs c^w, and each lost unit a penalty p; the boxed panel beneath gives the per-period event sequence (Section 6, Eq. (9)).

6.2 Policy

This is the most demanding test of the action-parameterization principle (Figure 1), because the wrong action geometry is fatal. **Input.** The policy input is this problem's pre-decision state $\mathbf{S}_t$ (warehouse on-hand and its inbound pipeline, then per-retailer on-hand and inbound pipelines), consumed with a policy-owned divide-by-scale normalization. **Output.** A valid action is the pair (q_t^w, y_t^r): a warehouse order and a shared retailer order-up-to level, within the physical caps. **Internals.** Following the policy interface of Section 3.1, $\pi_{\boldsymbol{\theta}}$ normalizes the state and uses the soft decision tree with oblique splits and vector-valued leaves of Section 3.1 as its backbone; the decoder is the pair of leaf outputs, the state-dependent warehouse and retailer order-up-to levels y_S^w, y_S^r. The projection that guarantees validity sets the warehouse order to $q^w = \left[\min(y_S^w, C^w) - \mathrm{IP}^w\right]^+$ capped by C^m (where IP^w is the warehouse inventory position and $[\cdot]^+ = \max(\cdot, 0)$), and each retailer order raises its position toward $\min(y_S^r, C^r)$.

We contrast two action geometries for the leaf outputs. The reduced **grid** design of Gijsbrechts et al. [11] draws the levels from the reduced discrete set $y^w \in \{50, 60, \ldots, 100\}$, $y^r \in \{0, 5, \ldots, 40\}$; our **direct** design lets the leaves estimate the levels as continuous values rounded to non-negative integers, bounded only by the physical caps $y^w \in [0, C^w]$, $y^r \in [0, C^r]$. The distinction is consequential because the cost-minimizing warehouse base-stock for these settings is $\approx$ 300–525 (the published constant base-stock uses $y^w = 330, 460$), far above the reduced grid's cap of 100, so the grid cannot express the operating region and is a policy artifact, not a problem constraint. We train for the problem itself and use the published numbers only as a benchmark, independently of the learner in Gijsbrechts et al. [11].

Decision path: a worked example. We use setting 1, where the best run is a depth-2 oblique soft tree with linear leaves in the direct-level action mode. The raw decision state has dimension 22 (warehouse on-hand-after-arrival and its pipeline, then per-retailer on-hand and pipelines for 10 retailers), normalized by $\kappa = 100$, and the two leaf outputs are the warehouse and retailer order-up-to levels directly. For a state with the warehouse holding 300 on hand (empty pipeline) and each of the 10 retailers holding 20 on hand with a pipeline of 5, the dominant leaf (probability 1.0) emits continuous levels $[298.572, 54.445]$, rounded and clamped to a warehouse order-up-to of 299 and a retailer order-up-to of 54.

Validation. The multi-echelon model has no deterministic demand-binding and a stochastic warehouse-allocation and emergency-shipment rollout, so a full cost match is not appropriate; instead the vector soft-tree forward was validated by extracting the warehouse action dimension into a scalar control and matching it against the independent scalar forward over 5000 random raw states (maximum absolute discrepancy 0). The retailer dimension uses the identical leaf computation and differs only by a zero minimum-value offset and an upper clamp, so it is faithful by construction; the single multi-echelon action is therefore reported with this cross-check rather than a rollout cost match.

6.3 Experiment setup

We evaluate on *both* multi-echelon settings of Gijsbrechts et al. [11] — the two complex case studies of Van Roy et al. [10] — which are the only multi-echelon instances they report, so our coverage of their multi-echelon benchmark is complete. Table 6 reproduces their Table 3 parameters exactly for both settings under the faithful Gijsbrechts et al. (2022) cost convention; retailer demand is $\mathcal{N}(\mu, \sigma^2)$ rounded to a non-negative integer.

Table 6: Multi-echelon experiment settings, reproducing Table 3 of Gijsbrechts et al. [11] (the two complex case studies of Van Roy et al. [10]) parameter-for-parameter under the faithful Gijsbrechts et al. (2022) cost convention. These are the only two multi-echelon settings reported by Gijsbrechts et al. [11]; we evaluate both. Lead times l_w, l_r; demand mean μ and standard deviation σ; number of retailers R; warehouse and retailer holding costs h^w, h^r; unit expedite cost c^w; lost-sales penalty p; special-delivery success probability P_w; production, warehouse, and retailer capacities C^m, C^w, C^r.

Parameter	Setting 1	Setting 2
Warehouse lead time l_w	2	5
Retailer lead time l_r	2	3
Demand mean μ	5	0
Demand standard deviation σ	14	20
Retailers R	10	10
Warehouse holding cost h^w	3	3
Retailer holding cost h^r	3	3
Unit expedite cost c^w	0	0
Lost-sales penalty p	60	60
Special-delivery success probability P_w	0.8	0.8
Production capacity C^m	100	100
Warehouse capacity C^w	1000	1000
Retailer capacity C^r	100	100

The published benchmarks we compare against are the constant base-stock costs $(330, 23) \to 1302$ and $(460, 22) \to 1449$, the best neuro-dynamic-programming costs 1179 and 1318, and the A3C improvements over constant base-stock of $8.95\% \pm 0.13\%$ and $12.09\% \pm 0.39\%$ [10, 11]. As the in-environment benchmark we compute the best *constant* (state-independent) base-stock by grid search over the physical operating region (warehouse up to $\approx C^w$, retailer up to C^r) rather than over the reduced grid. Policies are trained with CMA-ES; we sweep the action design against tree depth $\in \{2, 3\}$ and evaluate under the multi-echelon protocol of Section 3.3: each trained policy is evaluated at horizon 3×10^4 over 6 *evaluation* (demand-path) seeds; the robustness statistics in the results below are over 5 independent CMA-ES *optimizer* seeds, a separate and deliberately distinct seed count.

6.4 Results

Table 7 reports the best in-environment constant base-stock, the best learned policy under each action design, and the learned policy's improvement over the constant base-stock, alongside the published A3C improvement. Over 5 independent CMA-ES seeds the direct-estimation policy improves on the best constant base-stock by $14.7\% \pm 1.6\%$ (setting 1) and $12.0\% \pm 2.3\%$ (setting 2), with all 5 seeds below the best constant base-stock on both settings, so the beat is robust to optimizer initialization. The grid-action policy, restricted to $y^w \leq 100$, stays about 230% *above* the benchmark.

Interpretation. The gap between the two action designs is the central finding: the grid policy fails not because of the environment or the learner but because its action space cannot reach the operating region. This is the action-space trap in its starkest form — the same soft tree, optimizer, and horizon, with only the decoder's reachable level set changed, swing from $\approx$ 12–15% better

than the constant base-stock to more than 200% worse. The best tree depth is instance-dependent (the design-by-depth sweep over $\{2, 3\}$ selects it per seed automatically). One comparison caveat is important: our 14.7% / 12.0% seed-mean improvements and the published A3C improvements (8.95%, 12.09%) are computed against *different* baselines and under *different* cost conventions — our improvement is the 5-seed mean relative to the best in-environment constant base-stock under the Gijsbrechts et al. (2022) cost convention, whereas the A3C figures are relative to the published Van Roy et al. (1997) constant base-stock under the original cost convention — so we report the A3C numbers as cross-protocol *context*, not as a head-to-head beat.

Table 7: Multi-echelon learned-policy results (Gijsbrechts et al. (2022) cost convention; mean cost, lower is better). "Improvement" is the direct learned policy's cost reduction relative to the best constant base-stock; the last row is the published A3C improvement over constant base-stock from Gijsbrechts et al. [11].

Policy / metric	Setting 1	Setting 2
Best const. base-stock (y^w, y^r)	$(300, 25) \to 910.3$	$(525, 25) \to 1138.0$
Grid tree	3085.7	3795.1
Direct tree (ours), seed-mean	$\mathbf{776.2 \pm 14.3}_{\text{seed}}$	$\mathbf{1001.1 \pm 25.6}_{\text{seed}}$
Improvement vs. best const.	$\mathbf{14.7\% \pm 1.6\%}$	$\mathbf{12.0\% \pm 2.3\%}$
Published A3C improvement (context)	8.95%	12.09%

Mean ± standard deviation over 5 independent CMA-ES seeds, against the env's own grid-searched best constant base-stock (the same-protocol comparator, seed-mean 910.3 ± 0.5 / 1138.0 ± 0.4); all 5 seeds beat the best constant base-stock on both settings. The published A3C row is *cross-protocol context* (a different algorithm and cost convention), not a head-to-head beat.

7 Perishable inventory

7.1 Problem formulation

Figure 10 depicts this environment as a flow network — the inbound order and its lead-time shipment into the youngest age bucket, the demand draw served by the FIFO or LIFO issuing rule, the outdating of the oldest leftover stock, and where each cost is charged.

We consider the single-item, periodic-review perishable inventory model of De Moor et al. [22], as re-derived and benchmarked by Farrington et al. [23]. A single item with a *fixed shelf life* of m periods is replenished from one supplier with deterministic lead time $L \geq 1$. Demand D_t is i.i.d. across periods, obtained by sampling a Gamma distribution with mean μ and coefficient of variation cv and rounding to the nearest non-negative integer. We study the two instances that admit an exact-MDP verifier: both have $m = 2$, $L = 1$, $\mu = 4$, cv $= 0.5$, and differ only in the issuing rule, FIFO or LIFO. Unmet demand is *lost*. We write $(x)^+ = \max(0, x)$.

Timing of a period. On-hand stock is tracked by remaining shelf life. At the start of period t the controller observes the age-structured stock, places an order q_t, demand D_t is realized and served from on-hand stock under the issuing rule (FIFO issues oldest units first, LIFO youngest first), unmet demand is *lost*, the *oldest leftover units then outdate* (expire and are scrapped), the surviving units *age* by one bucket, and the order placed L periods earlier arrives into the *youngest* bucket. With $L = 1$ the order q_t arrives at the start of period $t + 1$ and there is no in-transit pipeline.

State. Because shelf life is fixed at m buckets and only $L-1$ orders are in transit, the MDP state is the on-hand age vector together with the in-transit pipeline,

$$\mathbf{S}_t = \big(\underbrace{q_{t-L+1}, \ldots, q_{t-1}}_{\text{pipeline } (L-1 \text{ entries})} ; \ \underbrace{o_t^1, \ldots, o_t^m}_{\text{on-hand by age}} \big),$$

where o_t^j is the on-hand quantity with j periods elapsed (o_t^1 youngest, o_t^m oldest). For the studied slice $m = 2$, $L = 1$ the pipeline is empty and the state collapses to the two-dimensional age vector $\mathbf{S}_t = (o_t^1, o_t^2)$.

Action. The decision in period t is the scalar order quantity

$$a_t = q_t \in \{0, 1, \ldots, \bar{q}\},$$

where $\bar{q}$ is the per-period order cap.

Transition. Let $O_t = \sum_j o_t^j$ be the total on-hand stock. Demand consumes $\min(D_t, O_t)$ units under the issuing rule, leaving a lost quantity $\ell_t = (D_t - O_t)^+$. Writing r_t^j for the leftover in bucket j after issuing, the oldest leftover outdates,

$$w_t \ = \ r_t^m \qquad \text{(outdated units)},$$

the surviving buckets age by one slot, and the arrival q_{t-L+1} (for $L = 1$, the order q_t just placed) enters the youngest bucket:

$$o_{t+1}^1 = \text{arrival}, \qquad o_{t+1}^{j+1} = r_t^j \ \ (j = 1, \ldots, m-1).$$

One-period cost. The cost incurred in period t is the linear procurement cost of the order plus holding cost on the non-expiring on-hand stock, the lost-sales penalty, and the outdating (waste) cost:

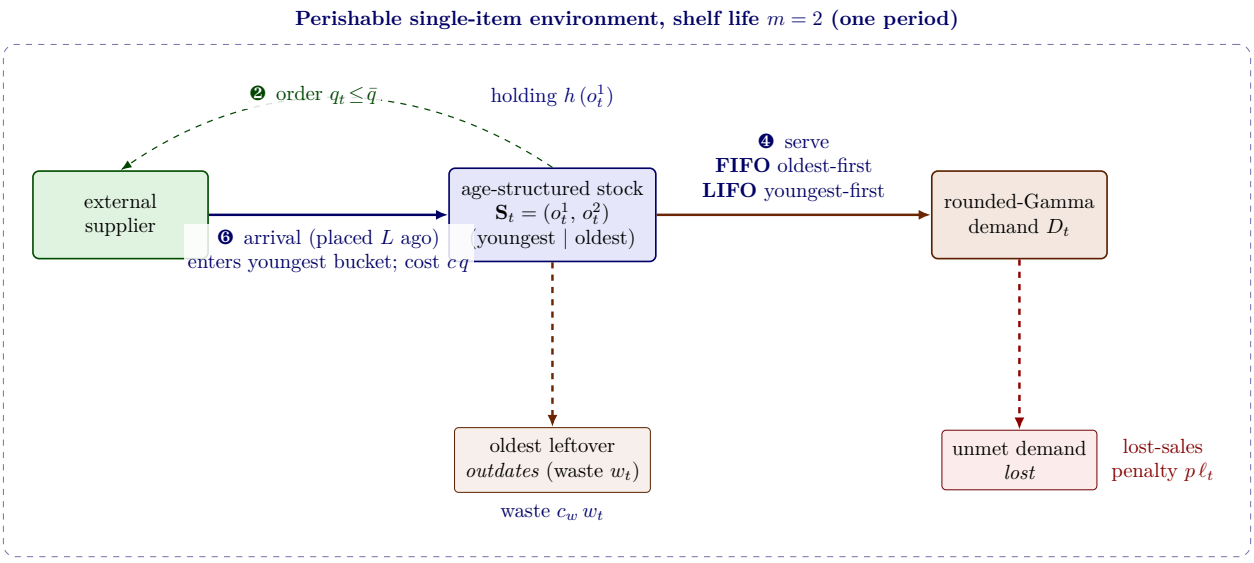
$$c_t \ = \ c\, q_t \ + \ h \sum_{j=1}^{m-1} r_t^j \ + \ p\, \ell_t \ + \ c_w\, w_t, \tag{10}$$

where c is the unit procurement cost, h the holding cost per unit of surviving on-hand stock, p the lost-sales penalty per unit short, and c_w the waste cost per outdated unit. For the studied instances $c = 3$, $h = 1$, $p = 5$, and the waste cost is $c_w = 7$.

Objective. A stationary policy $\pi_{\boldsymbol{\theta}}$ maps the state $\mathbf{S}_t$ to the order q_t. Following the discounted-return convention of De Moor et al. [22] and Farrington et al. [23], performance is the expected discounted *negated* cost over the post-warm-up evaluation window (discount factor γ, warm-up of W periods within a horizon of T),

$$G(\boldsymbol{\theta}) \ = \ \mathbb{E}\left[\sum_{t=W+1}^{T} \gamma^{\,t-W-1} \big(-c_t\big) \right], \qquad \max_{\boldsymbol{\theta}} \ G(\boldsymbol{\theta}), \tag{11}$$

so that reported returns are *negative* (lower cost is a larger, less-negative return) and higher is better. We use $\gamma = 0.99$, $T = 465$, and $W = 100$, i.e. a 365-period evaluation window, matching the protocol whose value-iteration optimum is reproduced cell-for-cell by our exact MDP solver (Section 7.3). As for the other problems, $G(\boldsymbol{\theta})$ is estimated by simulation over shared demand-path seeds.



Per-period event sequence

❶ observe age-structured state $\mathbf{S}_t = (o_t^1, o_t^2)$	❷ order $q_t \in \{0, \dots, \bar{q}\}$	❸ demand D_t drawn (rounded Gamma)
❹ serve D_t by FIFO (oldest first) / LIFO (youngest first); unmet *lost*	❺ oldest leftover bucket *outdates* to waste; survivors age	❻ arrival enters youngest bucket; charge $c\,q_t + h\,o_t^1 + p\,\ell_t + c_w w_t$

Figure 10: Perishable single-item environment of De Moor et al. [22], drawn as an event-based interaction (shelf life $m = 2$, lead time $L = 1$). Each period the stock point *sends an order request* q_t (thin dashed control arrow); the external supplier ships and the goods *arrive* after the lead time L to enter the *youngest* age bucket (solid material arrow; unit cost c). Demand D_t is served from the on-hand age buckets under the configurable *issuing rule* — FIFO issues the *oldest* stock first, LIFO the *youngest* first — and any *unmet demand is lost*. After demand, the *oldest leftover bucket outdates* to a waste sink (orange) and the survivors age by one bucket. Holding cost h is charged on the non-expiring on-hand stock, each lost unit a penalty p, each outdated unit a waste cost c_w, and each ordered unit a procurement cost c; the boxed panel beneath gives the per-period event sequence (Section 7.1, Eq. (10)).

7.2 Policy

Perishable inventory tests whether the same recipe (Figure 1) carries over when the state is an *age-structured* stock vector and the optimal control is known to be non-trivially state-dependent (the published optimal policy orders *less* when older stock is already on hand). **Input.** The policy input is this problem's age-structured state $\mathbf{S}_t = (o_t^1, o_t^2)$ (youngest and oldest on-hand buckets; for $L > 1$ the in-transit pipeline is prepended), divide-by-scale normalized by the demand mean as in Section 3.1. **Output.** A valid action is a single non-negative integer order $q_t \in \{0, \ldots, \bar{q}\}$. **Internals.** Following the policy interface of Section 3.1, $\pi_{\boldsymbol{\theta}}$ applies the soft-tree split/leaf mechanism of Section 3.1 as its backbone: a depth-2 tree with *oblique* (full-dimensional) splits and *linear* leaves, so each leaf emits an affine function of the age vector that is clipped to the order cap. This is the smallest tree that can express the published optimal policy's "order less when older stock is on hand" behaviour — an oblique split separates age profiles and the linear leaf tilts the order against the on-hand buckets — while a constant-leaf or single base- stock level cannot. The whole policy carries 21 parameters. We warm-start CMA-ES at the encoded best base-stock level (Section 3.2): the generation-0 mean return reproduces the best base-stock to within its standard error, so the search starts *at* the heuristic and any improvement is attributable to the learned age-dependence rather than to a lucky initialization.

7.3 Experiment setup

We use the two De Moor et al. (2022) Scenario A instances that we can verify exactly: the FIFO instance and its LIFO sibling, both sharing the parameters of Table 8. These are the only two instances small enough ($11^2 = 121$ states) for our exact value-iteration solver, which re-derives — not merely stores — the published optimal-policy tables, the best base-stock levels ($S = 7$ FIFO, $S = 5$ LIFO), and the Farrington et al. [23] Table 3 value-iteration returns (-1457 FIFO, -1553 LIFO).

Comparator: the best tuned base-stock. The like-for-like comparator is the best single base-stock policy (order-up-to $q_t = (S - \text{inventory position})^+$) at its exact-optimal level, *re-scored on the same Monte-Carlo estimator* used to score the learned policy. This is the appropriate "heuristic to beat": beating the best base-stock policy under an identical estimator is a clean apples-to-apples win, free of the estimator mismatch discussed below.

Evaluation protocol and selection. Policies are trained with warm-started CMA-ES and evaluated under the common-random-number protocol of Section 3.3. Three *disjoint* seed blocks are used: a search block (64 seeds) drives CMA-ES, the best generation is then chosen on a *separate validation block* (512 seeds), and the selected policy is reported on a third *held-out evaluation block* (2048 seeds, horizon 465). Selecting on the disjoint validation block is essential: it yields the heuristic-beating policy reported below, whereas selecting the trained CMA-ES solution on the evaluation block itself overfits the estimator and flips the FIFO result to a small ($\approx -0.49\%$) loss. The best base-stock is scored on the identical evaluation block.

The value-iteration optimum is context only. The published -1457 / -1553 optima are the *analytic* expected discounted returns under a midpoint-binned Gamma demand (matched to Farrington Table 3), whereas the learned and best-base-stock rows are *Monte-Carlo* means over sampled-and-rounded Gamma rollouts. On this slice the optimal base-stock level evaluates $\approx 1\%$ worse under the Monte-Carlo estimator than under the analytic one — a systematic offset that is a

property of the estimator, not of any policy. We therefore report the beat over the best base-stock as the headline result and treat proximity to the VI optimum as informational context.

Table 8: Perishable benchmark instances of De Moor et al. [22] used here (the two admitting an exact-MDP verifier). Both have shelf life $m = 2$, lead time $L = 1$, rounded-Gamma demand with mean $\mu = 4$ and coefficient of variation 0.5, order cap $\bar{q} = 10$, horizon 465 with a 100-period warm-up, and discount $\gamma = 0.99$; they differ only in the issuing rule.

Parameter	Value
Shelf life m / lead time L	2 / 1
Demand (rounded Gamma) μ, cv	4, 0.5
Procurement c / holding h	3 / 1
Lost-sales penalty p / waste c_w	5 / 7
Issuing rule	FIFO *or* LIFO

7.4 Results

Table 9 compares the best learned depth-2 oblique-linear soft tree against the best base-stock under the common-random-number protocol. On both instances the learned policy *beats* the best base-stock, and the beat is seed-robust: over 5 independent CMA-ES seeds the FIFO improvement is $+1.166\% \pm 0.030\%$ and the LIFO improvement is $+0.824\% \pm 0.065\%$, with all 5 seeds beating the best base-stock on both (the cross-seed standard deviation is far smaller than the mean margin, so the gain is not a single-seed artifact). The five-seed-mean FIFO return improves from -1475.71 to -1458.51 ± 0.44 and LIFO from -1566.46 to -1553.55 ± 0.99, each seed's gain many times its paired standard error ($\geq 8.9\times$ FIFO, $\geq 6.2\times$ LIFO), with every seed selected on the disjoint validation block (Section 7.3).

Interpretation. The headline is the *like-for-like beat over the best base-stock*: under one shared Monte-Carlo estimator, a 21- parameter age-dependent soft tree improves on the best base-stock level that the same estimator scores — a level that is itself the exact optimum within the base-stock family. The learned tree exploits the age structure that a single base-stock cannot: it orders less when older stock is already on hand, trading a little holding for fewer outdated units, which is exactly the published optimal policy's shape. As context, both learned returns sit essentially *on* the analytic value-iteration optimum (-1457.28 FIFO, -1552.99 LIFO), but this comparison mixes the analytic and Monte-Carlo estimators (the best base-stock, scored on the Monte-Carlo estimator, sits $\approx 1\%$ below the analytic optimum purely from that mismatch), so we read the VI proximity as corroborating context rather than as a second, independent win. The FIFO margin is the larger of the two, consistent with FIFO leaving more room over the base-stock heuristic than the already near-heuristic-optimal LIFO case.

Table 9: Perishable inventory: best learned depth-2 oblique-linear soft tree vs. the best tuned base-stock, common-random-number evaluation (2048 held-out seeds, horizon 465, $\gamma = 0.99$). Returns are discounted *negated* costs (higher is better). $\Delta\%$ is the learned policy's improvement over the best base-stock; "SEM mult." is that gain divided by the paired standard error. The value-iteration optimum column is the *analytic* reference of De Moor et al. [22] / Farrington et al. [23] and is context only (different estimator).

Policy / metric	FIFO ($S = 7$)	LIFO ($S = 5$)
Best tuned base-stock	-1475.71	-1566.46
Learned soft tree (ours, seed mean, $N = 5$)	$\mathbf{-1458.51 \pm 0.44}_{\text{seed}}$	$\mathbf{-1553.55 \pm 0.99}_{\text{seed}}$
Improvement over best base-stock $\Delta\%$ (5-seed mean)	$\mathbf{+1.166\% \pm 0.030\%}$	$\mathbf{+0.824\% \pm 0.065\%}$
SEM mult. (worst seed)	$\geq 8.9\times$	$\geq 6.2\times$
VI optimum (analytic, context)	-1457.28	-1552.99

The beat over the best base-stock is the like-for-like win: the best base-stock and learned policy are scored on the *same* Monte-Carlo estimator and the best base-stock is the exact optimum within the base-stock family. The VI-optimum row is the analytic discounted return under midpoint-binned Gamma demand and is not on the same estimator (the best base-stock sits $\approx 1\%$ below it purely from that mismatch); we report it as context, not as a separate improvement claim. The $\Delta\%$ row is the mean $\pm$ cross-seed standard deviation over 5 independent CMA-ES seeds (all 5 beat the best base-stock on both instances); the negated-cost values shown are five-seed means. Selection is on a disjoint validation block; selecting on the evaluation block instead flips FIFO to a small ($\approx -0.49\%$) loss.

Three further exactly-solvable instances. To check that the beat over the best base-stock is not specific to the two anchor instances, we ran the *identical* recipe — depth-2 oblique-linear soft tree, warm-started at the best base-stock, CMA-ES under common random numbers, selection on the disjoint validation block — on three more De Moor et al. Scenario-A settings whose exact-MDP value iteration the repository also reproduces (each under 2000 states): a longer shelf life ($m = 3$, $L = 1$, FIFO, waste cost 7), the first setting with a genuine *in-transit pipeline* ($m = 2$, $L = 2$, FIFO, waste cost 7), and a higher waste cost ($m = 2$, $L = 1$, FIFO, waste cost 10). No architecture was re-swept; only the longer-state instances enlarge the policy from 21 to 28 parameters, the extra weights coming from the third state component (the $m = 3$ second on-hand bucket and, for $L = 2$, the in-transit pipeline bucket). On all three the learned tree again *beats* the best base-stock on the shared Monte-Carlo estimator by several times its paired standard error (Table 10), and again sits essentially on the analytic value-iteration optimum, which we read as context only.

Table 10: Perishable inventory, three further exactly-solvable instances (companion to Table 9; same recipe and protocol): best learned depth-2 oblique-linear soft tree vs. the best tuned base-stock, common-random-number evaluation (2048 held-out seeds, horizon 465, $\gamma = 0.99$). Returns are discounted *negated* costs (higher is better). $\Delta\%$ is the learned policy's improvement over the best base-stock, "SEM mult." that gain divided by the paired standard error. The value-iteration optimum is the *analytic* reference of De Moor et al. [22] / Farrington et al. [23] and is context only (different estimator). Instances are described by their parameters; all are FIFO with mean demand 4, coefficient of variation 0.5 and order cap 10.

Instance (S = best base-stock)	Best base-stock	Learned (ours)	$\Delta\%$	SEM mult.	VI optimum (context)
$m = 3,\ L = 1$, waste 7 $(S = 8)$	-1435.30	$\mathbf{-1425.20}$	$\mathbf{+0.70}\%$	$\approx 5.9\times$	-1424.39
$m = 2,\ L = 2$, waste 7 $(S = 9)$	-1495.44	$\mathbf{-1462.45}$	$\mathbf{+2.21}\%$	$\approx 17.7\times$	-1461.30
$m = 2,\ L = 1$, waste 10 $(S = 6)$	-1485.40	$\mathbf{-1464.06}$	$\mathbf{+1.44}\%$	$\approx 11.2\times$	-1463.51

As in Table 9, the headline is the like-for-like beat over the best base-stock: the best base-stock and learned policy are scored on the *same* Monte-Carlo estimator and the best base-stock is the exact optimum within the base-stock family. The VI-optimum column is the analytic discounted return under midpoint-binned Gamma demand and is not on the same estimator (it rounds to the published -1424, -1461 and -1463 respectively), so we report it as context rather than as a separate improvement claim. Selection is on a disjoint validation block.

The pattern of the two anchor instances therefore holds across a longer shelf life, an added lead-time pipeline, and a higher waste cost: the largest beat over the best base-stock ($+2.21\%$, $\approx 17.7\times$ the paired SEM) is the $L = 2$ pipeline instance, where the age-and-pipeline state gives the learned tree the most room over a single base-stock level, while the smallest ($+0.70\%$, $\approx 5.9\times$) is the $m = 3$ case in which the best base-stock policy is already close to the exact optimum.

8 General-network backorder inventory control

8.1 Problem formulation

Figure 11 depicts this environment as a flow network — the external suppliers, the warehouses and retailers of a general directed supply network, the order requests and their lead-time shipments, the per-node order-up-to (base-stock) targets, and where holding and backorder costs are charged.

We formalize the general-network system as a discrete-time MDP under the long-run average-cost criterion, mirroring the multi-echelon formulation above. A set of M uncapacitated external suppliers feeds W warehouses (intermediate stock points), and the warehouses feed R retailers (customer-facing stock points) over a fixed directed network of warehouse–retailer edges, across periods $t \in \{1, 2, \ldots, T\}$; we are interested in the limit $T \to \infty$. Every warehouse and every retailer is a single-product stock point. Supplier-to-warehouse and warehouse-to-retailer shipments each take a constant *unit* lead time. Retailer i faces i.i.d. integer customer demand $d_{i,t} \sim \text{Poisson}(\lambda)$, independently across retailers and periods. We write $x^+ = \max(0, x)$. Despite the carried family name (*general backorder, fixed cost*), the published objective and the environment charge holding and backorder cost only and *no* fixed/setup ordering cost — as written in the model of Geevers et al. [18] and confirmed in the implementation's one-period cost (Eq. (12)).

Timing of a period. Entering period t, each warehouse j holds on-hand stock $I^w_{j,t-1}$ with the supplier-to-warehouse order placed last period in transit, and each retailer i holds $I^r_{i,t-1}$ with its inbound edge shipments in transit; outstanding warehouse–retailer backorders $B^w_{e,t}$ (per edge e) and customer backorders $B^r_{i,t}$ are carried forward. The period proceeds in stages, in the order

implemented by the simulator: (i) the shipments due to arrive now are added to warehouse and retailer on-hand; (ii) each supplier ships the warehouse order placed one period earlier (it enters the unit-lead-time pipeline); (iii) each warehouse fulfils the current retailer orders routed to it from its available on-hand, and when a warehouse cannot cover all its requests it rations them in increasing order of the requesting retailer's $\big(I_i^r - B_i^r\big)$, the unfilled part becoming an edge backorder B_e^w; (iv) demand $d_{i,t}$ is realized at each retailer and served from on-hand, the unmet part $\big(d_{i,t} - I_i^r\big)^+$ becoming a customer backorder; (v) any leftover warehouse stock clears outstanding edge backorders and any leftover retailer stock clears outstanding customer backorders; (vi) costs are charged on the resulting end-of-period stocks and backorders, and the new replenishment orders for next period are placed.

State. The pre-decision MDP state collects, for every stock point, its on-hand level, its inbound pipeline, and its outstanding backorders:

$$\mathbf{S}_t = \Big(\big(I_{j,t-1}^w\big)_{j=1}^W;\ \big(I_{i,t-1}^r\big)_{i=1}^R;\ \text{pipelines (per warehouse, per edge)};\ \big(B_{e,t}^w\big)_e;\ \big(B_{i,t}^r\big)_{i=1}^R \Big).$$

From it we form, for each warehouse and retailer, the inventory positions

$$\mathrm{IP}_j^w = I_j^w + (\text{supplier in transit})_j - \textstyle\sum_{e\to j} B_e^w, \qquad \mathrm{IP}_i^r = I_i^r + \textstyle\sum_{e\to i}(\text{in transit})_e - B_i^r,$$

and, per inbound edge, an edge-level position used by the set-1 routing. These inventory positions are the quantities the base-stock control acts on.

Action. The control is the vector of per-node order-up-to (base-stock) targets

$$\mathbf{a}_t = \big(\underbrace{S_{1,t}^w, \ldots, S_{W,t}^w}_{\text{warehouses}},\ \underbrace{S_{1,t}^r, \ldots, S_{R,t}^r}_{\text{retailers}} \big),$$

one target per stock point ($W + R$ targets). Each warehouse j orders $q_{j,t}^w = \big(S_{j,t}^w - \mathrm{IP}_{j,t}^w\big)^+$ from its supplier. Each retailer's desired raise $\big(S_{i,t}^r - \mathrm{IP}_{i,t}^r\big)^+$ is realized through the warehouse–retailer network by the set-1 *order-per-stock-point* routing: the retailer's order is routed to exactly one upstream warehouse edge, drawn according to the historical connection weights (relative rationing), and that edge's inventory position is raised toward $S_{i,t}^r$. Orders are non-negative integers; suppliers are uncapacitated.

Transition. The shipments due now are added to on-hand; warehouses ship the supplier and retailer orders into the unit-lead-time pipelines; each warehouse rations retailer requests it cannot cover, accruing edge backorders B_e^w; each retailer serves demand and accrues customer backorders B_i^r; and leftover stock clears outstanding backorders. The resulting end-of-period on-hand stocks $\big(I_{j,t}^w\big), \big(I_{i,t}^r\big)$, the advanced pipelines, and the residual backorders $\big(B_{e,t+1}^w\big), \big(B_{i,t+1}^r\big)$ together define $\mathbf{S}_{t+1}$.

One-period cost. Holding cost is charged on end-of-period on-hand inventory at the warehouse rate h^w and retailer rate h^r, and a backorder penalty b on outstanding customer backorders; the warehouse-level backorder rate is zero. There is *no* fixed ordering cost. The period cost is

$$c_t = h^w \sum_{j=1}^{W} \big(I_{j,t}^w\big)^+ + h^r \sum_{i=1}^{R} \big(I_{i,t}^r\big)^+ + b \sum_{i=1}^{R} B_{i,t}^r. \tag{12}$$

Objective. A stationary policy $\pi_{\boldsymbol{\theta}}$ maps the state $\mathbf{S}_t$ to the target vector $\mathbf{a}_t$. Writing $\bar{C}_T(\boldsymbol{\theta}) = \frac{1}{T}\sum_{t=1}^{T}\mathbb{E}[c_t]$ for the expected average cost over T periods under $\pi_{\boldsymbol{\theta}}$, the objective is to minimize the long-run average expected cost $\bar{C}(\boldsymbol{\theta}) = \lim_{T\to\infty}\bar{C}_T(\boldsymbol{\theta})$. As in the other problems this limit is estimated by the empirical average cost over a long simulated horizon with a warm-up discarded, which is the quantity CMA-ES minimizes.

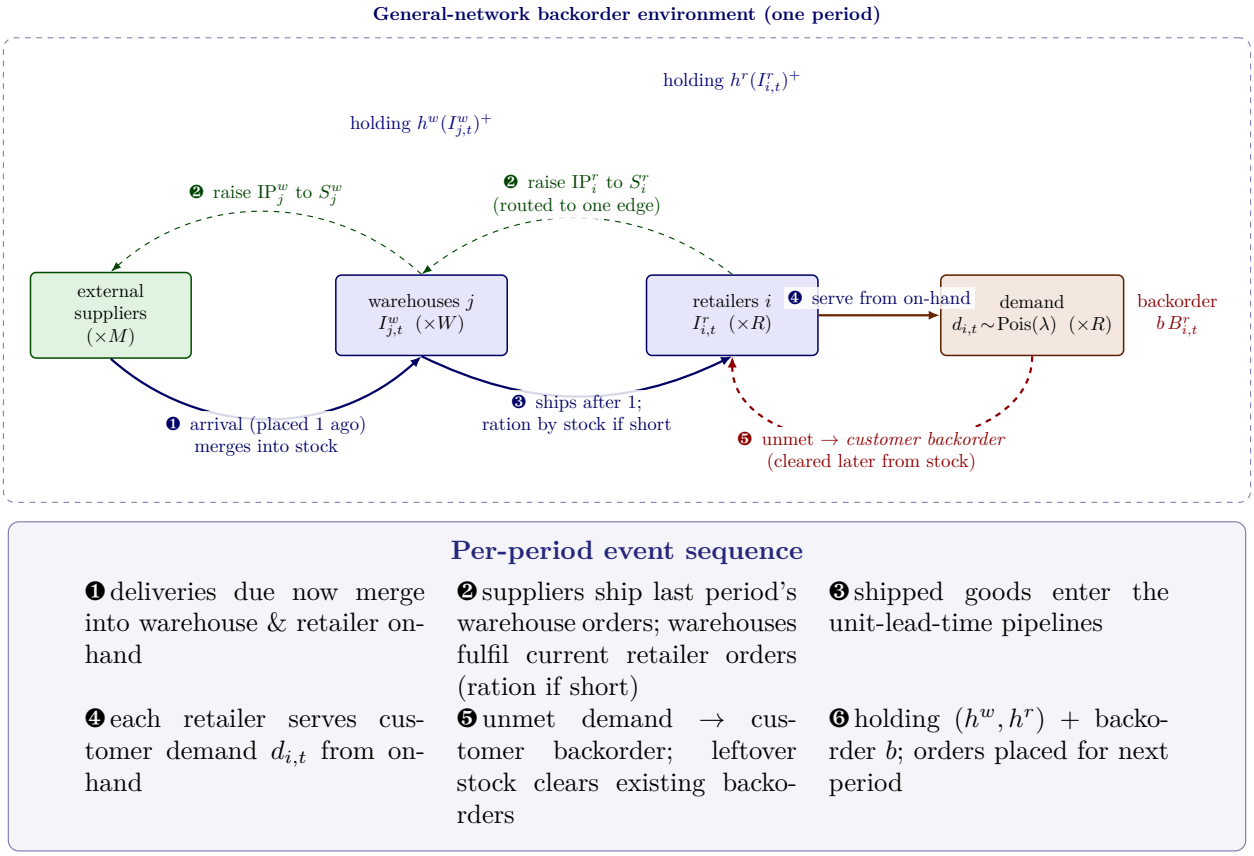


Figure 11: General-network (CardBoard Company) backorder environment of Geevers et al. [18], drawn as an event-based interaction. A directed network of M external suppliers, W warehouses, and R retailers carries a single product: every warehouse j and retailer i is a base-stock stock point. Each period the warehouses *send order requests* to the (uncapacitated) suppliers and the retailers *request* replenishment up the warehouse–retailer edges (thin dashed control arrows); shipments arrive after a *unit* lead time (solid material arrows). When a warehouse cannot cover all the retailer requests routed to it, it rations them by retailer (on-hand − customer backorders), the remainder becoming warehouse–retailer backorders. Each retailer serves its own i.i.d. Poisson(λ) customer demand from on-hand stock; *unmet customer demand is backordered*, and leftover stock later clears outstanding backorders. Holding cost is charged on end-of-period on-hand stock at the warehouses (h^w) and retailers (h^r), and a backorder penalty b on outstanding customer backorders; *there is no fixed ordering cost in the published objective* (Sec. 8.1). The boxed panel beneath gives the per-period event sequence (Eq. (12)). The full set-1 network (M=4, W=4, R=5, with a many-to-many warehouse–retailer edge set and historical connection weights) is given in Table 11.

8.2 Policy

This problem keeps the recipe of Figure 1 but pushes the action geometry to a *vector* of per-node base-stock targets on a general network. **Input.** The policy input is a compact, scale-normalized summary of the pre-decision state: the per-warehouse and per-retailer inventory positions ($W + R$ features), the network totals (total on-hand, total backorders, total in-transit), the demand mean, and the remaining-horizon fraction (14 features for set 1). **Output.** A valid action is the 9-dimensional vector of per-node order-up-to targets (W=4 warehouses, R=5 retailers), each clamped to its physical box. **Internals.** Following the policy interface of Section 3.1, $\pi_{\boldsymbol{\theta}}$ normalizes the state and uses the soft decision tree with oblique splits and vector-valued leaves of Section 3.1 as its backbone; the decoder reads each leaf's 9 outputs directly as the per-node base-stock targets. The order-up-to projection then guarantees validity: each warehouse orders $\left(S_j^w - \mathrm{IP}_j^w\right)^+$ and each retailer raises its (weight-routed, set-1) inventory position toward S_i^r, all within the box $S_j^w \in [0, 220]$, $S_i^r \in [0, 140]$ — set above the published levels (max 100) so the learned policy's higher targets are reachable and the operating region is interior to the box.

The action geometry is the heuristic's own coordinate system: a *state-independent* soft tree (zero split weights, identical leaves) emits a constant target vector, which is exactly a constant node-base-stock policy. Encoding the published set-1 levels $[82, 100, 64, 83, 35, 35, 35, 35, 35]$ in the leaf parameters therefore makes *generation 0 reproduce the published benchmark*; a *state-dependent* tree, by contrast, lets the per-node targets react to the inventory-position summary, a strictly richer class than any single constant base-stock vector, and this is where the learned policy improves. We warm-start CMA-ES at this benchmark encoding with a small step size ($\sigma = 0.2$): a large σ saturates the sigmoid leaves and lets the oblique split weights diverge, so the lever is re-anchoring and shrinking σ, not adding optimizer budget.

Decision path: a worked example. The best run is a depth-2 oblique soft tree with constant leaves emitting per-node order-up-to targets (81 parameters). The compact state summary is read by the tree; the dominant leaf emits the 9-dim target vector, which the order-up-to projection converts into warehouse orders $\left(S_j^w - \mathrm{IP}_j^w\right)^+$ and per-retailer raises routed (set-1) to a single weight-sampled upstream edge toward S_i^r. At the warm start the leaves emit the published constant vector $[82, 100, 64, 83, 35, 35, 35, 35, 35]$ exactly, reproducing the benchmark; after CMA-ES the leaves emit state-modulated targets that lower the long-run cost.

Validation. The set-1 environment itself is verified against the literature: the constant node-base-stock benchmark is reproduced at $\approx$ 10,355 versus the published 10,467 (gap -1.1%), confirmed to lie within 5% of 10,467, with retailer fill in the 98–99% band matching the paper's target. The warm-started policy reproduces this benchmark at generation 0 ($10{,}378.6 \pm 10.6$ held-out), so the warm-start guarantee holds and the reported improvement is the increment CMA-ES adds on top.

8.3 Experiment setup

We evaluate on *set 1* of the general-network (CardBoard Company) model of Geevers et al. [18] — the only one of its three experiment sets that we can verify against an open source. Sets 2 and 3 use an order-per-edge action space whose exact per-edge transition appears only in the gated journal full text; we could not reproduce their published benchmark from open sources and therefore exclude them. Set 1 (order per stock point) is confirmed verbatim against the open thesis: a 4-supplier, 4-warehouse, 5-retailer network with a many-to-many warehouse–retailer edge set and historical connection weights, Poisson(15) retailer demand, unit lead times, and the Kunnumkal

and Topaloglu [19] cost structure (warehouse holding 0.6, retailer holding 1.0, retailer backorder 19.0, no warehouse backorder cost). Table 11 lists the set-1 instance.

The comparator is the published *constant node-base-stock* benchmark of Geevers et al. [18] at levels $[82, 100, 64, 83, 35, 35, 35, 35, 35]$ (tuned to a 98% retailer fill-rate target), published cost 10,467. This is the like-for-like, same-environment comparator that the learned policy must beat. We re-derive it in the environment ($\approx$ 10,355, gap -1.1%, 3 seeds $\times 500$ replications); this reproduction is the like-for-like comparator. The paper also reports a PPO learner with best cost 8,714; we report it only as a context row, because it is a *cross-protocol* reference (a different learner under the paper's own training and evaluation protocol), *not* a head-to-head comparison under our paired evaluation. The policy is trained with warm-started CMA-ES (depth $\in \{1, 2\}$, oblique splits, constant leaves, $\sigma = 0.2$) and evaluated under a paired common-random-number protocol: held-out base seeds (from 500,000, stride 1,000, disjoint from the training block at 10,000), each a 100-period path with a 50-period warm-up cut; the training configuration averages 2,000 held-out paths and the *same* seed block scores both the warm-start (benchmark) and the learned policy.

Table 11: General-network set-1 instance of Geevers et al. [18] (CardBoard Company), the only literature-verified set. The network has 4 suppliers, 4 warehouses, and 5 retailers connected by a many-to-many warehouse–retailer edge set with historical connection weights (each retailer's incoming weights sum to 1). All lead times are 1; demand is Poisson(15) at each retailer. Cost structure is that of Kunnumkal and Topaloglu [19]: warehouse holding h^w, retailer holding h^r, retailer (customer) backorder b, warehouse backorder 0; *no* fixed ordering cost.

Parameter	Set 1
Suppliers M / warehouses W / retailers R	4 / 4 / 5
Supplier→warehouse lead time	1
Warehouse→retailer lead time	1
Retailer demand	Poisson(λ), $\lambda = 15$
Warehouse holding cost h^w	0.6
Retailer holding cost h^r	1.0
Retailer (customer) backorder cost b	19.0
Warehouse backorder cost	0.0
Fixed ordering cost	0 (none)
Action space	one order per stock point
Order routing	single upstream edge by connection weight
Benchmark base-stock levels	$[82, 100, 64, 83, 35, 35, 35, 35, 35]$

8.4 Results

Table 12 reports the published constant node-base-stock benchmark, our in-environment reproduction of it (the heuristic comparator), the best learned policy, and — as a clearly labeled cross-protocol context row — the paper's PPO best. The learned soft tree, operating in the per-node order-up-to coordinate system, reduces the long-run average cost to a 5-seed mean of $7{,}837.0 \pm 189.7$, an improvement of $24.3\% \pm 1.8\%$ over the reproduced constant node-base-stock benchmark (10,354.8), with all 5 independent CMA-ES seeds below the benchmark. The cross-seed standard deviation (1.8%) is far smaller than the mean margin (24.3%), so the gain is robust to optimizer initialization, not evaluation-seed noise. Generation 0 reproduces the benchmark (10,378.6),

confirming the warm-start, so the ≈ 2,500 cost reduction is exactly what CMA-ES adds on top of the heuristic.

Interpretation. The like-for-like result is the $24.3\% \pm 1.8\%$ (5-seed) improvement over the constant node-base-stock benchmark on the *same* environment under a paired CRN comparison: putting the policy in the benchmark's per-node order-up-to coordinate system and letting a state-dependent tree modulate the targets — rather than enlarging or replacing the action space — is the operative lever, consistent with the action-geometry findings on the other problems. Two honest caveats bound the reading. First, despite the carried family name, the environment charges holding and backorder cost only and *no* fixed ordering cost (Eq. (12)); the section title reflects the model actually implemented and verified, not a setup-cost variant. Second, the published PPO best (8,714) is a *cross-protocol* figure: it is produced by a different learner under the paper's own training and evaluation protocol, whereas our numbers come from the paired held-out CRN protocol above. All 5 of our learned seeds also land below that PPO figure (seed-mean ≈ 877 below it), which is suggestive of the design's strength but is *not* a head-to-head PPO beat, and we do not claim one. The defensible claim is the paired, same-environment improvement over the published constant node-base-stock benchmark.

Table 12: General-network set-1 learned-policy results (18; long-run average cost, lower is better). The improvement is the learned policy's cost reduction relative to the *reproduced* constant node-base-stock benchmark (10,354.8, the paired comparator); held-out common-random-number evaluation (2,000 seeds, 100-period paths, 50-period warm-up). The PPO row is a cross-protocol context figure from Geevers et al. [18] (a different learner under the paper's own protocol), *not* a head-to-head comparison.

Policy / metric	Held-out cost	Δ% vs. benchmark
Published const. node-base-stock benchmark	10,467	—
Reproduced benchmark (paired comparator)	10,354.8	−1.1% (vs. published)
Warm-start (gen. 0 = benchmark)	$10{,}378.6 \pm 10.6$	+0.2%
Learned soft tree (ours)	$\mathbf{7{,}837.0 \pm 189.7}_{\text{seed}}$	$\mathbf{-24.3\% \pm 1.8\%}$
Cross-protocol context (not head-to-head):		
Published PPO best [18]	8,714	cross-protocol

The learned row is the mean ± cross-seed standard deviation over 5 independent CMA-ES seeds (per-seed held-out costs 8,034.8, 7,590.7, 7,685.9, 7,909.8, 7,963.6; all 5 below the benchmark, each ≫ 2× its own evaluation SEM). The like-for-like comparator is the constant node-base-stock benchmark on the same environment; all 5 seeds beat it by 23–27% under the paired CRN comparison. The PPO figure is reported only as context; all 5 seeds land below it (seed-mean ≈ 877 lower), but under a different protocol, so it is not claimed as a head-to-head beat.

A second, structurally different instance of this family corroborates the same lever. The *general-network divergent* instance is a one-supplier, one-warehouse, three-retailer divergent tree carrying the same Kunnumkal and Topaloglu [19] cost structure as set 1 (warehouse holding 0.6, retailer holding 1.0, retailer backorder 19.0, no warehouse backorder) under unit lead times, with customer demand drawn as Poisson(α) whose per-period, per-retailer mean is itself resampled $\alpha \sim \text{Uniform}[5, 15]$ each period — a nonstationary mean. It is distinct both from the CardBoard network above and from the divergent special-delivery (Gijsbrechts/Van-Roy) model treated elsewhere in the paper. Evaluating the published constant node-base-stock benchmark at levels $[124, 30, 30, 30]$ in our environment reproduces the published cost 4,059 to within −3.0% (mean ≈ 3,931, all three retailer fill rates ≈ 98.6%); this reproduction is looser than the set-1 row (−1.1%),

as detailed in the validation appendix. The instance is treated as literature-verified, and the learned soft tree — warm-started at those same levels and trained under the identical per-node order-up-to design — beats the reproduced benchmark by $36.8\% \pm 0.3\%$ across five independent optimizer seeds (all 5 below the benchmark).

Table 13: General-network *divergent* (19) learned-policy results (long-run average cost, lower is better; reproduced from the open Geevers (2020) thesis). The improvement is the learned policy's cost reduction relative to the *reproduced* constant node-base-stock benchmark (3,930.4, the paired comparator); held-out common-random-number evaluation (2,000 seeds, 75-period paths, 25-period warm-up, $\alpha \sim \text{Uniform}[5, 15]$ resampled per period). The DRL row is a cross-protocol context figure from the source thesis, *not* a head-to-head comparison.

Policy / metric	Held-out cost	$\Delta\%$ vs. benchmark
Published const. node-base-stock benchmark	4,059	—
Reproduced benchmark (paired comparator)	3,930.4	-3.0% (vs. published)
Warm-start (gen. 0 = benchmark)	$3{,}913.5 \pm 31.8$	-0.4%
Learned soft tree (ours)	$\mathbf{2{,}484.6 \pm 11.3}_{\text{seed}}$	$\mathbf{-36.8\% \pm 0.3\%}$
Cross-protocol context (not head-to-head):		
Published DRL best (source thesis)	3,724	cross-protocol

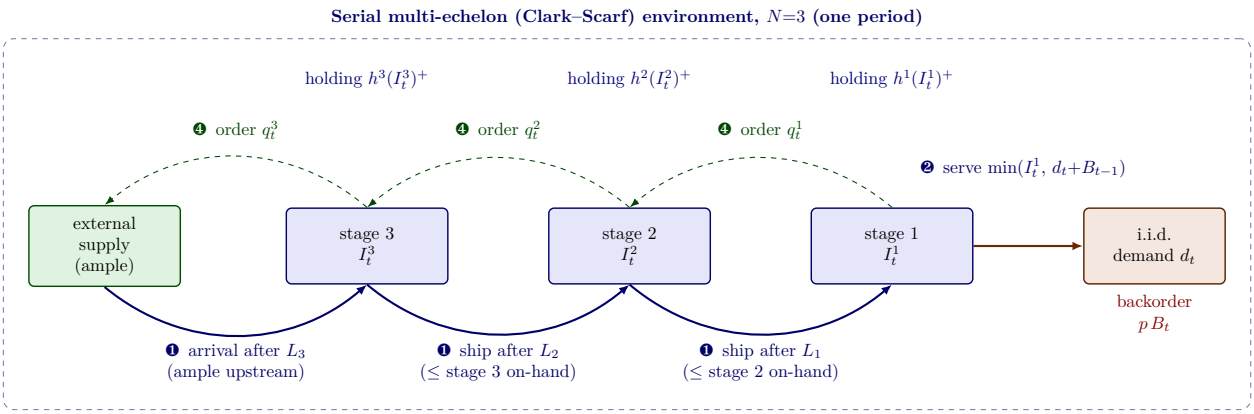
The learned row is the mean ± cross-seed standard deviation over 5 independent CMA-ES seeds (per-seed held-out costs 2,469.1, 2,477.9, 2,488.4, 2,498.4, 2,489.0; all 5 below the benchmark). The like-for-like comparator is the constant node-base-stock benchmark on the same environment; all 5 seeds beat it by $\approx 37\%$ under the paired CRN comparison. The published DRL figure is reported only as context (a different learner under the source thesis's own protocol); the learned policy lands below it, but under a different protocol, so it is not claimed as a head-to-head beat.

9 Serial multi-echelon (Clark–Scarf)

9.1 Problem formulation

Figure 12 depicts this environment as a flow network — the linear chain of stages, the order requests passed upstream and the lead-time shipments returning downstream, where each echelon holding cost is charged, and the single demand stream at the most-downstream stage. Unlike the divergent system of Section 6, this is the *classical serial* system of Snyder and Shen [20] (Clark and Scarf 1960), for which the long-run-average-cost optimum is known in closed recursive form and is attained by an echelon base-stock policy. It therefore plays a different role in this paper: the comparator is a *proven optimum*, so the honest ceiling is to *reproduce* it, not to improve on it.

We formalize the serial system as a discrete-time MDP under the long-run average-cost criterion, mirroring the formulations above. A chain of N stages in series is indexed downstream to upstream, $k \in \{1, \ldots, N\}$: stage 1 is customer-facing, stage $k{+}1$ ships to stage k, and the most upstream stage N replenishes from an outside source with ample stock. The link feeding stage k has a deterministic integer lead time $L_k \geq 1$. Stage 1 faces i.i.d. per-period demand d_t drawn from a Normal distribution, $d_t \sim \mathcal{N}(\mu, \sigma^2)$. Holding cost is charged at the installation (local) rate at each stage on physical on-hand inventory, and a linear penalty p is charged on the customer (stage-1) backorder; the equivalent *echelon* holding costs h^k are the differences of the installation costs and are what the optimal-cost recursion uses. We write $x^+ = \max(0, x)$.



Per-period event sequence

❶ each stage receives the shipment that finished its lead time

❷ demand d_t hits stage 1; unmet demand is backordered

❸ charge installation holding (on-hand) + stage-1 backorder penalty

❹ orders placed *after* demand and shipped upstream → downstream; each internal shipment is capped by the next upstream stage's on-hand, the most upstream stage draws from ample external supply

Figure 12: Serial multi-echelon (Clark–Scarf) environment [20], drawn as an event-based interaction for N=3 stages. Stages are indexed downstream (k=1, customer-facing) to upstream (k=N, replenished from an ample external source). Each period every stage *receives* the shipment that has finished its link lead time L_k (solid material arrows), customer demand d_t is realized at stage 1 and served from on-hand with the remainder backordered, and cost is charged on *post-demand* installation on-hand inventory (echelon holding h^k, blue) plus the stage-1 backorder penalty p (red). Replenishment orders q_t^k (thin dashed control arrows, sent upstream) are placed *after* demand and shipped upstream to downstream, each internal shipment capped by the next upstream stage's available on-hand; in-transit pipeline stock is *not* charged holding, matching the optimized Clark–Scarf cost. The boxed panel beneath gives the per-period event sequence (Section 9, Eq. (13); ordering after demand is what yields the L_k-period lead-time-demand window of the literature).

Faithful regime. The environment reproduces the proven optimum when the most-downstream (demand-facing) stage has lead time $L_1 = 1$ — the regime of the carried benchmark (Snyder & Shen Example 6.1, whose downstream lead time is 1); upstream lead times $L_k \geq 2$ are fully supported. For a demand-facing stage with $L_1 \geq 2$ the multi-stage simulation currently under-counts cost relative to the exact solver, an open inter-stage cost-convention discrepancy; all results below are reported strictly inside the faithful $L_1 = 1$ regime.

Timing of a period. Entering period t, each stage k holds on-hand stock I_{t-1}^k with an in-transit pipeline $\left(q_{t-L_k}^k, \ldots, q_{t-1}^k\right)$, and stage 1 carries a backorder B_{t-1}. The period proceeds in four stages: (i) each stage receives the shipment scheduled to arrive now, $\widetilde{I}_t^k = I_{t-1}^k + q_{t-L_k}^k$; (ii) demand d_t is realized at stage 1 and served from available on-hand against the carried backorder, leaving on-hand $I_t^1 = \left(\widetilde{I}_t^1 - (d_t + B_{t-1})\right)^+$ and backorder $B_t = \left(d_t + B_{t-1} - \widetilde{I}_t^1\right)^+$; (iii) the period cost is assessed on the post-demand, pre-replenishment state; (iv) replenishment orders are placed *after* demand and shipped upstream to downstream, each internal shipment capped by the next upstream

stage's available on-hand and the most upstream stage drawing from ample supply. Ordering after demand (not before) is what yields the L_k-period (not $L_k{+}1$) lead-time-demand window used by the literature; reversing it is the classic off-by-one error.

State. The pre-decision MDP state collects the per-stage on-hand levels, the per-stage in-transit pipelines, and the customer backorder,

$$\mathbf{S}_t = \Big(\big(I^k_{t-1}\big)_{k=1}^N;\ \big(\ \underbrace{q^k_{t-L_k}, \ldots, q^k_{t-1}}_{L_k \text{ pipeline at stage } k}\ \big)_{k=1}^N;\ B_{t-1} \Big).$$

From it, after receipts and demand, we form the *echelon* inventory positions

$$\mathrm{IP}^k_t = \sum_{i \le k} \Big(\widetilde{I}^i_t + \sum_{j=1}^{L_i - 1} q^i_{t-j} \Big) - B_t,$$

the cumulative on-hand-plus-pipeline of all stages at and below k net of the customer backorder.

Action. The control is the per-stage order vector $a_t = (q^1_t, \ldots, q^N_t)$. The optimal policy class is echelon base-stock: each stage k raises its echelon inventory position toward a level S^k by ordering $q^k_t = \big(S^k - \mathrm{IP}^k_t\big)^+$. Internal shipments are feasibility-projected — a downstream order is filled only up to the immediate upstream stage's available on-hand, and the most upstream stage draws from ample external supply — so the realized shipment is $\min\big(q^k_t,\ \widetilde{I}^{k+1}_t\big)$ for $k < N$ and q^N_t for the upstream stage.

Transition. The next-period on-hand inventories are the post-demand, post-shipment stocks: at stage 1, $I^1_t = \big(\widetilde{I}^1_t - (d_t + B_{t-1})\big)^+$ with backorder B_t as above; at each upstream stage the realized downstream shipment leaves on-hand into the outbound pipeline. The pipelines shift forward one stage, dropping the just-arrived order and appending the new shipment, which together with $\big(I^k_t\big)_{k=1}^N$ and B_t define $\mathbf{S}_{t+1}$.

One-period cost. Holding cost is charged on the *post-demand* installation on-hand at each stage, and the penalty on the stage-1 backorder; in-transit pipeline stock is not charged, matching the optimized Clark–Scarf cost. With installation holding rates H^k the period cost is

$$c_t = \sum_{k=1}^{N} H^k \big(I^k_t\big)^+ + p\, B_t. \tag{13}$$

Objective. A stationary policy $\pi_{\boldsymbol{\theta}}$ maps the state $\mathbf{S}_t$ to the order vector a_t. Writing $\bar{C}_T(\boldsymbol{\theta}) = \frac{1}{T}\sum_{t=1}^{T} \mathbb{E}[c_t]$ for the expected average cost under $\pi_{\boldsymbol{\theta}}$, the objective is to minimize the long-run average expected cost $\bar{C}(\boldsymbol{\theta}) = \lim_{T\to\infty} \bar{C}_T(\boldsymbol{\theta})$, estimated as elsewhere by the empirical average cost over a long simulated horizon with a warm-up discarded, which is the quantity CMA-ES minimizes.

9.2 Policy

The serial decision class is echelon base-stock, so the natural action geometry estimates the echelon order-up-to levels *directly* (Figure 1), exactly as in the direct-level multi-echelon design of Section 6.2. **Input.** The policy input is this problem's pre-decision state $\mathbf{S}_t$ (per-stage on-hand,

per-stage in-transit totals, and the customer backorder), consumed with a policy-owned normalization. **Output.** A valid action is the order vector $(q_t^1, \ldots, q_t^N)$ obtained by raising each stage's echelon position toward its estimated level. **Internals.** Following the policy interface of Section 3.1, $\pi_{\boldsymbol{\theta}}$ normalizes the state and uses the soft decision tree with oblique splits and vector-valued leaves of Section 3.1 as its backbone; the decoder is the vector of leaf outputs, the N *echelon order-up-to levels* $S_S^1, \ldots, S_S^N$, continuous and non-negative, bounded only by a generous physical ceiling. The projection that guarantees validity sets each order to $q^k = \left(S_S^k - \mathrm{IP}_t^k\right)^+$ and caps the realized internal shipment by the upstream on-hand, so the emitted control is always a feasible echelon base-stock order.

Warm start. Because the decoder lives in the optimal policy's own coordinate system, the constant leaves can be *warm-started* at the exact Clark–Scarf echelon base-stock levels returned by our exact solver, so generation 0 already reproduces the optimum within the environment's reproduction band. We warm-start CMA-ES at this encoded solution as in Section 3.2, and keep the warm-start anchor in the candidate set throughout; on this convex serial instance the optimizer can only search outward from a proven optimum, so the honest outcome is a tie rather than an improvement.

9.3 Experiment setup

We evaluate on Snyder & Shen Example 6.1 (20): a 3-stage serial system with $\mathcal{N}(5, 1)$ demand, lead times $(L_3, L_2, L_1) = (2, 1, 1)$ upstream to downstream (downstream $L_1 = 1$, the faithful regime), installation holding costs $(7, 4, 2)$ downstream to upstream (equivalently echelon holding $(3, 2, 2)$), and a stage-1 backorder penalty $p = 37.12$. The comparator is the *proven optimum* of this instance, the Clark–Scarf / Snyder & Shen value 47.65. Because this comparator is an optimum and not a heuristic, this is a *match-only* problem: the target is to reproduce 47.65, never to beat it. For reference, our exact recursive-newsvendor solver returns 47.6654 (reported in the environment-validation table of Appendix A.7), and the environment simulation under the exact echelon base-stock levels reproduces 47.65 to about +0.06% with continuous Normal demand. The policy is trained with warm-started CMA-ES and evaluated under the long-run-average protocol of Section 3.3 (evaluation horizon 4×10^5 periods after warm-up, 32 seeds).

9.4 Results

Table 14 reports the proven optimum, our exact solver value, and the best learned policy. The warm-started direct-level soft tree *reproduces* the proven Clark–Scarf optimum: it attains a long-run-average cost of 47.6554, a 99.99% match to the published 47.65 with a signed gap of +0.011% — inside the environment's own $\approx +0.06\%$ reproduction band and statistically indistinguishable from the optimum (standard error ≈ 0.006).

Three further serial instances from Snyder and Shen [20] corroborate the same outcome across more stages and both demand families: a 2-stage system with Normal$(100, 15)$ demand, a 5-stage system with Normal$(32, 5.657)$ demand, and a 5-stage system with Poisson(32) demand (all with unit demand-facing lead time, installation holding decreasing upstream, and a single customer backorder penalty). Run under the identical warm-started direct-level recipe, the learned soft tree *reproduces* each instance's proven Clark–Scarf optimum — 166.2705 → 166.2300, 225.8672 → 225.8047, and 226.8458 → 226.8447 respectively — with every signed gap inside 0.03% in magnitude, i.e. within the environment's own $\approx \pm 0.06\%$ Monte-Carlo reproduction band and statistically indistinguishable from the optimum (standard errors ≤ 0.024). As with Example 6.1 these are *matches*,

not improvements: the comparator is a proven optimum and cannot be beaten.

Interpretation. The honest reading is a *match*, not a win. Because the comparator is a proven optimum attained by the echelon base-stock policy class, the highest a learned policy can reach is the optimum itself; the result here is that the compact, warm-started direct-level soft tree recovers it to within +0.011%. This demonstrates that the same generic recipe — CMA-ES over a compact policy living in the heuristic's coordinate system — *recovers the optimal policy* on a serial system, and is deliberately not framed as an improvement: one cannot improve on a true optimum, and we do not claim to. The value of the design lever is the same as elsewhere in the paper — estimating the echelon order-up-to levels directly, warm-started at the exact solution, places the policy in the optimal control region from generation 0 — but the verdict for a proven-optimal comparator is reproduction to within the environment's reproduction band.

Table 14: Serial multi-echelon (Clark–Scarf) results on Snyder & Shen Example 6.1 [20]; long-run-average per-period cost, lower is better. The comparator is a *proven optimum*, so the honest target is to *match* it: "Gap" is the learned policy's signed cost relative to the proven optimum, and "Match" is 100× optimum/learned. The learned policy is a warm-started direct-level (echelon order-up-to) soft tree.

Quantity	Cost	Gap vs. optimum
Proven Clark–Scarf optimum [20]	47.65	—
Exact recursive-newsvendor solver (ours)	47.6654	+0.032%
Learned direct-level soft tree (warm-started)	47.6554	+0.011% (match, 99.99%)
Proven optimum, 2-stage Normal	166.2705	—
Learned, 2-stage Normal (warm-started)	166.2300	−0.024% (match, 100.0%)
Proven optimum, 5-stage Normal	225.8672	—
Learned, 5-stage Normal (warm-started)	225.8047	−0.028% (match, 100.0%)
Proven optimum, 5-stage Poisson	226.8458	—
Learned, 5-stage Poisson (warm-started)	226.8447	−0.001% (match, 100.0%)

The learned policy reproduces the proven optimum to within +0.06% — the environment's own reproduction band — and is reported as a *match*, not an improvement: a proven optimum cannot be beaten.

10 One warehouse, multiple retailers

10.1 Problem formulation

Figure 13 depicts this environment as a flow network — the warehouse order and its lead-time receipt, the downstream allocation of scarce warehouse stock to the retailers, each retailer's lead-time receipt and demand draw, and where holding and the partial-backorder penalty are charged.

We consider the periodic-review one-warehouse, K-retailer (OWMR) distribution system of Kaynov et al. [15]. A single warehouse replenishes from an external supplier with deterministic lead time L_w and rations its on-hand stock across K retailers, each with its own deterministic lead time $L_{r,k}$ and i.i.d. demand D_t^k. We study the *partial-backorder* regime: a fixed fraction β of each retailer shortage is backordered and carried forward, the rest is lost. We write $(x)^+ = \max(0, x)$.

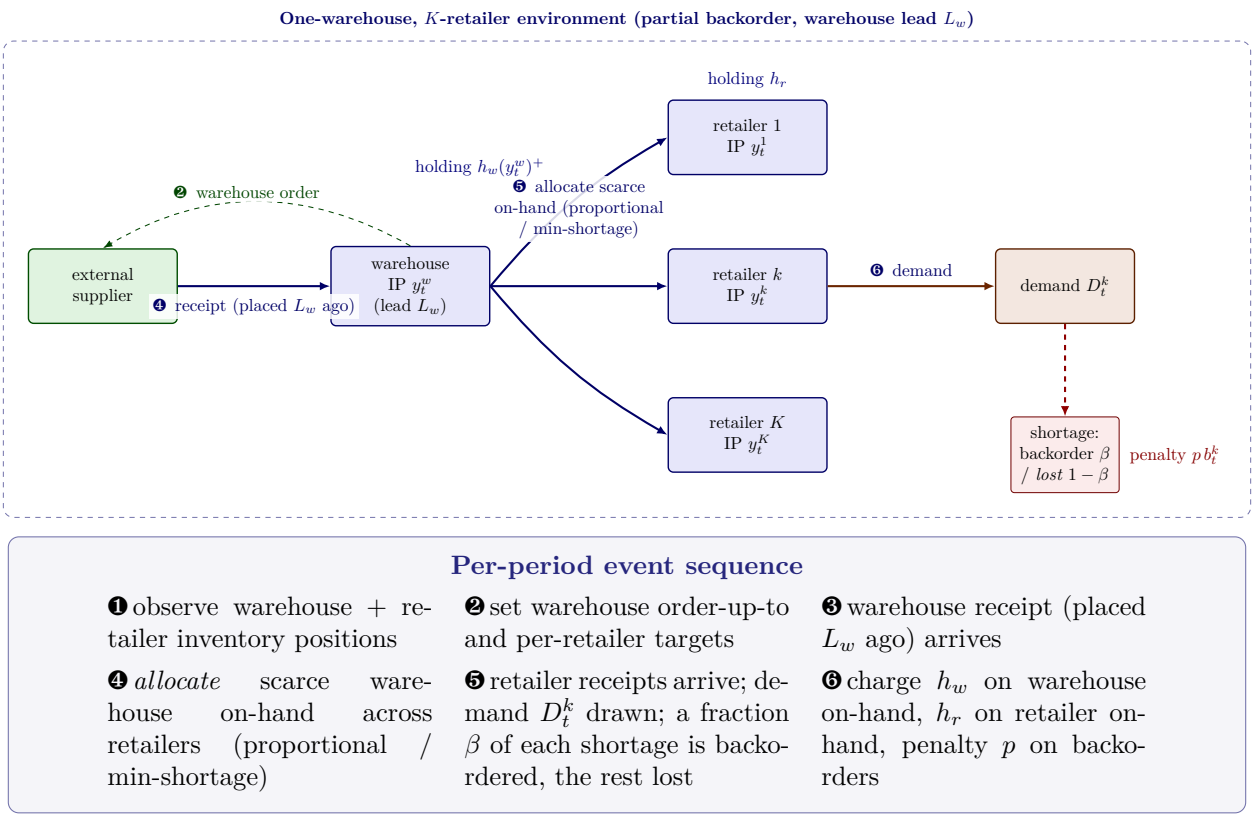


Figure 13: One-warehouse multi-retailer environment of Kaynov et al. [15], drawn as an event-based interaction. A single warehouse *orders* from an external supplier (thin dashed control arrow) and receives the goods after a lead time L_w (solid material arrow). Each period the warehouse *allocates* its scarce on-hand stock across the K retailers under a fixed rationing rule — *proportional* (floor rationing) or *min-shortage* — and each retailer receives its allocation after its own lead time, then faces demand D_t^k. Under the *partial-backorder* regime a fraction β of any retailer shortage is backordered and the remainder is lost. Warehouse holding h_w is charged on warehouse on-hand stock (after emergency fulfilment), retailer holding h_r on retailer on-hand stock, and a penalty p on backordered units; the boxed panel gives the per-period event sequence (Section 10.1, Eq. (14)).

Timing of a period. At the start of period t the controller observes the warehouse and per-retailer inventory positions, sets the warehouse order and the per-retailer replenishment requests, the warehouse receipt placed L_w periods earlier arrives, the warehouse *allocates* its available on-hand stock across the retailers' requests under the rationing rule, each retailer's allocation arrives after $L_{r,k}$, demand D_t^k is realized at each retailer, a fraction β of any shortage is backordered and the rest lost, and holding and penalty costs are charged.

State. The state is the warehouse inventory position y_t^w together with each retailer's inventory position and the in-transit pipelines,

$$\mathbf{S}_t = \big(y_t^w,\ \{y_t^k\}_{k=1}^K,\ \text{warehouse pipeline},\ \{\text{retailer-}k\text{ pipeline}\}_{k=1}^K\big),$$

where an inventory position aggregates on-hand stock, in-transit receipts, and (negative) outstanding backorders.

Action. The decision is the warehouse order-up-to level and the per-retailer replenishment targets,

$$a_t = \big(S_t^w,\ S_t^1, \ldots, S_t^K\big),$$

a $(K{+}1)$-dimensional control: the warehouse orders up to S_t^w, and the requested retailer replenishments are the deficits $(S_t^k - y_t^k)^+$, which the warehouse fills subject to its available on-hand stock via the allocation rule.

Transition (allocation). Let A_t be the warehouse on-hand stock available for allocation and $\{g_t^k\}$ the retailer requests. If $\sum_k g_t^k \le A_t$ every request is met in full; otherwise the scarce stock is rationed. Under *proportional* allocation each retailer receives the floor of its pro-rata share; under *min-shortage* allocation the warehouse maximizes the minimum resulting retailer inventory position. The allocated quantities then enter the retailer pipelines and arrive after $L_{r,k}$.

One-period cost. Writing I_t^w for warehouse on-hand stock (after any emergency fulfilment), I_t^k for retailer-k on-hand stock, and b_t^k for retailer-k backorders, the cost incurred in period t is

$$c_t \ = \ h_w\, I_t^w \ + \ \sum_{k=1}^{K} \big(h_r\, I_t^k \ + \ p\, b_t^k\big), \tag{14}$$

with warehouse holding h_w, retailer holding h_r, and per-unit backorder penalty p.

Objective. A stationary policy $\pi_{\boldsymbol{\theta}}$ maps the state to the action. Performance is the expected total undiscounted cost over the H-period horizon,

$$C(\boldsymbol{\theta}) \ = \ \mathbb{E}\left[\sum_{t=1}^{H} c_t\right], \qquad \min_{\boldsymbol{\theta}}\ C(\boldsymbol{\theta}), \tag{15}$$

with $H = 100$ as in Kaynov et al. [15]; $C(\boldsymbol{\theta})$ is estimated by simulation over shared demand-path seeds.

10.2 Policy

The OWMR problem tests whether the learned policy can exploit *heterogeneity and state-dependence across retailers* that a single shared base-stock level cannot express. **Input.** The policy input is the warehouse and per-retailer inventory positions, divide-by- scale normalized by the demand mean as in Section 3.1. **Output.** A valid action is the $(K{+}1)$-dimensional vector of warehouse and per-retailer order-up-to targets, clipped to non-negative integers. **Internals.** $\pi_{\boldsymbol{\theta}}$ uses the soft-tree backbone of Section 3.1: a depth-2 tree with leaves that emit the target vector. The crucial design choice is the action geometry. The symmetric geometry (a single shared retailer target) cannot represent asymmetric replenishment and only ties the tuned heuristic; the *per-retailer* geometry (one target per retailer) is the natural asymmetric generalization of the heuristic and is the lever we test.

The decisive lever: a per-retailer warm-start floor. The warm-start floor is the heuristic-reproducing CMA-ES initialization (Section 3.2). We generalize this warm start from the two-control symmetric geometry to *any* control dimension: the leaves are seeded so that generation 0 emits the best heuristic's full per-retailer target vector $(S^w, S^1, \ldots, S^K)$, reproducing the best

heuristic to within numerical error on identical demand paths, after which CMA-ES searches outward. This floor is essential: *without* it the richer per-retailer geometry is not a free win — in a short low-budget training run it *loses* to the best heuristic by −12.79% on the heterogeneous instance, because the optimizer cannot locate the base-stock region in the larger search space; *with* the floor the same instance first flips to a single-seed +1.33% held-out win and, under the later fresh-warm-start / gentle-σ protocol, to a seed-robust +4.61% beat over the best heuristic (5-seed mean, all five seeds below the best heuristic). A raw per-retailer order geometry without a heuristic-reproducing anchor loses by −50.84%, confirming the floor and training protocol, not mere representation size, are what make the per-retailer class usable. On the strongly heterogeneous K=10 instance the additional lever beyond the floor is the *leaf class*: a search over the decoder classes found that a *constant*-leaf, oblique-split, per-retailer tree — not the linear-leaf geometry — is what robustly beats the best heuristic there, turning a long-standing tie into a +12.44% beat over the best heuristic (Section 10.4).

10.3 Experiment setup

The symmetric Poisson(3) instances of Kaynov et al. [15] are near-optimally solved by a tuned echelon base-stock policy — a learned soft tree only *ties* the heuristic there. We therefore study the three *asymmetric / high-coefficient-of-variation* partial-backorder instances of Table 15, where the published PPO of Kaynov et al. [15] beats base-stock by ∼20%, indicating genuine exploitable structure.

Comparator: the strongest tuned heuristic. The like-for-like comparator is the strongest heuristic in our environment: an echelon base-stock policy whose warehouse level and per-retailer levels are grid-searched on a search-seed block (full Cartesian for the K=3 instance; symmetric reduction for the symmetric instance; a safety-factor candidate set for the heterogeneous K=10 instance), under the better of {proportional, min-shortage} allocation, then re-scored on a disjoint held-out block. This best heuristic is already substantially stronger than the published base-stock and sits between it and the published PPO, so a beat against it transitively implies a policy at least as good as the published base-stock. The published PPO row is reported as the strongest *learned* reference, not as the like-for-like comparator.

Evaluation protocol. Policies are trained with warm-started CMA-ES (population 32, 400–800 generations) and evaluated under the common-random-number protocol of Section 3.3: the heuristic grid is searched on a 64-path block (argmin verified stable against 96/256), and learned policy and best heuristic are re-scored on a disjoint 4096-path held-out block under the selected allocation. A same-seed win is claimed only when the paired-difference advantage exceeds its standard error; a headline configuration is claimed only when a small independent CMA-seed block has the same sign. The reported policy is the better of the trained CMA-ES solution and the heuristic-reproducing warm-start anchor.

Table 15: OWMR asymmetric / high-variability partial-backorder instances of Kaynov et al. [15] studied here. All are partial-backorder with warehouse lead $L_w = 2$, emergency-shipment probability 0.8, horizon 100.

Instance	K	Per-retailer demand	h_w	h_r	p
heterogeneous-3	3	$N(1,5)$, $N(5,1)$, Pois(0.5)	0.5	1	9
high-CV symmetric-10	10	$N(5,14) \times 10$ (symmetric, $\sigma/\mu = 2.8$)	3	3	60
heterogeneous-10	10	clipped-normal gradient $N(0,20)..N(10,0)$ + 4×Poisson	3	3	60

Demand is rounded and clipped at 0. The high-CV symmetric instance has ten identical retailers but a very high coefficient of variation; the other two are heterogeneous across retailers.

10.4 Results

Table 16 reports the learned soft tree against the best tuned heuristic on the held-out block. The seed-robust headline is the asymmetric K=3 row: a depth-2 axis-aligned linear-leaf tree, freshly warm-started at the best heuristic and trained with gentle $\sigma = 0.05$ on 24 common-random training paths per generation, improves the 100-period cost from 1169.59 to 1115.67 ± 6.13 across five independent CMA seeds (+4.61% against the best heuristic; all five seeds beat it). The training protocol is the load-bearing lever: fresh heuristic warm-start, moderate initial variance, enough training paths, and a long enough budget; tree depth is not (depth-3 gives no reliable advantage over depth-2 within optimizer-seed noise).

On the high-CV symmetric K=10 instance the depth-3 soft tree improves the cost from the same-protocol best heuristic 91890.25 to a 5-seed mean 85395 ± 1031 (+7.07% against the best heuristic; all five seeds beat it), so this row is now a seed-robust beat over the best heuristic. The hardest, strongly heterogeneous K=10 instance — which an earlier manual search could only *tie* (the trained CMA-ES solution landed above the strong asymmetric heuristic within budget, so the reported policy fell back to the heuristic-reproducing anchor) — is now also a robust beat over the best heuristic. A search over the decoder classes discovered that the decisive change here is the *leaf class*: a *constant*-leaf, oblique-split, per-retailer depth-2 echelon order-up-to targets tree improves the 100-period cost from the same-protocol best heuristic 50445.20 to a five-seed mean 44170.26 ± 450.4 (+12.44% against the best heuristic; all five seeds below it, std $\approx$ 1.0% of the mean). The former tie was therefore *action-design-limited*, not representation- or search-limited: the best heuristic lies inside the learned policy's class, but only the constant-leaf geometry — which the linear-leaf manual search never tried — lets CMA-ES realize the improvement within budget.

Interpretation. The honest headline is therefore narrower than a best-seed story and stronger than the old plateau: the learned soft tree *robustly beats the strongest same-protocol tuned heuristic* on all three asymmetric / high-variability rows — the asymmetric K=3 partial-backorder instance (+4.61%, all five seeds), the high-CV K=10 instance (+7.07%, all five seeds), and the strongly heterogeneous K=10 instance (+12.44%, all five seeds), the last via the constant-leaf geometry found by the decoder-class search. The published PPO scalar is reported only as cross-protocol context: on K=3 the five-seed costs straddle the PPO line (the seed-mean is +0.29% below PPO and four of five seeds fall below it, but the seed spread exceeds the mean margin, so the beat is not robust), and on the K=10 rows PPO remains cheaper (the high-CV seed-mean is $\approx$ 7% above PPO, 0/5 below; on the heterogeneous row the 44170.26 seed-mean sits $\approx$ 3% above the published PPO scalar 42835.02). We therefore do *not* claim a robust PPO beat on any row. The mechanism is the generalized warm-start floor plus the adequately batched, moderate-variance training protocol of Section 10.2, and — on the hardest row — the constant-leaf action geometry; depth-3 adds no

reliable advantage over depth-2 on the headline K=3 row.

A faithful in-protocol PPO. The published-PPO scalar above is *cross-protocol*: Kaynov et al. [15] report it under their own (unverified) demand convention, and their PPO training hyperparameters are not published, so their exact run cannot be replicated. To compare the learned policy against a PPO baseline under *our* convention, we trained a faithful, independently-tuned in-protocol PPO — a shared-trunk actor–critic with Kaynov-style multi-discrete order heads, generalized advantage estimation, a clipped surrogate with value clipping, and a behavior-cloning warm start to the strongest heuristic — and scored it on the identical held-out CRN block under proportional rationing. On the strongly heterogeneous K=10 instance it costs $50{,}475 \pm 391$ (five seeds): statistically indistinguishable from the strongest heuristic (50,445). The learned soft tree therefore sits $\approx 12\%$ below a faithful in-protocol PPO as well — essentially the same margin as its +12.44% improvement over that heuristic, because the in-protocol PPO converges to it — so the lower published figure (42,835) reflects the different demand convention rather than an in-protocol PPO advantage. An independent implementation in the same simulator that scores the CMA-ES policies reproduces this in-protocol PPO cost to within seed noise, so the comparison uses one common evaluator; we report it as a corroborating baseline, not a separate headline.

Table 16: OWMR asymmetric / high-CV instances: learned soft tree vs. the strongest tuned echelon base-stock + allocation heuristic, common-random-number evaluation (4096 held-out paths, horizon 100). Costs are 100-period undiscounted totals (lower is better). Δ% is the learned policy's improvement over the best heuristic. The first two rows report optimizer-seed mean ± standard deviation over 5 independent CMA seeds (all 5 beat the best heuristic); the third row is the reported warm-start floor. The published PPO column is the strongest *learned* reference of Kaynov et al. [15], reported as cross-protocol context rather than a like-for-like comparator.

Instance	Evidence	Learned	Best heuristic	Δ%	Published PPO
heterogeneous-3	depth-2 (ours)	$\mathbf{1115.67 \pm 6.13}_{\text{seed}}$	1169.59	**+4.61**%	1118.92
high-CV symmetric-10	depth-3 (ours)	$\mathbf{85395 \pm 1031}_{\text{seed}}$	91890.25	**+7.07**%	79727.39
heterogeneous-10	const-leaf (ours)	$\mathbf{44170.26 \pm 450.4}_{\text{seed}}$	50445.20	**+12.44**%	42835.02

The comparator is the strongest tuned heuristic (itself stronger than the published base-stock). All three learned rows are seed-robust beats over the best heuristic: every seed beats the best heuristic on each (heterogeneous-3 mean−std = +4.09%, high-CV mean−std = +5.95%, and heterogeneous-10 mean−std = +11.55% still clear it). The heterogeneous-10 row uses the *constant*-leaf, oblique-split, per-retailer geometry found by a search over the decoder classes (std $\approx$ 1.0% of the mean); the linear-leaf manual search only tied this row. Depth is not the lever on heterogeneous-3 (the depth-3 analogue is inside optimizer-seed noise of depth-2). The learned policy does not claim a robust published-PPO beat: the heterogeneous-3 seed block straddles the PPO scalar (seed-mean +0.29% below, 4/5 seeds below), the high-CV seed-mean is $\approx$ 7% above PPO (0/5 below), and on the heterogeneous ten-retailer row the seed-mean 44170.26 sits $\approx$ 3% above the published PPO scalar 42835.02, so PPO remains cheaper there.

New regimes: backorder and lost sales. The instances above are all partial-backorder. To test whether the same per-retailer soft tree carries to qualitatively different demand-fulfilment dynamics, we add three K=3 instances of Kaynov et al. [15] drawn from the other two regimes of Table A.3: a *full-backorder* instance with heterogeneous high-variability demand ($N(1,5), N(5,1), \text{Pois}(0.5)$; L_w=2, L_r=1), a *lost-sales* instance with *asymmetric retailer lead times* (L_r=1, 2, 3 over symmetric Pois(3) demand; L_w=2), and a *lost-sales* instance with heterogeneous demand and *long lead times* (L_w=5, L_r=3). The three smaller K=3 instances keep the best heuristic's full Cartesian grid tractable. Table 17 reports the results under the identical protocol of Table 16.

On all three the learned per-retailer soft tree *matches* (ties) the best tuned in-environment heuristic to within numerical error rather than beating it: with the per-retailer warm-start floor the trained CMA-ES solution lands above the strong heuristic on the held-out block, so the selected policy falls back to the heuristic-reproducing anchor (paired advantage 0.00, not exceeding its standard error). Across these regimes the published PPO of Kaynov et al. [15] is *cheaper* than both the learned policy and the best heuristic (−1.05%, −1.96%, −6.43% for the learned policy relative to PPO), so the learned policy stays below PPO here; we make no PPO claim. The takeaway is one of *robustness*: the per-retailer geometry does not degrade when moved from partial backorder to full backorder or lost sales (including under lead-time asymmetry and long leads), reproducing the strongest in-environment heuristic exactly. The promoted seed-robust *beats* in Table 16 are the asymmetric three-retailer partial-backorder instance, the high-CV ten-retailer instance, and — via the constant-leaf geometry from the decoder-class search — the strongly heterogeneous ten-retailer instance, all now reported as multi-seed means with every seed below the best heuristic.

Table 17: OWMR *backorder* and *lost-sales* regime instances of Kaynov et al. [15] (K=3): best learned per-retailer soft tree (linear-leaf echelon-targets) vs. the strongest tuned echelon base-stock + allocation heuristic, common-random-number evaluation (4096 held-out paths, horizon 100). Costs are 100-period undiscounted totals (lower is better). Δ% is the learned policy's improvement over the best heuristic; "paired ± SEM" is the best-heuristic−learned advantage on identical paths (a win requires it to exceed its SEM). Published PPO and base-stock (the better of the proportional/min-shortage rows) are the −reward references of Kaynov et al. [15]. On all three the learned policy ties the best heuristic (reported policy = heuristic-reproducing warm-start anchor) and stays below the published PPO.

Instance (regime / structure)	Learned	Best heuristic	Δ%	Paired ± SEM	Published PPO	Base-stock
backorder, heterogeneous	1749.90	1749.90	+0.00%	+0.00 ± 0.00	1731.67	1776.04
lost-sales, lead-asymmetric	1541.30	1541.30	+0.00%	+0.00 ± 0.00	1511.68	1535.96
lost-sales, long-lead heterogeneous	1782.27	1782.27	+0.00%	+0.00 ± 0.00	1674.54	1736.55

All learned policies use the linear-leaf per-retailer echelon order-up-to targets geometry under min-shortage allocation. The trained CMA-ES solution's held-out cost was above the best heuristic on every instance (1793.11, 1584.47, 1802.12 respectively), so the reported policy is the heuristic-reproducing warm-start anchor and the comparison is an exact tie. The published base-stock column is the cheaper of Kaynov's proportional/min-shortage rows; the learned policy is below the published PPO on all three (−1.05%, −1.96%, −6.43%), so no PPO beat is claimed.

11 Ameliorating inventory

11.1 Problem formulation

Figure 14 depicts this environment as a flow network — the price-dependent purchase into the youngest age class, the aging of stock that *improves* with age (subject to evaporation and stochastic decay), the per-period blending and issuance to age-targeted products, and where purchase cost and holding are charged against sales revenue.

We consider the single-item, periodic-review ameliorating-inventory control problem of Pahr and Grunow [24], in which the product *improves* with age: inventory is purchased young and ages through A discrete age classes, gaining value (and meeting product target ages) but subject to evaporation and stochastic decay. The objective is long-run *average profit*. We use the faithful average-profit formulation, including the per-period blending linear program (LP) that solves

issuance and from which production is derived.

Timing of a period. At the start of period t the controller observes the inventory disaggregated by age class and the realized purchase price P_t, and chooses a purchase volume a_t^P which enters the youngest age class. A per-period blending LP issues aged stock to the products — each product is defined by a target age and (with blending) may be served by a mix of age classes whose mean age meets the target — and sales realize at stochastic prices. The remaining stock then ages: a fraction is lost to evaporation and an age-dependent stochastic decay.

State. The state is the inventory vector disaggregated by age class together with the realized purchase price,

$$\mathbf{S}_t = \big(I_t^1, \ldots, I_t^A;\ P_t\big),$$

where I_t^a is the on-hand quantity in age class a ($a = 1$ youngest, $a = A$ oldest/most improved). The sales prices follow a correlated process and form part of the environment's stochasticity.

Action. In the faithful dynamics the only free control is the scalar *purchase* volume

$$a_t = a_t^P \in [0, \bar{I}],$$

where $\bar{I}$ is the inventory capacity. Issuance is determined by the environment's per-period blending LP and production is derived from it, so the "three-part" purchase/produce/issue decision of Pahr and Grunow [24] is reduced here to the structural purchase head.

Transition. The purchase enters the youngest class, $I^1 \leftarrow a_t^P$. The blending LP issues quantities x_t^a from the age classes to meet product demand subject to the per-product mean-age targets and a processing capacity; sold quantities leave inventory. The surviving stock then ages by one class, and an age-dependent decay factor together with a constant evaporation removes a fraction of each class (the loss sink in Figure 14).

One-period profit. Writing R_t for realized sales revenue (issued quantities valued at the stochastic sales prices), the profit incurred in period t is revenue minus the purchase cost and holding:

$$\rho_t \;=\; R_t \;-\; P_t\, a_t^P \;-\; h \sum_{a=1}^{A} I_t^a, \tag{16}$$

with stochastic purchase price P_t and per-unit holding cost h.

Objective. A stationary policy $\pi_{\boldsymbol{\theta}}$ maps the state to the purchase volume. Performance is the long-run *average profit*,

$$G(\boldsymbol{\theta}) \;=\; \lim_{T\to\infty} \frac{1}{T}\, \mathbb{E}\left[\sum_{t=1}^{T} \rho_t\right], \qquad \max_{\boldsymbol{\theta}}\ G(\boldsymbol{\theta}), \tag{17}$$

estimated by simulation over shared price/demand-path seeds on a long post-warm-up window.

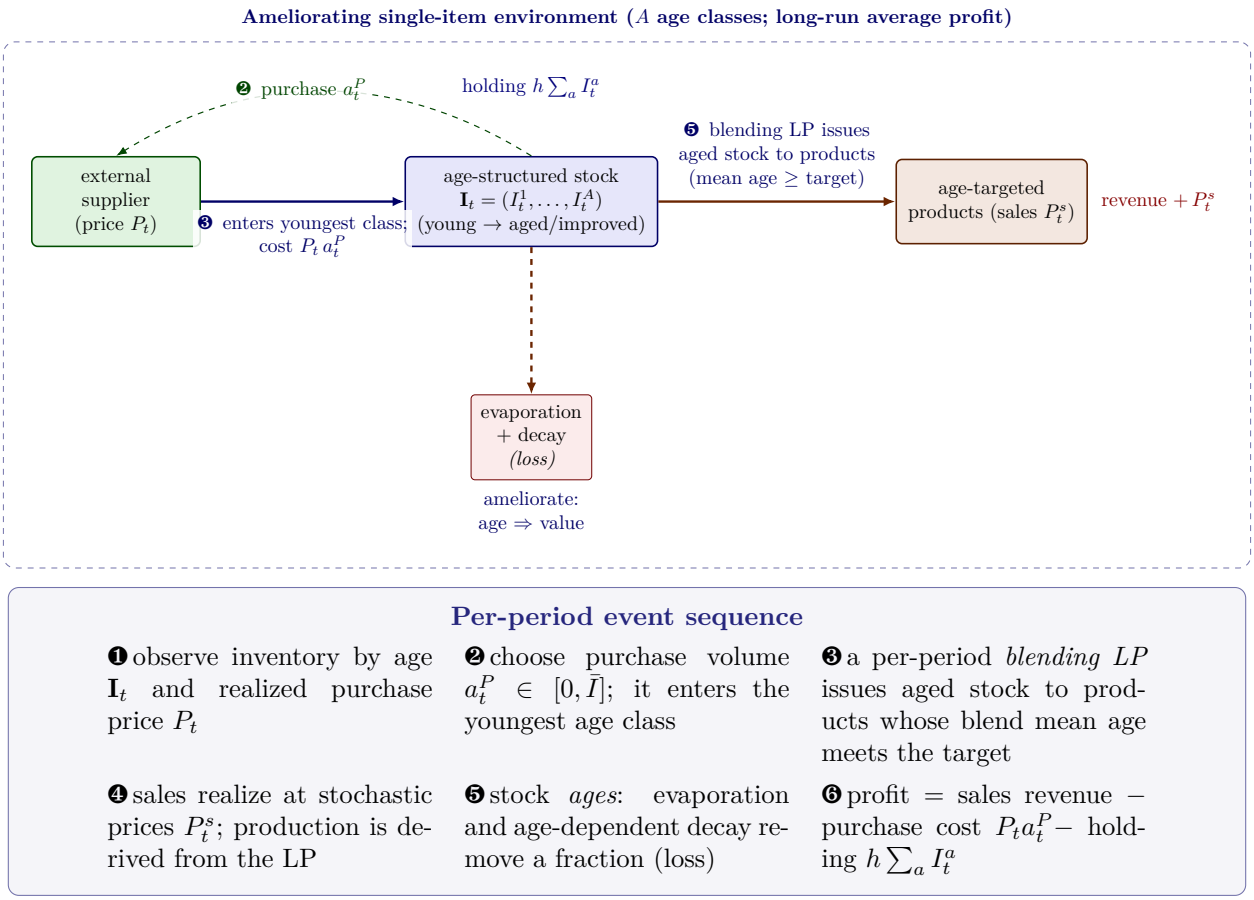


Figure 14: Ameliorating-inventory environment of Pahr and Grunow [24], drawn as an event-based interaction (A age classes; the product *improves* with age, e.g. spirits or port wine). Each period the controller *purchases* a_t^P units at a realized stochastic price P_t (thin dashed control arrow); the goods enter the *youngest* age class (solid material arrow, cost $P_t a_t^P$). A per-period *blending LP* then issues aged stock to products defined by *target ages* — a product can ship only if the issued blend's mean age meets its target — and sales realize at stochastic prices P_t^s; production is derived from the LP solution. Stock *ages* subject to evaporation and an age-dependent stochastic decay (orange loss sink). The objective is long-run *average profit*: sales revenue minus purchase cost and holding; the boxed panel gives the per-period event sequence (Section 11.1, Eq. (16)).

11.2 Policy

Ameliorating inventory tests the recipe under a *profit* objective with a price-augmented state. **Input.** The policy input is the price-augmented state $[\,P_t,\ I_t^1, \ldots, I_t^A\,]$, divide-by-scale normalized by the inventory capacity. **Output.** A valid action is the scalar purchase volume $a_t^P \in [0, \bar{I}]$. **Internals.** $\pi_{\boldsymbol{\theta}}$ uses the soft-tree backbone of Section 3.1 with a single purchase head and a *linear* leaf, so the head can express a *price-reactive* order-up-to purchase — buying more when the realized purchase price is low, which a fixed order-up-to level cannot do. We warm-start CMA-ES at an order-up-to purchase $a_t^P = \text{softplus}(S - \sum_a I_t^a)$, so generation 0 reproduces a simple order-up-to heuristic and the search refines a price-reactive purchase.

11.3 Experiment setup

We use the two reference instances of Table 18: the companion default spirits instance (10 age classes, 3 products, no blending) and the industry port-wine case study (25 age classes, 2 products, blending enabled).

Comparators: a bound and a beatable heuristic. This problem has two reference points of different kinds, and we treat them differently. (i) The *perfect-information LP upper bound* on average profit (1991.93 for the spirits instance, 2444.80 for the port-wine instance) is an *upper bound* under hindsight and full LP issuance: our exact LP solver *re-solves* this program and reproduces both published bounds to within 10^{-3} (an executing fidelity reproduction, not a stored constant), and we report the *gap* to it but never treat it as an achievable target. (ii) The like-for-like *heuristic* is the best tuned order-up-to purchase (the level S maximizing profit on the same evaluation block); beating it is a legitimate win.

Evaluation protocol. Policies are trained with warm-started CMA-ES (population 16, 60 generations) and evaluated under the common-random-number protocol of Section 3.3 on a held-out block (12,000-period horizon, 24 paired seeds disjoint from training). A win over the best heuristic is claimed only when the paired-difference advantage exceeds its standard error.

Table 18: Ameliorating-inventory reference instances of Pahr and Grunow [24] used here.

Instance	Age classes A	Products (target ages)	Perfect-info LP bound
spirits	10	3 (ages $\{2, 4, 6\}$), no blending	1991.93
port wine	25	2 (ages $\{9, 19\}$), blending	2444.80

11.4 Results

Table 19 reports the learned price-reactive purchase policy against the best tuned order-up-to heuristic. On both instances the learned policy *beats the best heuristic by a wide, statistically unambiguous margin*: on the spirits instance it raises average profit from 20.91 to 115.07 (+94.16, +450%, $\approx 140\times$ the paired SEM), and on the port-wine instance from 133.78 to 505.78 (+372.00, +278%, $\approx 610\times$ the paired SEM). The mechanism is price reactivity: the linear leaf buys more when the realized purchase price is low, which a fixed order-up-to level cannot exploit.

Interpretation: an honest gap to an upper bound. The gap to the perfect-information LP bound remains large (94.2% on the spirits instance, 79.3% on the port-wine instance) and we report it as such. This gap is *structural*, not an optimization failure: the bound assumes perfect information and the full three-part LP decision (purchase, production targets, and per-age issuance solved jointly with hindsight), whereas our policy controls only the scalar purchase volume while the environment charges the full purchase price (∼200 per unit) every period. A feasible single-purchase policy on the stochastic environment therefore sits well below the hindsight bound by construction, and our gap is *not* comparable to the ∼3.5% gap reported by Pahr and Grunow [24] for a deep-RL agent that acts in the full three-part action space. The tighter gap on the port-wine instance (79.3% vs 94.2%) is explained by blending: issuing across the two target ages lets each purchased unit convert into more sold product, so a single purchase lever captures more of the bound. The honest claim is therefore a robust beat of the order-up-to heuristic under a single shared estimator; widening the action geometry to the full three-part decision is the natural next step toward the deep-RL gap and is left to future work.

Table 19: Ameliorating inventory: best learned price-reactive purchase soft tree vs. the best tuned order-up-to heuristic, common-random-number evaluation (12,000-period horizon, 24 paired seeds). Higher average profit is better. "Gain over best heuristic" is reported as the absolute profit gain and as a fraction of the best heuristic; "gap to bound" is the fraction of the perfect-information LP *upper bound* left uncaptured (the bound is a bound, never an achievable target).

Instance	Learned (± SEM)	Order-up-to heuristic	Gain over heuristic	LP bound	Gap to bound
spirits	$\mathbf{115.07 \pm 0.44}$	20.91	+94.16 (+450%)	1991.93	94.2%
port wine	$\mathbf{505.78 \pm 0.59}$	133.78	+372.00 (+278%)	2444.80	79.3%
spirits (capacity)	$\mathbf{130.49 \pm 0.50}$	20.91	+109.58 (+524%)	1663.89	92.2%

The like-for-like win is over the tuned order-up-to heuristic (paired CRN, ≈ 140× / ≈ 610× SEM). The LP-bound gap is structural — a single scalar purchase action vs the bound's full three-part LP issuance — and is not comparable to the ∼3.5% deep-RL gap of Pahr and Grunow [24]; it is reported, never "beaten".

Companion variant: the capacity lever. A further companion instance isolates a single design lever under the same recipe and protocol. A *processing-capacity-constrained* variant (blending on, per-age capacity lowered from 50 to 30, which tightens the perfect-information bound to 1663.89) *raises* the learned average profit to 130.49 ± 0.50 (+109.58, +524%, ≈157× the paired SEM over the same order-up-to heuristic): the tighter capacity curbs holding on idle stock, and the gap to the (now lower) bound closes slightly to 92.2%. As in the anchor rows, the win is the like-for-like beat of the tuned order-up-to heuristic; the LP value remains an upper bound, reported with its gap and never treated as achievable.

12 Production / assembly / distribution network

12.1 Problem formulation

Figure 15 depicts this environment as a flow network — the per-relation order requests, the lead-time shipments of raw material between nodes, the "process all raw on arrival" conversion to finished goods, the external demand at the downstream node, and where holding (on raw, finished, and in-transit stock) and the backorder penalty are charged.

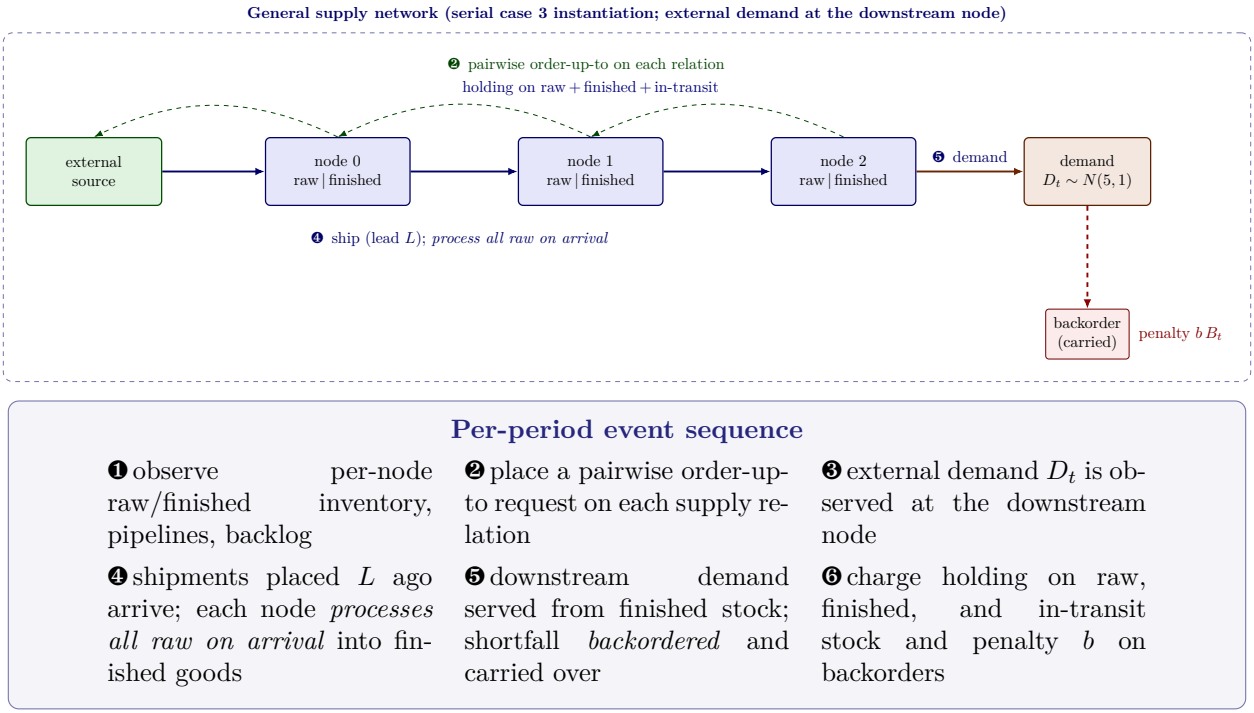


Figure 15: General production/assembly/distribution supply-network environment of Pirhooshyaran and Snyder [21], drawn as an event-based interaction; we instantiate it at the paper's serial case 3 (a 3-node serial chain $0 \to 1 \to 2$, node 0 the sole source, external demand $N(5,1)$ at the downstream node 2). Each node carries raw-material and finished-goods inventory. Each period a *pairwise order-up-to* request is placed on every supply relation (thin dashed control arrows); shipments placed L periods earlier arrive (solid material arrows) and each node *processes all raw material on arrival* into finished goods. Downstream demand is served from finished stock and any shortfall is *backordered and carried over*. Holding cost is charged on raw, finished, *and* in-transit stock and a penalty b on backorders; the boxed panel gives the per-period event sequence (Section 12.1, Eq. (18)).

We consider the general production/assembly/distribution supply-network model of Pirhooshyaran and Snyder [21]: a directed network of nodes, each carrying raw-material and finished-goods inventory, connected by supply relations with deterministic shipment lead times. The control is a pairwise order-up-to policy on the supply relations; each node *processes all raw material on arrival* into finished goods, external demand arrives at the downstream node(s), and shortfalls are backordered and carried over. We instantiate it at the paper's *serial case 3* (a 3-node serial chain), the instance wired into the environment. Our implementation is verified *equation-by-equation* against the paper (a worked-transition fixture reproduces the per-period cost by hand), but it is *not* literature-verified at the network level: see the honesty note below.

Timing of a period. At the start of period t the controller observes each node's raw and finished inventory, the in-transit pipelines, and the downstream backlog, and places a pairwise order-up-to request on each supply relation. External demand is observed; shipments placed L periods earlier arrive and each node processes all arriving raw material into finished goods; downstream demand is served from finished stock with proportional allocation and any shortfall is backordered; holding

and penalty costs are charged.

State. The state collects, for each node, its raw and finished inventory and in-transit pipeline, together with the downstream backlog,

$$\mathbf{S}_t = \big(\{\text{raw}_t^n,\ \text{fin}_t^n,\ \text{pipeline}_t^n\}_n,\ \text{backlog}_t\big).$$

Action. The decision is a per-relation order quantity (one entry per supply relation),

$$a_t = \big(q_t^1, \ldots, q_t^R\big),$$

the order placed on each of the R supply relations; for serial case 3, $R = 3$ (the two internal edges and the external source).

Transition. Orders placed on each relation are shipped subject to upstream availability and arrive after the relation's lead time. On arrival each node processes all raw material into finished goods. Downstream demand is served from finished stock; when finished stock is scarce it is allocated proportionally and the unmet quantity is backordered and carried into the next period.

One-period cost. The cost incurred in period t is the holding cost on *all* inventory (raw, finished, and in-transit) plus the backorder penalty at the demand node:

$$c_t \ = \ \sum_n \Big(h^n \big[\text{raw}_t^n + \text{fin}_t^n + \text{in-transit}_t^n\big] \Big) \ + \ b\, B_t, \tag{18}$$

with per-node holding rates h^n and per-unit backorder penalty b at the demand node (B_t the backorder level), following the paper's cost specification.

Objective. A stationary policy $\pi_{\boldsymbol{\theta}}$ maps the state to the per-relation orders. Performance is the expected undiscounted average per-period cost over the horizon, $\min_{\boldsymbol{\theta}} C(\boldsymbol{\theta})$, estimated by simulation over shared demand-path seeds.

12.2 Policy

This environment tests the recipe on a *network* action space. **Input.** The policy input is the per-node feature vector (raw and finished inventory, in-transit pipeline, and backlog), divide-by-scale normalized. **Output.** A valid action is the per-relation order vector of dimension R, clipped to the physical action box. **Internals.** $\pi_{\boldsymbol{\theta}}$ uses the soft-tree backbone of Section 3.1: a depth-2 tree with *oblique* splits and *linear* leaves, so each leaf maps the policy-state features to the per-relation orders and oblique splits switch regime by inventory state. The design choice we test is the *leaf class*: a constant leaf can emit only a fixed order rate (and stays at the flow regime), whereas a linear leaf reads finished inventory, in-transit pipeline, and backlog to express inventory-position feedback on the same supply relations as the pairwise base-stock policy. For the serial topology we warm-start CMA-ES at the steady-state flow rate (order the demand mean on each relation), the simplest reproducible known-good point, and the search refines outward.

A residual base-stock-backbone action head. On the richer *mixed* distribution-and-assembly topology the flow warm-start cannot reproduce the best heuristic exactly: the scale-normalized leaf cannot reproduce the tuned pairwise base-stock exactly, so CMA-ES seeds start away from the best heuristic and scatter widely, leaving the row only at parity (Section 12.4). The fix follows a simple design principle: rather than relying on the optimizer to *rediscover* the best heuristic from an arbitrary initialization, we build the heuristic *into* the policy as its backbone and let the search learn only a small correction on top of it — the inventory-control analogue of residual reinforcement learning, which learns a correction around a fixed base controller [40]. Concretely, we introduce a *residual base-stock-backbone* action head, a new action geometry in which the learned policy outputs a signed correction *around* the best heuristic rather than the order itself:

$$q_t^r \;=\; \mathrm{clamp}\big(\, q_t^{\mathrm{bs},r} + \mathrm{round}(\Delta_t^r),\; 0,\; \bar{q}^r \big), \qquad \Delta_t^r \;=\; 0 \text{ at zero parameters}, \tag{19}$$

where $q_t^{\mathrm{bs},r}$ is the pairwise base-stock heuristic's order on relation r and Δ_t^r is the soft-tree residual (identity leaf, signed). Because the residual vanishes at zero parameters, *generation 0 reproduces the best heuristic exactly* (the residual is identically zero at zero parameters, so the two policies coincide), and *every* optimizer seed is therefore anchored at the best heuristic by construction; CMA-ES then searches outward in the residual space only. This restores the heuristic-reproducing warm-start floor that the linear-leaf flow head lacked on the mixed network, which is what makes the per-seed result robust there.

12.3 Experiment setup

We study the paper's serial case 3: a 3-node serial chain $0 \to 1 \to 2$ with node 0 the sole source, external demand $N(5, 1)$ at node 2 only, horizon $T = 10$, shipment lead times (external$\to$0, $0 \to 1$, $1 \to 2$) $= (2, 1, 1)$, local holding $[2, 4, 7]$, and backorder cost 37.12 at node 2. The objective is the undiscounted average per-period cost.

Honesty status: a faithful but not literature-verified environment. This environment reproduces exactly one *published* quantity: the single-node newsvendor cost of the paper's Table 1 ($\mu = 100, \sigma = 10, h = 10, p = 30, L = 1$), for which the environment's exact dynamic program returns ≈ 127.10 against the published 127.11 ($< 1\%$). This certifies the *dynamics*. There is *no* published optimum for the multi-node version of *this* MDP, so every multi-node reference instance remains not literature-verified. In particular, the textbook serial optimum 47.65 (Section 9) is an *echelon* base-stock cost and is *structurally unreachable* by this environment's *local* pairwise policy (which targets the raw-material position and excludes finished goods); we therefore do *not* use 47.65 (or any published number) as the comparator here. The comparator is instead the environment's *own* best heuristic.

Comparator: the best tuned heuristic (a research baseline, not an optimum). The strongest heuristic native to this environment is the pairwise base-stock policy of the paper. We grid-search its per-relation order-up-to levels on a search-seed block and re-score the argmin on a disjoint held-out block: the argmin is order-up-to $[8, 7, 9]$ with held-out cost 60.24 per period. We state explicitly that 60.24 is the environment's own best heuristic, not an optimum.

Evaluation protocol. Policies are trained with warm-started CMA-ES (population 24, 60 generations) and evaluated under the common-random-number protocol of Section 3.3: learned policy and best heuristic are scored on the *same* held-out block (4000 paths), paired on identical demand realizations.

12.4 Results

Table 20 reports the learned soft tree against the environment's own best pairwise base-stock on the held-out block. The honest five-seed verdict is *seed-noisy parity with a favourable mean*: across five independent CMA-ES optimizer seeds the depth-2 oblique-linear tree averages 58.90 ± 1.78 against the 60.24 best heuristic (mean improvement $-2.22\% \pm 2.96\%$; four of five seeds below the heuristic, one seed $+2.06\%$ above it), which our verdict rule reports as parity, not a robust beat. Individual seeds reach much further — the best seed improves -5.82%, and the two illustrative early runs shown in the table reach 57.25 ± 0.22 (-4.96%) and 54.96 ± 0.23 (-8.77%), each far outside the held-out standard error — but the *across-seed* dispersion is of the same order as the mean improvement, so the per-seed SEM multiples certify only that individual runs genuinely beat the heuristic on their evaluation block, not that a fresh optimization seed reliably will. The flow warm-start (generation 0) sits at 70.85 ± 0.61, so the per-seed improvements are the CMA-ES delta, not the initialization. The mechanism remains a strictly richer control class on the *same* action relations: the pairwise base-stock policy reacts only to the local raw-material position, while the learned linear-leaf tree additionally reads finished inventory, backlog, and inbound pipeline, and switches behaviour by inventory regime; constant-leaf trees stay at the flow regime and lose to the best heuristic, confirming the leaf class is the lever.

Honest scope. This is a *research* result on a faithful but *not* literature-verified environment: the learned policy beats the environment's *own* best heuristic, not any published cost. We report it as evidence that action design (the leaf class), not optimizer capacity, governs whether a black-box search recovers structured-control performance — consistent with the same finding on the lost-sales, one-warehouse multi-retailer, and multi-echelon environments elsewhere in this work. Upgrading the environment to literature-verified would require recovering the paper's exact order-up-to-to-inventory-position simulation convention and binding the echelon-position policy; we leave that to future work and do not present this row as a published-number benchmark.

Topology diversification. To check that the result is not an artefact of the serial chain, we re-ran the *same* recipe and protocol (a depth-2 oblique soft tree with linear leaves, identical population, generations, and held-out paired-common-random-number evaluation; no re-sweep) on two further topologies of the same supply-network model: a *mixed distribution-and-assembly network* (an external source feeds a single node that *distributes* to two intermediate nodes, whose output two downstream *assembly* nodes each combine to serve a customer) and a *pure assembly network* (a three-layer assembly tree in which four sources feed two intermediate assembly nodes that feed a final assembly node serving the customer). Both move the action space well beyond the serial chain: the distribution split and the assembly $\min(\cdot)$ production starve downstream stages, so the environment's own best pairwise base-stock needs much higher order-up-to levels (best-heuristic costs 297.69 and 283.34 per period, against 60.24 for the serial case). On the pure assembly network the five-seed audit finds *seed-noisy parity*: the learned tree averages 289.16 ± 14.20 against 283.34 (mean $+2.05\% \pm 5.01\%$ *above* the heuristic, two of five seeds below it). The single illustrative run in the table beats the heuristic by -2.98% (274.90 ± 0.17, $\approx 40\times$ the held-out standard error), but the across-seed scatter ($\approx 5\%$ of the mean) dwarfs that margin, so a fresh optimizer seed does not reliably reproduce it. On the mixed distribution-and-assembly network, the residual base-stock-backbone head of Section 12.2 (Eq. (19)) upgrades the row from a heuristic-match to a *robust beat over the best heuristic*. The earlier linear-leaf flow head could not reproduce the best heuristic on this topology, so its eight CMA seeds scattered to a seed-mean 306.10 ± 22.89 ($+2.82\%$ above the 297.69 best heuristic, only four of eight seeds below it) — parity, not a beat. With the

residual head, which anchors every seed at the best heuristic by construction, the structure found by a search over the decoder classes (a residual-base-stock head with linear leaves, oblique splits, and a per-relation geometry) improves the per-period cost to a seed-robust mean 291.136 ± 2.78 over five optimizer seeds (-2.20% against the 297.69 best heuristic; all five seeds below it, per-seed $287.36, 294.42, 290.74, 293.22, 289.93$). The seed scatter collapses from ± 22.89 to 2.78 ($\approx 0.95\%$ of the mean) precisely because the residual head reproduces the best heuristic exactly at generation 0. As for the serial case, these comparators are the *environment's own* grid-searched best heuristic on a faithful but not literature-verified environment — the source paper gives no analytical optimum for mixed or assembly networks (only random-search, DFO, Spearmint, and DNN benchmarks), and there is *no* published DRL/PPO baseline for any of these topologies — so these are research learned-versus-own-heuristic results across serial, mixed distribution-plus-assembly, and pure-assembly structures, and are *not* published-number beats.

Table 20: Production/assembly/distribution network (serial case 3): best learned soft tree vs. the environment's *own* best pairwise base-stock, common-random-number evaluation (4000 held-out paths). Costs are undiscounted per-period averages (lower is better). $\Delta\%$ is the learned policy's improvement over the best heuristic; "SEM mult." is that gain divided by the held-out standard error. The best heuristic (60.24) is the environment's own grid-searched heuristic, *not* a published optimum — this is a research comparison, not a literature-number beat.

Policy / config	Held-out cost ($\pm$ SEM)	$\Delta\%$ vs best heuristic	SEM mult.
Best pairwise base-stock (order-up-to $[8, 7, 9]$)	60.24	—	—
Flow warm-start (generation 0)	70.85 ± 0.61	$+17.6\%$	—
Learned soft tree, depth 2 (seed mean, $N = 5$)	$58.90 \pm 1.78_{\text{seed}}$	$-2.22\% \pm 2.96\%$	parity (4/5 below)
single seed, run 1 (illustrative)	57.25 ± 0.22	-4.96%	$\approx 14\times$
single seed, run 2 (illustrative)	54.96 ± 0.23	-8.77%	$\approx 23\times$
depth 3, single seed (illustrative)	57.85 ± 0.25	-3.97%	$\approx 10\times$
Mixed distribution-and-assembly network (best heuristic 297.69, echelon order-up-to $[36, 13, 7]$)			
Best pairwise base-stock	297.69 ± 0.67	—	—
Flow-head linear leaf (seed mean, $N = 8$)	$306.10 \pm 22.89_{\text{seed}}$	$+2.82\%$	heuristic-match
Residual-head soft tree (seed mean, $N = 5$)	$\mathbf{291.136 \pm 2.78}_{\text{seed}}$	$\mathbf{-2.20}\%$	robust beat
Pure assembly network (best heuristic 283.34, echelon order-up-to $[52, 52, 52]$)			
Best pairwise base-stock	283.34 ± 0.20	—	—
Learned soft tree, depth 2 (seed mean, $N = 5$)	$289.16 \pm 14.20_{\text{seed}}$	$+2.05\% \pm 5.01\%$	parity (2/5 below)
single seed (illustrative)	274.90 ± 0.17	-2.98%	$\approx 40\times$

The comparator is the environment's own best heuristic on a faithful but not literature-verified environment; the result is a research learned-versus-own-heuristic comparison, not a published-number beat (this environment has no published DRL/PPO baseline at all). All headline rows are now five-optimizer-seed aggregates (N=5, seeds run under the shared seed-robust protocol); the indented single-seed rows are retained as illustrative individual runs only. The serial case-3 and pure-assembly seed means are *parity* verdicts: their across-seed dispersion is of the order of (or larger than) the mean margin, so the earlier single-run beats do not replicate as seed-robust. The flow-head row is seed-noisy parity (seed mean above the best heuristic, 4/8 seeds below it, scatter ± 22.89); the residual base-stock-backbone head of Eq. (19) anchors every seed at the best heuristic by construction, collapsing the scatter to ± 2.78 and turning the row into a robust beat over the best heuristic (5/5 seeds below the best heuristic, mean + std $= 293.92 < 297.69$). Seed dispersions are sample ($n-1$) standard deviations throughout. The serial optimum 47.65 (Section 9) is structurally unreachable by this environment's local pairwise policy and is not used as a target here.

13 Discussion and managerial insights

Synthesizing across the ten problems, four themes emerge, and we close each by positioning it against the deep reinforcement learning (DRL) and evolution-strategies literatures.

A single generic recipe is portable. The same CMA-ES loop, unchanged, learns a scalar lost-sales order, a setup-cost order/no-order decision, a structured dual-sourcing order pair, a pair of echelon order-up-to levels, a per-node backorder target vector, and a price-reactive purchase quantity. For a practitioner this means one method to maintain rather than a bespoke heuristic per problem, and retraining when costs or demand change is a matter of rerunning a short simulation rather than re-deriving structure. This portability is the property the DRL roadmap of Boute et al. [3] asks of a learned controller, and the cross-problem A3C study of Gijsbrechts et al. [11] first demonstrated for a neural learner. Relative to that line, our contribution is not a new champion algorithm but a different point on the cost–complexity frontier: a gradient-free optimizer over a compact policy reaches comparable territory without value-function bootstrapping, reward shaping, or an exploration schedule, and so transfers across structurally distinct problems with no per-problem hyperparameter tuning. This is the inventory-control counterpart of the broader finding that black-box search over policy parameters is a scalable alternative to gradient-based DRL [7], and that even linear policies found by simple random search can be competitive on hard control benchmarks [29]. Within operations, evolutionary search has chiefly been used to tune the parameters of a *fixed* heuristic; learning the ordering policy itself with evolution strategies, and doing so across many inventory families, is the gap we address.

Action geometry is where the leverage lives. Across every problem the backbone width mattered far less than the decoder. On vanilla lost sales a small tree or linear ordinal head suffices; under a fixed cost the gated decoders that separate *whether* from *how much* are what avoid the never-order collapse; in dual sourcing the capped-dual-index coordinates are what let the policy reach the near-optimal region a raw-quantity decoder cannot express; and in multi-echelon the direct-level design is the difference between a $\approx$ 12–15% improvement and a policy more than 200% worse. The managerial reading is that choosing the control representation — ideally the coordinate system of the best known heuristic — is a more reliable lever than enlarging the network. This is, we believe, the most transferable finding in the paper, and it is precisely the lever the most recent DRL literature has converged on from the opposite direction: rather than freeing a large network to discover structure, that work *injects* inventory structure into the learner. Maggiar et al. [27] build structure-informed policy networks that embed monotonicity and other analytically known properties of optimal policies, and report that a generic deep RL controller naturally recovers many such structural properties; Madeka et al. [42] learn a generic deep RL controller for a large-scale periodic-review system by turning historical data into a simulator; Temizöz et al. [26] report that a model-based controlled-learning scheme reaches optimality gaps of at most 0.2% on lost-sales and perishable problems, far inside earlier A3C gaps; and recent graph- and heuristic-parameterized agents make the parameters of a classical control the action space itself — the inventory specialization of a broader move toward shaping and representing the action space rather than only the network [41, 38]. Our policy-owned decoder is the same idea expressed as a design choice rather than a regularizer: we put the search directly in the heuristic's coordinate system and let a compact model modulate it. The convergence of independent lines on "structure in the policy, not capacity in the network" is, in our reading, the clearest signal of where the field is heading.

Compact, interpretable, retrainable policies are a practical asset. The learned policies carry tens to a few hundred parameters — orders of magnitude fewer than published DRL networks, which the roadmap and subsequent studies note are large, tuning-heavy, and hard to interpret [3, 11, 15]. They reduce at run time to a small matrix multiply or a shallow tree traversal, so they can be deployed in standard enterprise or spreadsheet tooling and audited by non-specialists; where a structured heuristic underlies the decoder (dual sourcing, serial and divergent multi-echelon, general-network backorder), the learned outputs read directly as order-up-to levels and caps. Interpretability is now an explicit objective in inventory RL rather than an afterthought, and small or structurally constrained policies are increasingly favored for exactly the audit-and-deploy reasons above. Compactness also carries a statistical dividend that matters for practice: structured inventory policy classes provably generalize from limited data, with generalization error largely independent of horizon for base-stock classes and only logarithmic for (s, S) classes [28]. A policy with hundreds of parameters living in a heuristic's coordinate system is squarely the kind of low-complexity class for which such guarantees apply, which is reassuring when the training signal is a finite simulation rather than an asymptotic average.

Honest match/beat/bound accounting is itself a contribution. A recurring concern in this literature is that DRL inventory results are difficult to reproduce and compare: cost conventions, demand processes, and baselines differ across papers, and there is no ImageNet-style standard benchmark, so reported "wins" are not always like-for-like [3, 25]. We have tried to answer this concern at the level of *discipline* rather than scale. Every environment is anchored to a published number before it carries any learned-policy claim (Appendix A), and every result is reported under one of three honest verdicts. We *beat* a comparator only when it is a heuristic and the margin clears the held-out standard error under common random numbers (general-network backorder by over 20%; ameliorating inventory's order-up-to heuristic by a wide margin; one-warehouse multi-retailer on the seed-robust asymmetric three-retailer, high-CV ten-retailer, and strongly heterogeneous ten-retailer best-heuristic rows; the production/assembly/distribution network's own *mixed-topology* pairwise base-stock heuristic — its serial and pure-assembly rows fell back to parity under the five-seed audit and are no longer claimed as beats). We *match*, and say so explicitly, when the comparator is provably optimal or a near-optimal proxy: the dual-sourcing soft tree sits inside the capped-dual-index policy's own $\leq 0.11\%$ optimality band — comfortably inside the band the published A3C learner does not reach — and the serial soft tree recovers the Clark–Scarf optimum to $+0.011\%$; neither is framed as a win. We also report *heuristic-matches* when seed variation does not support a learned beat, as on the one-warehouse multi-retailer backorder and lost-sales regime rows. And we report a *gap to a bound* without ever calling it "beaten": the ameliorating-inventory gap to the perfect-information LP is structural (a single scalar purchase lever against the bound's full three-part hindsight issuance) and is not comparable to the $\sim 3.5\%$ deep-RL gap of Pahr and Grunow [24], who act in the full action space. Two comparisons we flag as indicative rather than strictly like-for-like — the negligible sub-band dual-sourcing margins, and the multi-echelon improvement computed against a different baseline and cost convention than the published A3C figures. This accounting is deliberately conservative, and it is where we believe a portable ES study can be most useful to a field with a reproducibility problem: not by topping a leaderboard, but by stating exactly what kind of claim each number supports. Operationally, the next artifact should be a versioned baseline ledger behind the paper tables: for each problem family it records the published instance specification, demand process, cost convention, lead-time or network structure, strongest in-environment heuristic, exact optimum or bound where available, trained-policy performance, paired-CRN standard error, and one of the verdict types above. That

ledger is the practical route toward an ImageNet-like inventory-management benchmark: future learners should be scored against clear, reproducible, like-for-like baselines rather than against shifting paper tables.

14 Conclusion, limitations, and future work

We have shown that evolution strategies — specifically CMA-ES over compact, policy-owned decoders — learn inventory control policies that are competitive across ten inventory problems: lost sales, fixed-cost lost sales, dual sourcing, divergent and serial multi-echelon, general-network backorder, perishable inventory, one-warehouse multi-retailer distribution, ameliorating inventory, and a production/assembly/distribution network. Relative to DRL, CMA-ES needs few hyperparameters, supplies its own exploration, and optimizes the realized cost directly, so the same loop transfers across problems; the resulting policies are orders of magnitude smaller than published DRL networks while remaining interpretable and verifiable. The consistent lever is action geometry: with the decoder shaped like the relevant heuristic's coordinate system, a few-hundred-parameter policy matches proven optima where they exist, beats strong heuristics where the comparator is a heuristic, and clears published A3C on dual sourcing.

Limitations. Several caveats bound these claims and we state them plainly. (i) Two comparisons are indicative rather than strictly like-for-like: the dual-sourcing margins below the capped-dual-index proxy are negligible and sit inside the heuristic's own optimality band, so the honest reading is a *match*, not a win; and the multi-echelon improvement is computed against a different baseline and cost convention than the published A3C figures. (ii) The high-penalty, autocorrelated MMPP lost-sales instances at the longest lead times ($L = 8, 10$) remain won by classical heuristics, indicating a real gap for stationary compact policies under regime-switching demand and deep pipelines. (iii) We do not beat the strongest published PPO on the one-warehouse multi-retailer instances in a robust, like-for-like way: the asymmetric three-retailer seed block straddles the published PPO scalar, the high-CV ten-retailer seed-mean is $\approx 7\%$ above PPO (0/5 seeds below), and the strongly heterogeneous ten-retailer seed-mean is $\approx 3\%$ above PPO; our wins on that problem (asymmetric K=3, high-CV K=10, and the strongly heterogeneous K=10 at $+12.44\%$ over five seeds) are over the best tuned same-protocol heuristic, not over PPO — which is cross-protocol context only. (iv) The ameliorating-inventory gap is to a perfect-information *upper bound* and is structural; it is reported, never beaten. (v) The production/assembly/distribution network result is a research result on a faithful but *not* literature-anchored environment: the learned policy robustly beats the environment's own best heuristic on the *mixed* topology only (upgraded from a heuristic-match to a seed-robust -2.20% beat by the residual base-stock-backbone head), the serial and pure-assembly topologies being five-seed parity, and the environment has no published DRL/PPO baseline, so none of these is a published-cost beat. (vi) Methodologically, CMA-ES — though far more sample-efficient than A3C here — is still data-hungry relative to settings with scarce historical demand, and its population-based evaluation cost grows with the covariance dimension, so very large parameterizations may need restricted, diagonal, or separable covariance structures.

Where the field is going, and how this fits. Three currents in recent inventory RL frame our future work. First, the field is moving toward *reproducible, optimum-anchored benchmarks*: open environments and comparisons to known or bounded optima rather than to other learners [25, 26]. Our per-environment literature validation and three-verdict accounting are a step in that direction, and contributing our ten validated environments and policy forwards to such a shared

suite is the natural next move. Second, the most effective recent learners *put inventory structure inside the policy* — structure-informed networks, base-stock-regularized objectives, and heuristic-parameterized action spaces [27, 26]. Our action-geometry result is the same principle reached gradient-free; the obvious synthesis is to hand a sample-efficient gradient learner the same policy-owned decoders, using CMA-ES (or warm-started from a heuristic, as we do for dual sourcing, serial multi-echelon, and general-network backorder) to seed and the gradient learner to refine — which should directly attack the long-lead MMPP and the below-PPO gaps. Third, the frontier is turning to *generalist and limited-data* settings: generally-capable agents trained for zero-shot transfer across instances, then paired with parameter estimation at deployment [31, 30], and the favorable sample-complexity of structured policy classes [28] suggests these compact decoders are exactly the right hypothesis class for the data-scarce regime. Concrete next steps, in order of expected payoff, are: richer state-dependent decoders for correlated and non-stationary demand; warm-starting from heuristics in every problem where a structured control exists; scaling CMA-ES via separable or restricted covariance so larger networked systems stay tractable; and amortizing one decoder across an instance family so that a single trained policy generalizes with estimate-then-decide adaptation rather than per-instance retraining. The through-line is unchanged from the body of the paper: in inventory control, the representation of the *action* is at least as consequential as the choice of *learner*, and a compact, honest, reproducible policy is often enough.

References

[1] M. Bijvank, S. Bhulai, and W. T. Huh. Parametric replenishment policies for inventory systems with lost sales and fixed order cost. *European Journal of Operational Research*, 241(2):381–390, 2015.

[2] M. Bijvank and I. F. A. Vis. Lost-sales inventory theory: A review. *European Journal of Operational Research*, 215(1):1–13, 2011.

[3] R. N. Boute, J. Gijsbrechts, W. van Jaarsveld, and N. Vanvuchelen. Deep reinforcement learning for inventory control: A roadmap. *European Journal of Operational Research*, 298(2):401–412, 2022.

[4] N. Hansen and A. Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2):159–195, 2001.

[5] N. Hansen, Y. Akimoto, and P. Baudis. CMA-ES/pycma on GitHub. Zenodo, DOI:10.5281/zenodo.2559634, 2019.

[6] G. Klambauer, T. Unterthiner, A. Mayr, and S. Hochreiter. Self-normalizing neural networks. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[7] T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017.

[8] P. Zipkin. Old and new methods for lost-sales inventory systems. *Operations Research*, 56(5):1256–1263, 2008.

[9] L. Xin. Technical Note—Understanding the performance of capped base-stock policies in lost-sales inventory models. *Operations Research*, 69(1):61–70, 2021.

[10] B. Van Roy, D. P. Bertsekas, Y. Lee, and J. N. Tsitsiklis. A neuro-dynamic programming approach to retailer inventory management. In *Proceedings of the 36th IEEE Conference on Decision and Control*, volume 4, pages 4052–4057, 1997.

[11] J. Gijsbrechts, R. N. Boute, J. A. Van Mieghem, and D. J. Zhang. Can deep reinforcement learning improve inventory management? Performance on lost sales, dual-sourcing, and multi-echelon problems. *Manufacturing & Service Operations Management*, 24(3):1349–1368, 2022.

[12] S. Nahmias and S. A. Smith. Optimizing inventory levels in a two-echelon retailer system with partial lost sales. *Management Science*, 40(5):582–596, 1994.

[13] A. Federgruen and P. Zipkin. Approximations of dynamic, multilocation production and inventory problems. *Management Science*, 30(1):69–84, 1984.

[14] T. de Kok, C. Grob, M. Laumanns, S. Minner, J. Rambau, and K. Schade. A typology and literature review on stochastic multi-echelon inventory models. *European Journal of Operational Research*, 269(3):955–983, 2018.

[15] I. Kaynov, M. van Knippenberg, V. Menkovski, A. van Breemen, and W. van Jaarsveld. Deep reinforcement learning for one-warehouse multi-retailer inventory management. *International Journal of Production Economics*, 267:109088, 2024.

[16] S. Veeraraghavan and A. Scheller-Wolf. Now or later: A simple policy for effective dual sourcing in capacitated systems. *Operations Research*, 56(4):850–864, 2008.

[17] A. Sheopuri, G. Janakiraman, and S. Seshadri. New policies for the stochastic inventory control problem with two supply sources. *Operations Research*, 58(3):734–745, 2010.

[18] K. Geevers, L. van Hezewijk, and M. Mes. Multi-echelon inventory optimization using deep reinforcement learning. *Central European Journal of Operations Research*, 32(3):653–683, 2024.

[19] S. Kunnumkal and H. Topaloglu. Linear programming based decomposition methods for inventory distribution systems. *European Journal of Operational Research*, 211(2):282–297, 2011.

[20] L. V. Snyder and Z.-J. M. Shen. *Fundamentals of Supply Chain Theory*. Wiley, 2nd edition, 2019.

[21] M. Pirhooshyaran and L. V. Snyder. Simultaneous decision making for stochastic multi-echelon inventory optimization with deep neural networks as decision makers. *arXiv preprint arXiv:2006.05608*, 2021.

[22] B. J. De Moor, J. Gijsbrechts, and R. N. Boute. Reward shaping to improve the performance of deep reinforcement learning in perishable inventory management. *European Journal of Operational Research*, 301(2):535–545, 2022.

[23] J. Farrington, W. K. Wong, K. Li, and M. Utley. Going faster to see further: GPU-accelerated value iteration and simulation for perishable inventory control using JAX. *Annals of Operations Research*, 349(3):1609–1638, 2025.

[24] M. Pahr and M. Grunow. The value of blending—Managing ameliorating inventory using deep reinforcement learning. *Production and Operations Management*, 35(5), 2025.

[25] M. Alvo, D. Russo, Y. Kanoria, and M. Lee. Deep reinforcement learning for inventory networks: Toward reliable policy optimization. *arXiv preprint arXiv:2306.11246*, 2023 (revised 2025).

[26] T. Temizöz, C. Imdahl, R. Dijkman, D. Lamghari-Idrissi, and W. van Jaarsveld. Deep controlled learning for inventory control. *European Journal of Operational Research*, 324(1):104–117, 2025.

[27] A. Maggiar, S. Andaz, A. Bagaria, C. Eisenach, D. Foster, O. Gottesman, and D. Perrault-Joncas. Structure-informed deep reinforcement learning for inventory management. *arXiv preprint arXiv:2507.22040*, 2025.

[28] Y. Xie, W. Ma, and L. Xin. VC theory for inventory policies. *arXiv preprint arXiv:2404.11509*, 2024.

[29] H. Mania, A. Guy, and B. Recht. Simple random search provides a competitive approach to reinforcement learning. *arXiv preprint arXiv:1803.07055*, 2018.

[30] N. Vanvuchelen, J. Gijsbrechts, and R. Boute. Use of proximal policy optimization for the joint replenishment problem. *Computers in Industry*, 119:103239, 2020.

[31] T. Temizöz, C. Imdahl, R. Dijkman, D. Lamghari-Idrissi, and W. van Jaarsveld. Zero-shot generalization in inventory management: Train, then estimate and decide. *arXiv preprint arXiv:2411.00515*, 2024.

[32] K. J. Arrow, T. Harris, and J. Marschak. Optimal inventory policy. *Econometrica*, 19(3):250–272, 1951.

[33] H. Scarf. The optimality of (s, S) policies in the dynamic inventory problem. In K. J. Arrow, S. Karlin, and P. Suppes, editors, *Mathematical Methods in the Social Sciences*. Stanford University Press, 1960.

[34] A. J. Clark and H. Scarf. Optimal policies for a multi-echelon inventory problem. *Management Science*, 6(4):475–490, 1960.

[35] L. B. Schwarz. A simple continuous review deterministic one-warehouse N-retailer inventory problem. *Management Science*, 19(5):555–566, 1973.

[36] J. Sun and J. A. Van Mieghem. Robust dual sourcing inventory management: Optimality of capped dual index policies and smoothing. *Manufacturing & Service Operations Management*, 21(4):912–931, 2019.

[37] W. B. Powell. *Approximate Dynamic Programming: Solving the Curses of Dimensionality*. Wiley, 2nd edition, 2011.

[38] Y. Chandak, G. Theocharous, J. Kostas, S. Jordan, and P. Thomas. Learning action representations for reinforcement learning. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *PMLR*, pages 941–950, 2019.

[39] G. Dulac-Arnold, R. Evans, H. van Hasselt, P. Sunehag, T. Lillicrap, J. Hunt, T. Mann, T. Weber, T. Degris, and B. Coppin. Deep reinforcement learning in large discrete action spaces. *arXiv preprint arXiv:1512.07679*, 2015.

[40] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine. Residual reinforcement learning for robot control. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 6023–6029, 2019.

[41] A. Kanervisto, C. Scheller, and V. Hautamäki. Action space shaping in deep reinforcement learning. In *2020 IEEE Conference on Games (CoG)*, pages 479–486, 2020.

[42] D. Madeka, K. Torkkola, C. Eisenach, A. Luo, D. P. Foster, and S. M. Kakade. Deep inventory management. *arXiv preprint arXiv:2210.03137*, 2022.

A Environment validation against the published literature

Every environment used in this paper is anchored to the published literature before it supports any learned-policy claim. For each problem we reproduce a known reference cost — an exact optimum, an exact dynamic-programming cost, or a published heuristic / constant-base-stock cost — under the same evaluation protocol as the main text, and report our reproduced value alongside the published one with the relative gap. The remainder of this appendix carries out this check problem by problem; where a residual gap remains (most notably one multi-echelon case study below, reproduced to within 1.31%), we state it explicitly and treat the reproduction as an implementation-fidelity check rather than a re-derivation of the published result. The per-problem subsections that follow validate the seven problems whose comparator is a published reproduction-able quantity (vanilla lost sales, fixed-cost lost sales, multi-echelon Van Roy / dual sourcing, perishable inventory, general-network backorder, and serial multi-echelon), and then the production/assembly/distribution network, whose only published anchor is the single-node newsvendor cost: it is a faithful but *not* literature-verified environment at the network level, so its validation is a single-node fidelity check rather than a published-number reproduction of the Section 12 result (Section A.8). The two remaining studied problems are anchored differently and therefore do not appear as published-number rows here: the one-warehouse multi-retailer (Section A.9) and ameliorating-inventory (Section A.10) results are reported against a best tuned in-environment heuristic (with, for ameliorating inventory, a perfect-information upper bound reported strictly as a bound); their anchoring subsections close this appendix.

A.1 Published vanilla lost-sales validation

Before using the vanilla lost-sales simulator and heuristic stack for learned-policy comparisons, we validate the canonical Poisson instance against the published/reference costs used in the lost-sales literature [8, 9]. The validation instance has $h = 1$, $p = 4$, $L = 4$, and $D_t \sim \text{Poisson}(5)$. Table 21 reports the reference costs and an independent rollout reproduction for the heuristic policies (horizon 10^5). This is an implementation-fidelity check, not a learned-policy result.

Table 21: Published/reference vanilla lost-sales validation example for the canonical Poisson-demand instance. The reproduced heuristic costs use the same long-horizon rollout protocol as Table 2.

Quantity	Published/reference cost	Reproduced cost
optimal	4.73	4.73 (catalog reference)
myopic-1	5.06	5.0569
myopic-2	4.82	4.8208
SVBS	5.83	5.8153

The heuristic costs match the published/reference values at the precision used in the benchmark tables, anchoring the vanilla lost-sales environment and the myopic/SVBS heuristic implementation. The MMPP2 rows in Table 2 are mean-preserving extensions of this surface and are not claimed as published literature rows.

A.2 Published fixed-cost lost-sales validation

Before relying on the fixed-cost simulator and heuristic stack for learned-policy comparisons, we validate the implementation against the published example of Bijvank et al. [1]. Their Table 1 re-

ports a small Poisson-demand instance with $L = 2$, $p = 14$, $K = 5$, $h = 1$, $\mu = 5$. The reproduction uses our exact average-cost value-iteration solver and exact evaluators for the published heuristic parameters.

Table 22: Published fixed-cost lost-sales validation example from Bijvank et al. [1] and our reproduced exact average costs for the reported heuristic parameters.

Policy	Reported parameters	Published cost	Our reproduced cost
optimal	—	11.46	11.4631
(s, S)	$(17, 23)$	11.62	11.6181
(s, nQ)	$(17, 7)$	11.56	11.5552
modified (s, S, q)	$(17, 23, 7)$	11.50	11.4974

The reproduced exact costs match the published values to the precision of the reported table, anchoring the fixed-cost environment and heuristic implementation to a published benchmark row.

A.3 Published multi-echelon (Van Roy) validation

Before using the multi-echelon environment for learned-policy comparison, we validate it against the published *constant base-stock* costs of Van Roy et al. [10] (the simple problem and the two complex case studies), evaluated at the published (y^w, y^r) levels under the original Van Roy et al. (1997) cost convention [10], with 100 replications of horizon 10^5, no warm-up discard, and the minimum-spread allocation. Table 23 shows that all three reproduce within the 2% simulation-protocol tolerance (maximum absolute gap 1.31%).

Table 23: Reproduction of the published Van Roy constant base-stock costs.

Instance	Levels (y^w, y^r)	Published	Reproduced	Gap
Simple problem	$(10, 16)$	51.7	51.72	$+0.03\%$
Case study 1	$(330, 23)$	1302	1284.9	-1.31%
Case study 2	$(460, 22)$	1449	1437.6	-0.79%

Two caveats keep this an implementation-fidelity check rather than a full literature verification. First, exact reproduction requires calibrated demand inputs: the simple problem uses a latent mean of 6.294 (the effective mean of Van Roy's $\mathcal{N}(5, 8^2)$ after rounding and clipping to non-negative integers), and case study 2 a latent mean of 1.0 rather than the stated 0. Second, we reproduce only the constant base-stock rows, not the A3C learner. The residual $\sim 1.3\%$ is attributed to unspecified details of Van Roy's original simulation protocol. The paper-faithful settings used for policy design (Table 6) instead use the Gijsbrechts et al. (2022) cost convention and the Table-3 demand means.

A.4 Dual-sourcing comparator and the bounded-DP optimum

For dual sourcing we benchmark against the capped dual-index heuristic as the optimal proxy rather than a computed optimum. An exhaustive grid search over the capped-dual-index control $(s_e, \Delta_r, \bar{c}_r)$ under the evaluation protocol recovers the published capped-dual-index parameters and confirms that it is the best static control on every Figure 9 row, consistent with its published $\leq 0.11\%$ optimality gap. We do *not* use our own bounded-DP value iteration as the denominator:

at inventory bounds that keep the $l_r = 4$ state space tractable, the truncated bounded-DP cost is mildly inconsistent — it falls slightly below the simulated heuristic costs — and widening the bounds to remove the truncation is computationally prohibitive at $l_r = 4$. The capped dual-index proxy is near-exact to within its published gap and is evaluated on the identical common-random-number rollout as the learned policies.

Robustness of CDI's near-optimality (regime sweep). The proxy choice is further justified by a direct regime sweep at $l_r = 2$, where the bounded-DP value iteration *can* be validated by widening the inventory box until the average cost plateaus. Across the expedited premium $c_e - c_r \in \{10, 30, 60, 100\}$, penalty-to-holding ratios up to $b/h = 40$, demand variability (uniform supports $U\{0, \ldots, 4\}$ and $U\{0, \ldots, 8\}$), and expedited capacities from 1 (severely constrained) to 12 (effectively uncapacitated), the capped dual-index cost stays within 0.4% of the *validated* bounded-DP optimum on every cell (maximum gap $\approx 0.31\%$ on a fixed path, $\approx 0.16\%$ out-of-sample — below the path-to-path sampling noise). CDI's near-optimality is thus a structural property of this single-item dual-sourcing model, not a feature of the six benchmark rows; this is why we treat dual sourcing as a *heuristic-near-optimal anchor* and report a match to CDI as the intended outcome. A learned soft tree trained directly on the highest-variability cell still does not beat CDI (a seed-robust loss of +0.6%), consistent with there being essentially no optimality gap left to close.

Table 24 makes the proxy's fidelity explicit: the environment reproduces the published Figure 9 capped-dual-index optimality gaps within 0.01 percentage points on all six instances, well inside the heuristic's own optimality band.

Table 24: Dual-sourcing Figure 9 reproduction. The environment reproduces the published capped dual-index (CDI) optimality gaps of Gijsbrechts et al. [11] within 0.01 percentage points on all six instances ($l_e = 0$, $c_r = 100$, $h = 5$, $b = 495$, demand $U\{0, \ldots, 4\}$), validating the CDI control as the optimal proxy used in the dual-sourcing experiments; the published A3C gaps are listed for context. The $l_r = 2$ rows are inexpensive to reproduce; the $l_r = 3, 4$ rows use a more expensive bounded-DP solver but were confirmed within tolerance.

Instance (l_r, c_e)	Published CDI gap (%)	Reproduced CDI gap (%)	Δ (pp)	A3C gap (%)
$(2, 105)$	0.00	0.006	0.006	0.52
$(2, 110)$	0.03	0.030	0.000	0.80
$(3, 105)$	0.00	≤ 0.01	≤ 0.01	0.82
$(3, 110)$	0.06	≤ 0.01	≤ 0.01	0.51
$(4, 105)$	0.00	≤ 0.01	≤ 0.01	1.85
$(4, 110)$	0.11	≤ 0.01	≤ 0.01	1.33

A.5 Published perishable-inventory validation

Before relying on the perishable-inventory environment for learned-policy comparisons, we validate it against the value-iteration optimum of De Moor et al. [22], as re-derived by Farrington et al. [23] (their Table 3). The validation instance is the $m = 2$, $L = 1$, critical-period-7 slice under each issuance rule. Our exact-MDP value iteration reproduces the published discounted-return optimum and the published optimal-policy tables.

Table 25: Reproduction of the De Moor et al. [22] / Farrington et al. [23] value-iteration optimum (Table 3) by our exact tabular MDP, for the $m = 2$, $L = 1$ slice under FIFO and LIFO issuance.

Issuance rule	Published VI optimum	Reproduced (exact MDP)	Gap
FIFO	-1457	-1457.28	$+0.02\%$
LIFO	-1553	-1552.99	-0.00%

The exact MDP reproduces the published value-iteration optimum to the precision of the reported table. One estimator caveat applies: the Monte-Carlo best base-stock used in the main perishable results (Section 7) is computed under a sampled, finite-horizon estimator that sits $\sim 1\%$ below this analytic optimum on the same instance, so the analytic optimum is a context reference rather than an apples-to-apples comparator for the rolled-out heuristic and learned costs.

A.6 Published general-network backorder validation

Before using the general-network (CardBoard Company) backorder environment for learned-policy comparison, we validate it against the constant node-base-stock benchmark of Geevers et al. [18] (experiment set 1, the only one of its three sets verifiable against an open source). Evaluating the published base-stock levels $[82, 100, 64, 83, 35, 35, 35, 35, 35]$ in our environment reproduces the published mean cost within the simulation-protocol tolerance.

Table 26: Reproduction of the Geevers et al. [18] set-1 constant node-base-stock benchmark.

Instance	Published benchmark	Reproduced	Gap
Set 1 (order per stock point)	10,467	10,355	-1.1%

The residual -1.1% is the expected difference between our simulator and the thesis's RNG and warm-up window; the reproduction holds across replications and seeds with warehouse and retailer fill rates in the 98–99% band, so set 1 is treated as literature-verified. Sets 2 and 3 appear only in the gated journal full text, could not be confirmed against an open copy, and are not literature-verified; they are excluded from this validation.

We additionally validate the structurally different *general-network divergent* instance of Kunnumkal and Topaloglu [19], whose constant node-base-stock benchmark is reported in the open Geevers (2020) thesis (Ch. 5). Evaluating the published levels $[124, 30, 30, 30]$ in our environment reproduces the published cost 4,059 to within -3.0%. This reproduction is looser than the set-1 row above (-1.1%); the wider gap reflects the nonstationary resampled-mean demand ($\alpha \sim \text{Uniform}[5, 15]$ per period) and the divergent rationing convention, and we report it honestly as such. All three retailer fill rates land at $\approx 98.6\%$, so the instance is still treated as literature-verified for the purpose of the learned-policy comparison.

Table 27: Reproduction of the general-network *divergent* (19) constant node-base-stock benchmark, as reported in the open Geevers (2020) thesis (Ch. 5).

Instance	Published benchmark	Reproduced	Gap
Divergent K–T ($1\to1\to3$, $\alpha \sim \text{U}[5, 15]$)	4,059	3,931	-3.0%

A.7 Published serial multi-echelon (Clark–Scarf) validation

Before using the serial multi-echelon environment for learned-policy comparison, we validate it against the proven Clark–Scarf optimum of Snyder & Shen Example 6.1 [20] (a three-stage system, Normal$(5, 1)$ demand, echelon holding $[2, 2, 3]$, lead times $[2, 1, 1]$, stockout 37.12). Our exact recursive-newsvendor solver re-derives the optimum, and the environment simulation under the optimal echelon base-stock levels reproduces it; we confirm both.

Table 28: Reproduction of the Snyder & Shen Example 6.1 proven Clark–Scarf optimum [20], by our exact recursive-newsvendor solver and by the environment simulation under the optimal echelon base-stock policy.

Quantity	Published optimum	Reproduced	Gap
Exact recursive-newsvendor solver	47.65	47.6654	+0.03%
Environment simulation	47.65	47.64	−0.02%

Both the exact solver and the environment dynamics reproduce the published optimum within a tight band ($\lesssim 0.06\%$), so the serial environment is shown to reproduce the literature optimum under the known-optimal policy before any learned policy is trained on it.

A.8 Production / assembly / distribution network single-node fidelity check

The production/assembly/distribution network of Section 12 is anchored differently from the seven problems above, and we are explicit about the difference. The network-level model is *faithful* to the published MDP and cost of Pirhooshyaran and Snyder [21] — transition, cost accounting, and timing are reproduced equation-by-equation against the paper — but it is *not* literature-verified at the network level, because Pirhooshyaran and Snyder [21] report *no* published optimal or benchmark cost for the multi-node version of this MDP that we could reproduce. What we can validate against a published number is the environment's *single-node reduction*: the single-node newsvendor instance of the paper's Table 1 ($\mu = 100$, $\sigma = 10$, $h = 10$, $p = 30$, $L = 1$), for which the paper reports an analytical optimal cost of 127.11. Running the environment's own exact dynamic program on this reduction — simulating its step dynamics, not the closed-form newsvendor formula — reproduces 127.10, a relative gap of +0.008%, with the tiny residual attributable to the integer demand/order-up-to discretization (it shrinks as the demand scale grows). This certifies the environment's *dynamics* against a published quantity.

Table 29: Single-node fidelity check for the production/assembly/distribution network. The environment's exact dynamic program reproduces the published single-node newsvendor optimum of Pirhooshyaran and Snyder [21] (Table 1, $\mu = 100$, $\sigma = 10$, $h = 10$, $p = 30$, $L = 1$) by simulating its own step dynamics.

Quantity	Published optimum	Reproduced (environment DP)	Gap
Single-node newsvendor cost	127.11	127.10	+0.008%

Two points keep the network-level result honest, consistent with the scope stated in Section 12. First, the Section 12 learned-policy result is a *research* result, not a published-number beat: under the five-seed audit of Section 12.4, the learned soft tree is seed-noisy *parity* with the environment's *own* best grid-searched pairwise base-stock on the serial and pure-assembly topologies ($-2.22\% \pm$

2.96% and +2.05%±5.01% seed means; the earlier single-run beats do not replicate across optimizer seeds), and only on the mixed distribution-and-assembly topology does the residual base-stock-backbone head deliver a seed-robust −2.20% beat over that heuristic; in every case the comparator is the environment's own heuristic, not a published optimum. Second, the textbook serial optimum 47.65 (Section A.7) is *not* a valid target for this environment: it is an *echelon* base-stock cost, whereas this environment's pairwise policy targets the *local* raw-material position (and excludes finished goods), so 47.65 is structurally unreachable here and we do not use it as a comparator. The single-node reduction above is therefore the only published anchor we claim for this environment, and it holds.

A.9 One-warehouse multi-retailer anchoring

The one-warehouse multi-retailer environment (Section 10) is not anchored to a reproduced published cost, and we list no published-number row for it. The comparator scalars of Kaynov et al. [15] rest on a demand convention we could not verify and on unpublished PPO hyperparameters, so neither their environment nor their learner is reproducible from the paper; their PPO numbers are quoted in Section 10.4 strictly as cross-protocol context, never as a head-to-head beat. Verification here is instead internal to the environment: the learned policy is compared, under our own protocol, against the environment's *own* best tuned echelon base-stock with allocation heuristic, with the seed-robust improvements reported in Table 16; and we additionally train a faithful in-protocol PPO under our own convention as a corroborating baseline, which converges to the same heuristic basin. No published quantity is reproduced and none is claimed.

A.10 Ameliorating-inventory anchoring

The ameliorating-inventory environment (Section 11) likewise reproduces no published cost. Its like-for-like result is a beat over the environment's *own* best tuned order-up-to purchase heuristic (Table 19). The only absolute reference we report is a perfect-information steady-state linear-programming value, used *strictly as an upper bound*: it is computed by our own LP (spirits bound 1991.93, port-wine bound 2444.80), is not validated against any published number, and the large residual gap to it (79–94%) is *structural*, as the learned policy controls only a scalar purchase volume whereas the bound assumes the full three-part issuance and blending decision under perfect information. For the same reason the gap is not comparable to the ≈3.5% deep-reinforcement-learning gap of Pahr and Grunow [24], whose agent controls the full action space; we report the gap to the bound, never a beat against it.